

VIETNAM NATIONAL UNIVERSITY, HO CHI MINH CITY
UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



DATABASE MANAGEMENT SYSTEMS (CO3021)

Assignment

“Mentor Finder for Students”

Instructor: Lê Thị Bảo Thu, *CSE-HCMUT*

Semester: 252

Class: L01

Students: Phạm Tấn Đạt - 2310689 (*Team 15*)

Dương Gia Bảo - 2310207 (*Team 15*)

Đoàn Nguyễn Nhật Vy - 2313989 (*Team 15*)

Võ Nguyễn Tường Vi - 2313882 (*Team 15*)

Mục lục

Danh mục hình vẽ	ii
Danh mục bảng biểu	ii
Danh sách thành viên và nhiệm vụ	iii
1 Các buổi họp	1
1.1 Buổi họp 02-02-2026	1
1.2 Buổi họp 23-02-2026	1
1.3 Buổi họp 09-03-2026	2
1.4 Buổi họp 23-03-2026	2
1.5 Buổi họp 06-04-2026	3
1.6 Buổi họp 20-04-2026	3
2 Giới thiệu	5
3 Phân tích hệ thống	6
3.1 Thực hiện phân tích chung hệ thống liên quan	6
3.2 Yêu cầu hệ thống	8
3.2.1 Yêu cầu chức năng	8
3.2.2 Yêu cầu phi chức năng	9
3.2.3 Yêu cầu chức năng không tương tác	10
3.3 Kiến trúc hệ thống	11
3.3.1 Mô hình tổng quát	11
3.3.2 Mô tả hệ thống đề xuất	12
3.3.3 Mô tả các ràng buộc ngữ nghĩa	15
3.3.4 Thiết kế EERD	17
3.3.5 Ánh xạ lược đồ	18
4 So sánh MS SQL Server và Cassandra	19

4.1	Data storage & management	19
4.1.1	Khái niệm chung về data storage và data management	19
4.1.2	Đặc trưng cơ chế data storage và data management trong MS SQL Server	19
4.1.3	Đặc trưng cơ chế data storage và data management trong Cassandra	20
4.1.4	Kịch bản so sánh thực tế giữa SQL Server và Cassandra	21
4.1.4.1	Kịch bản 1: So sánh cơ chế sắp xếp dữ liệu	21
4.1.4.2	Kịch bản 2: So sánh hiệu suất INSERT hàng loạt	23
4.1.4.3	Kịch bản 3: So sánh kiến trúc lưu trữ vật lý	25
4.1.5	Đánh giá và quyết định	27
4.2	Indexing	28
4.2.1	Khái niệm chung về Indexing	28
4.2.2	Đặc trưng cơ chế Indexing trong MS SQL Server	28
4.2.2.1	Tổng quan các loại Index trong MS SQL Server	28
4.2.2.2	Đặc điểm cốt lõi của các cơ chế Indexing trong MS SQL Server	31
4.2.3	Đặc trưng cơ chế Indexing trong Apache Cassandra	31
4.2.3.1	Tổng quan các loại Index trong Apache Cassandra	32
4.2.3.2	Đặc điểm cốt lõi của các cơ chế Indexing trong Apache Cassandra	34
4.2.4	Kịch bản so sánh thực tế giữa SQL Server và Cassandra	34
4.2.4.1	Kịch bản 1: Truy xuất bằng (=) với khóa chính	34
4.2.4.2	Kịch bản 2: Truy xuất trên cột chưa đánh index	35
4.2.4.3	Kịch bản 3: Đánh chỉ mục phụ trên cột thường	36
4.2.4.4	Kịch bản 4: Truy vấn theo khoảng giá trị	38
4.2.4.5	Kịch bản 5: Truy vấn có Group by	39
4.2.4.6	Kịch bản 6: Truy vấn có Group by với ColumnStore Index trong SQL Server	40
4.2.5	Đánh giá và quyết định	42
4.3	Query processing	43

4.3.1	Khái niệm chung về xử lý truy vấn	43
4.3.2	Đặc trưng cơ chế xử lý truy vấn trong MS SQL Server	44
4.3.3	Đặc trưng cơ chế xử lý truy vấn trong Apache Cassandra	46
4.3.4	Kịch bản so sánh thực tế giữa SQL Server và Cassandra	48
4.3.4.1	Kịch bản 1: Hiệu ứng plan cache	48
4.3.4.2	Kịch bản 2: Pipeline biên dịch và thực thi truy vấn	50
4.3.4.3	Kịch bản 3: Tối ưu hóa truy vấn bằng lọc sớm dữ liệu	53
4.3.5	Đánh giá và quyết định	57
4.4	Data Backup and Recovery	58
4.4.1	Khái niệm chung về data backup và recovery	58
4.4.2	Đặc trưng cơ chế backup và recovery trong MS SQL Server	59
4.4.3	Đặc trưng cơ chế backup và recovery trong Apache Cassandra	59
4.4.4	Kịch bản so sánh thực tế giữa SQL Server và Cassandra	60
4.4.4.1	Kịch bản 1: Phục hồi sau thảm họa hệ thống	61
4.4.4.2	Kịch bản 2: Tính linh hoạt trong xử lý sự cố dữ liệu cục bộ	61
4.4.5	Đánh giá và quyết định	61
5	Hiện thực hệ thống	63
5.1	Giao diện dành cho Guest	63
5.1.1	Trang chủ (Landing Page)	63
5.1.2	Trang duyệt Mentor (Browse Mentor)	64
5.1.3	Trang hồ sơ Mentor	65
5.1.4	Trang đăng nhập	66
5.1.5	Trang đăng ký tài khoản Mentee	67
5.1.6	Trang đăng ký tài khoản Mentor	68
5.2	Giao diện dành cho Mentee	69
5.2.1	Trang đặt lịch với Mentor	69
5.2.2	Trang thanh toán qua Stripe	71
5.2.3	Dashboard quản lý của Mentee	72

5.3	Giao diện dành cho Mentor	73
5.3.1	Trang chỉnh sửa hồ sơ Mentor	73
5.3.2	Trang quản lý gói mentoring (My Plans)	75
5.3.3	Dashboard điều phối của Mentor	76
5.4	Giao diện dành cho Admin	77
5.4.1	Dashboard Overview	78
5.4.2	Mentor Statistics	79
5.4.3	Monthly Trends	80
5.4.4	Categories	81
5.4.5	User Management	82
5.4.6	Invoices & Revenue	83
6	Kết luận	84
	Tài liệu tham khảo	85

Danh mục hình vẽ

1	Giao diện trang chủ MentorCruise	6
2	Giao diện trang hồ sơ của Mentor	7
3	Giao diện trang quản lý (Dashboard)	7
4	Mô hình EERD của hệ thống	17
5	Lược đồ CSDL quan hệ ánh xạ từ EERD đã thiết kế	18
6	Kết quả truy vấn 10 dữ liệu đầu tiên của MS SQL Server	21
7	Kết quả truy vấn 10 dữ liệu đầu tiên của Apache Cassandra	22
8	Thời gian thực thi khi chèn 500.000 dòng vào bảng companies trên MS SQL Server	23
9	Thời gian thực thi khi chèn 500.000 dòng vào bảng companies trên Apache Cassandra	24
10	Cấu trúc file lưu trữ vật lý của SQL Server trong hệ thống Mentoria . . .	26
11	Thông tin node Cassandra và trạng thái bộ nhớ theo mô hình LSM-Tree	26
12	Kết quả thực thi Kịch bản 1 với SQL Server	35
13	Kết quả thực thi Kịch bản 1 với Cassandra	35
14	Kết quả thực thi Kịch bản 2 với SQL Server	36
15	Kết quả thực thi Kịch bản 2 với Cassandra	36
16	Kết quả thực thi Kịch bản 3 với SQL Server	37
17	Kết quả thực thi Kịch bản 3 với Cassandra	37
18	Kết quả thực thi Kịch bản 4 với SQL Server	38
19	Kết quả thực thi Kịch bản 4 với Cassandra	38
20	Kết quả thực thi Kịch bản 5 với SQL Server	40
21	Kết quả thực thi Kịch bản 5 với Cassandra	40
22	Kết quả thực thi Kịch bản 6 với SQL Server	41
23	Kết quả thực thi Kịch bản 1 với SQL Server	49
24	Kết quả thực thi Kịch bản 1 với Cassandra	50
25	Kết quả thực thi Kịch bản 2 với SQL Server	52
26	Kết quả thực thi Kịch bản 3 với SQL Server	55

27	Kết quả thực thi Kịch bản 3 với Cassandra	56
28	Giao diện trang chủ với trợ lý AI tích hợp	63
29	Giao diện duyệt và lọc danh sách mentor	64
30	Giao diện trang hồ sơ chi tiết của Mentor	65
31	Giao diện trang đăng nhập	66
32	Giao diện đăng ký tài khoản Mentee	67
33	Giao diện đăng ký tài khoản Mentor	68
34	Giao diện đặt lịch buổi tư vấn với Mentor	69
35	Giao diện thanh toán tích hợp với Stripe Checkout	71
36	Giao diện Dashboard quản lý phiên mentoring của Mentee	72
37	Giao diện chỉnh sửa hồ sơ chuyên môn của Mentor	73
38	Giao diện quản lý các gói mentoring	75
39	Giao diện Dashboard quản lý phiên và lịch khả dụng của Mentor	76
40	Giao diện tổng quan hệ thống	78
41	Giao diện thống kê hiệu suất Mentor	79
42	Giao diện theo dõi xu hướng theo tháng	80
43	Giao diện thống kê theo lĩnh vực chuyên môn	81
44	Giao diện quản lý người dùng	82
45	Giao diện theo dõi hóa đơn và doanh thu	83

Danh mục bảng biểu

1	Danh sách thành viên và nhiệm vụ	iii
2	So sánh cơ chế Backup và Recovery giữa hai DBMS	62

Danh sách thành viên và nhiệm vụ

STT	Họ và tên	MSSV	Nhiệm vụ	Hoàn thành
1	Phạm Tấn Đạt	2310689	- Hiện thực chủ đề Indexing - Thiết kế EERD - Code backend	100%
2	Dương Gia Bảo	2310207	- Hiện thực chủ đề Query processing - Tổng hợp báo cáo - Thiết kế EERD - Code backend	100%
3	Đoàn Nguyễn Nhật Vy	2313989	- Hiện thực chủ đề Data storage & management - Ảnh xạ lược đồ - Code frontend	100%
4	Võ Nguyễn Tường Vi	2313882	- Hiện thực chủ đề Data backup & recovery - Ảnh xạ lược đồ - Code frontend	100%

Bảng 1: Danh sách thành viên và nhiệm vụ

1 Các buổi họp

1.1 Buổi họp 02-02-2026

Thời gian 02/02/2026, 20:00 – 21:30

Hình thức Online (Google Meet)

Tham dự Phạm Tấn Đạt, Dương Gia Bảo, Đoàn Nguyễn Nhật Vy, Võ Nguyễn Tường Vi

Thư ký Phạm Tấn Đạt

Nội dung:

- Các thành viên tự giới thiệu, khảo sát kỹ năng: Đạt và Bảo quen code backend; Nhật Vy và Tường Vi thiên về frontend; cả nhóm đã có kinh nghiệm SQL Server từ môn học Cơ sở dữ liệu trước đó nhưng chưa có kinh nghiệm Cassandra.
- Đọc và phân tích đề bài; xác định 4 chủ đề chính tương ứng 4 thành viên.
- Thống nhất domain ứng dụng: **Mentoria** – nền tảng kết nối mentor-mentee.
- Phân công sơ bộ: Đạt – Indexing; Bảo – Query Processing; Nhật Vy – Data Storage & Management; Tường Vi – Data Backup & Recovery.

Việc cần làm: Cài đặt Cassandra; Đạt + Bảo phác thảo EERD sơ bộ.

1.2 Buổi họp 23-02-2026

Thời gian 23/02/2026, 20:00 – 21:30

Hình thức Online (Google Meet)

Tham dự Phạm Tấn Đạt, Dương Gia Bảo, Đoàn Nguyễn Nhật Vy, Võ Nguyễn Tường Vi

Thư ký Dương Gia Bảo

Nội dung:

- Khởi động lại sau kỳ nghỉ Tết; xác nhận tất cả đã cài đặt xong môi trường.

- Review EERD sơ bộ do Đạt + Bảo chuẩn bị; nhóm góp ý bổ sung thực thể.
- Chốt tech stack: backend Node.js + Express, frontend React + Tailwind CSS, triển khai database của hệ thống trên SQL Server.

Việc cần làm: Đạt + Bảo hoàn thiện EERD; Nhật Vy + Tường Vi thiết kế giao diện và ánh xạ lược đồ.

1.3 Buổi họp 09-03-2026

Thời gian 09/03/2026, 20:00 – 21:00

Hình thức Online (Google Meet)

Tham dự Phạm Tấn Đạt, Dương Gia Bảo, Đoàn Nguyễn Nhật Vy, Võ Nguyễn Tường Vi

Thư ký Đoàn Nguyễn Nhật Vy

Nội dung:

- Review và chốt EERD, lược đồ hoàn chỉnh.
- Nhật Vy và Tường Vi trình bày giao diện hệ thống.

Việc cần làm: Bảo sinh dữ liệu mẫu và setup project trên Github; các thành viên còn lại bắt đầu triển khai chủ đề của mình.

1.4 Buổi họp 23-03-2026

Thời gian 23/03/2026, 20:00 – 21:30

Hình thức Online (Google Meet)

Tham dự Phạm Tấn Đạt, Dương Gia Bảo, Đoàn Nguyễn Nhật Vy, Võ Nguyễn Tường Vi

Thư ký Võ Nguyễn Tường Vi

Nội dung:

- Báo cáo tiến độ của các chủ đề.
- Chốt các functional requirements cho ứng dụng.

- Phân công ứng dụng: Bảo + Đạt – API backend; Nhật Vy + Tường Vi – UI React.

Việc cần làm: Hoàn thiện các chủ đề lý thuyết; song song bắt đầu phát triển ứng dụng.

1.5 Buổi họp 06-04-2026

Thời gian 06/04/2026, 20:00 – 21:30

Hình thức Online (Google Meet)

Tham dự Phạm Tấn Đạt, Dương Gia Bảo, Đoàn Nguyễn Nhật Vy, Võ Nguyễn Tường Vi

Thư ký Phạm Tấn Đạt

Nội dung:

- Demo ứng dụng: các luồng tìm kiếm mentor, đặt lịch, đánh giá hoạt động ổn; ghi nhận lỗi cần sửa (loading state, validate conflict lịch).
- Kiểm tra đã đủ 8 loại query-update theo đề bài: đã đủ INSERT, DELETE, UPDATE, query đơn/đa điều kiện, JOIN, subquery, aggregate.
- Review bản báo cáo: cần bổ sung số liệu đo thời gian, ảnh chụp màn hình kết quả, kết luận gắn với domain Mentoria.

Việc cần làm: Mỗi thành viên hoàn thiện chương báo cáo của mình; Bảo tổng hợp toàn bộ báo cáo.

1.6 Buổi họp 20-04-2026

Thời gian 20/04/2026, 20:00 – 21:30

Hình thức Online (Google Meet)

Tham dự Phạm Tấn Đạt, Dương Gia Bảo, Đoàn Nguyễn Nhật Vy, Võ Nguyễn Tường Vi

Thư ký Dương Gia Bảo

Nội dung:

- Chạy thử toàn bộ luồng ứng dụng: hoạt động ổn định, các lỗi buổi trước đã được sửa.
- Review báo cáo: kiểm tra chính tả, số liệu nhất quán, đủ các mục theo rubric.
- Phân công thuyết trình: Đạt – Indexing; Bảo – Query Processing + giới thiệu dự án + demo ứng dụng; Nhật Vy – Data Storage & Management; Tường Vi – Backup & Recovery.

Việc cần làm: Cả nhóm bắt đầu chuẩn bị slide thuyết trình, tiếp tục hoàn thiện ứng dụng và chỉnh sửa lại báo cáo cho đầy đủ hơn.

2 Giới thiệu

Trong bối cảnh nền kinh tế tri thức phát triển nhanh chóng, nhu cầu tìm kiếm sự cố vấn chuyên môn (mentorship) ngày càng trở nên phổ biến trong giới sinh viên và người đi làm. Tuy nhiên, quá trình kết nối giữa người học (mentee) và người hướng dẫn (mentor) hiện nay vẫn còn nhiều hạn chế: thiếu công cụ tìm kiếm chuyên biệt, khó đánh giá năng lực mentor, không có hệ thống quản lý lịch hẹn hay theo dõi tiến trình học tập một cách tập trung.

Xuất phát từ thực tế đó, nhóm đã xây dựng Mentoria – một nền tảng web kết nối mentor và mentee, cho phép tìm kiếm và đặt lịch tư vấn theo nhiều tiêu chí như kỹ năng, lĩnh vực chuyên môn, ngôn ngữ, quốc gia và mức giá. Hệ thống cung cấp đầy đủ các luồng nghiệp vụ từ đăng ký tài khoản, tìm kiếm và lọc mentor nâng cao, đặt lịch và thanh toán, đến đánh giá sau buổi tư vấn và quản trị hệ thống bởi Admin.

Về mặt kỹ thuật, Mentoria được xây dựng theo kiến trúc Client–Server ba lớp: frontend React + Vite + TypeScript, backend Node.js + Express, và cơ sở dữ liệu SQL Server. Cơ sở dữ liệu được thiết kế theo mô hình quan hệ với EERD đầy đủ, ánh xạ sang lược đồ quan hệ và triển khai hoàn chỉnh trên SQL Server bao gồm các stored procedures, triggers và functions phục vụ logic nghiệp vụ.

Trong khuôn khổ môn học Hệ Quản trị Cơ sở Dữ liệu (CO3021), nhóm đã sử dụng Mentoria như domain thực tế để nghiên cứu, thực nghiệm và so sánh hai hệ quản trị cơ sở dữ liệu: MS SQL Server (quan hệ, ACID) và Apache Cassandra (NoSQL phân tán, BASE). Mỗi thành viên phụ trách một chủ đề kỹ thuật riêng biệt gắn với bài toán thực của hệ thống.

Báo cáo này trình bày toàn bộ quá trình nghiên cứu lý thuyết, thực nghiệm đo lường hiệu năng, và đánh giá tổng hợp nhằm đưa ra quyết định lựa chọn DBMS phù hợp nhất cho hệ thống Mentoria trong môi trường sản xuất thực tế.

3 Phân tích hệ thống

3.1 Thực hiện phân tích chung hệ thống liên quan

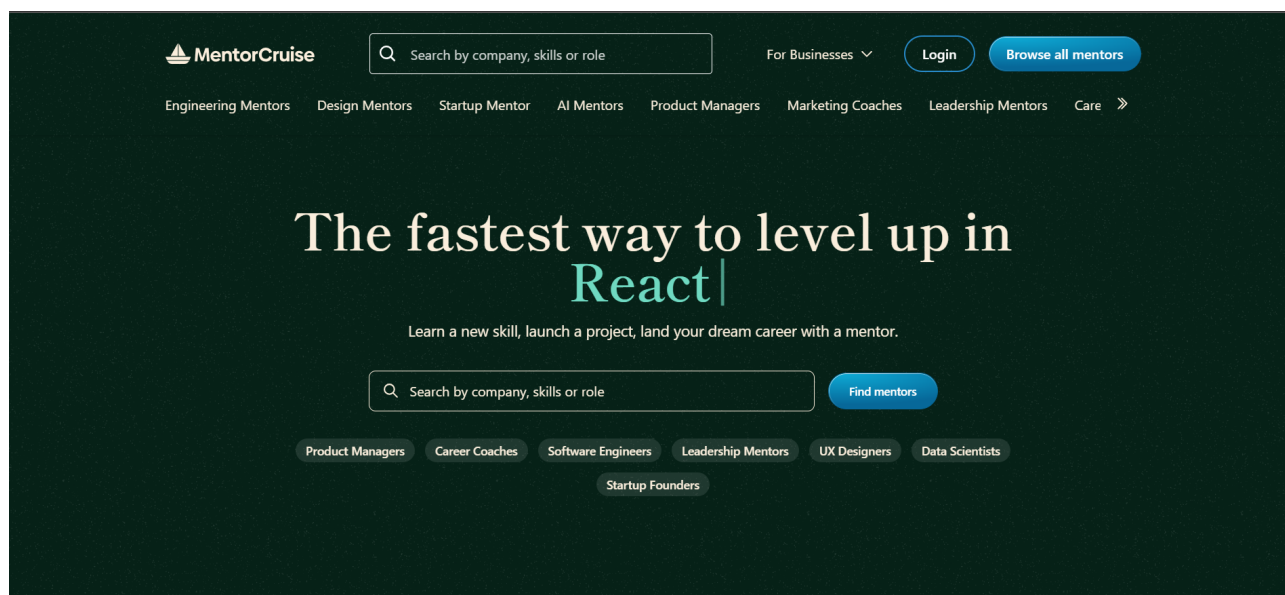
Tên hệ thống: MentorCruise.

Liên kết: mentorcruise.com.

MentorCruise là nền tảng kết nối giữa những người có kinh nghiệm (Mentor) và người học hoặc người đang tìm định hướng nghề nghiệp (Mentee). Hệ thống hoạt động theo mô hình mentor-mentee trực tuyến, nơi Mentee có thể lựa chọn Mentor phù hợp với nhu cầu học tập hoặc phát triển nghề nghiệp của mình.

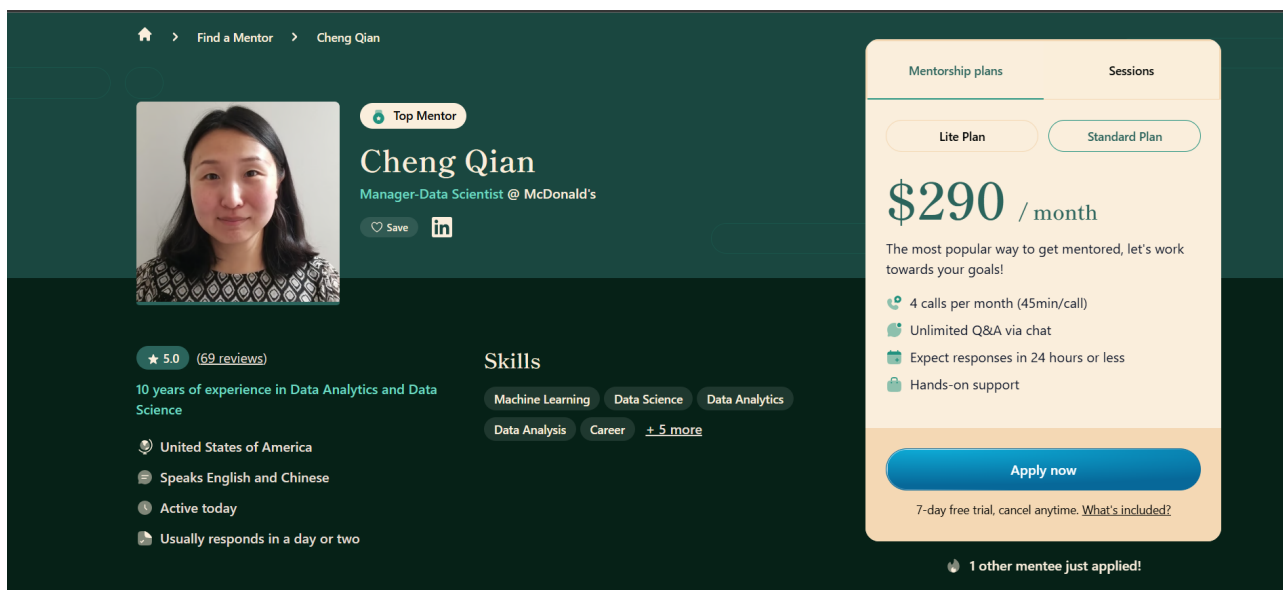
Hình ảnh và giao diện chính:

- **Trang chủ:** Nơi người dùng có thể tìm kiếm các Mentor dựa trên lĩnh vực chuyên môn.



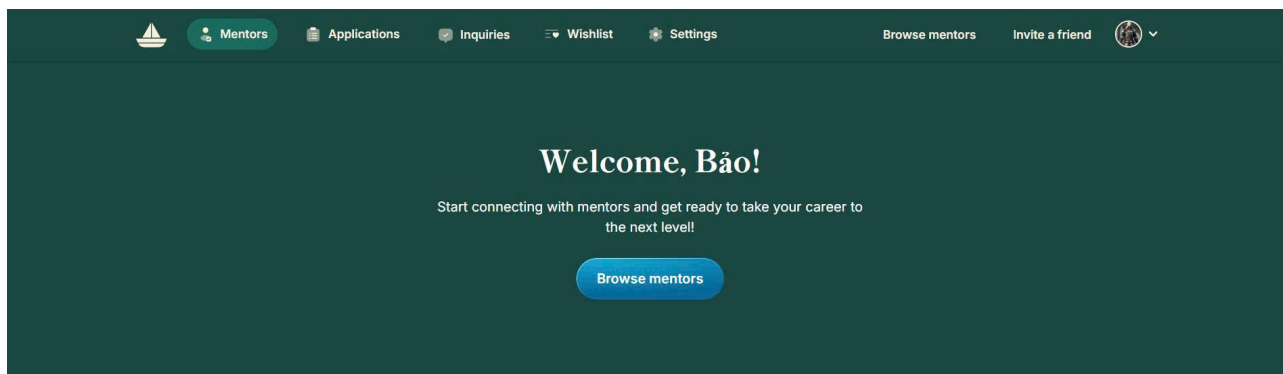
Hình 1: Giao diện trang chủ MentorCruise

- **Trang hồ sơ Mentor:** Hiển thị thông tin chi tiết về một Mentor, bao gồm kinh nghiệm, kỹ năng, các gói mentoring và đánh giá từ các Mentee trước đó.



Hình 2: Giao diện trang hồ sơ của Mentor

- **Trang quản lý:** Giao diện quản lý cho cả Mentor và Mentee để theo dõi tiến độ, lịch hẹn và trao đổi.



Hình 3: Giao diện trang quản lý (Dashboard)

3.2 Yêu cầu hệ thống

3.2.1 Yêu cầu chức năng

Chức năng chung:

- Đăng nhập, đăng xuất hệ thống.
- Phân quyền theo vai trò.

Guest:

- Guest có thể truy cập và duyệt qua một phần nội dung nền tảng mà không cần đăng ký tài khoản.
- Guest có thể xem các thông tin tổng quan về dịch vụ mentorship, các mentor nổi bật, hoặc phần giới thiệu.
- Guest có thể đăng ký trở thành Mentee hoặc Mentor để sử dụng đầy đủ tính năng của nền tảng.

Mentee:

- Mentee có thể tìm kiếm mentor theo các tiêu chí như ngành nghề, kỹ năng và kinh nghiệm.
- Mentee có thể xem hồ sơ của mentor bao gồm bằng cấp, thời gian khả dụng và đánh giá từ các mentee khác.
- Mentee có thể đặt lịch và chi trả cho các phiên cố vấn thông qua hệ thống thanh toán trực tuyến.

Mentor:

- Mentor có thể tạo và quản lý hồ sơ chuyên nghiệp phản ánh chính xác trình độ và lĩnh vực của mình.
- Mentor có thể tùy chỉnh lịch rảnh và quản lý các lịch hẹn với mentee.
- Mentor có thể tạo các phiên cố vấn phù hợp cho mentee tham khảo.

Admin:

- Admin có thể theo dõi các thông tin tổng quan của hệ thống .

- Admin có thể quản lý thông tin của người dùng hệ thống và chấp nhận hồ sơ ứng tuyển trở thành mentor.
- Admin có thể xem doanh thu của hệ thống và chi tiết các giao dịch được trả về thông qua Stripe API.

3.2.2 Yêu cầu phi chức năng

Hiệu suất:

- Hệ thống phải đảm bảo tốc độ phản hồi nhanh, thời gian xử lý trung bình cho các thao tác thường xuyên (đăng nhập, tìm kiếm, đặt lịch hẹn, thống kê) không vượt quá 2 giây.
- Có khả năng phục vụ đồng thời tối thiểu 1000 người dùng trực tuyến mà không gây gián đoạn hoặc suy giảm đáng kể hiệu suất.

Khả năng mở rộng:

- Cơ sở dữ liệu và hạ tầng lưu trữ cần được thiết kế linh hoạt, cho phép phân vùng dữ liệu hoặc mở rộng dung lượng mà không làm gián đoạn dịch vụ.
- Kiến trúc hệ thống hỗ trợ bổ sung hoặc nâng cấp các module mới (ví dụ: tích hợp AI, phân tích dữ liệu nâng cao) mà không ảnh hưởng đến các chức năng đang vận hành.

Bảo mật và an toàn:

- Mọi thao tác người dùng được mã hóa thông qua HTTPS/TLS.
- Dữ liệu cá nhân của người dùng phải được mã hóa trong quá trình lưu trữ và truyền tải.
- Có cơ chế sao lưu và khôi phục dữ liệu định kỳ để hạn chế rủi ro mất mát.

Tin cậy và sẵn sàng:

- Hệ thống có thể hoạt động liên tục với thời gian sẵn sàng tối thiểu 99% trong năm.
- Có cơ chế phát hiện và xử lý lỗi để giảm thiểu thời gian gián đoạn dịch vụ.

Bảo trì và nâng cấp:

- Mã nguồn và tài liệu kỹ thuật được tổ chức rõ ràng để dễ dàng bảo trì, sửa lỗi và mở rộng.
- Thời gian bảo trì định kỳ cần được thông báo trước cho người dùng, giới hạn tối đa trong vòng 4 giờ cho mỗi lần bảo trì nhằm giảm thiểu gián đoạn dịch vụ.
- Trong trường hợp khẩn cấp (ví dụ: phát hiện lỗ hổng bảo mật nghiêm trọng, lỗi hệ thống gây ngưng trệ dịch vụ), đội ngũ kỹ thuật được phép triển khai bảo trì ngoài kế hoạch, với cơ chế thông báo nhanh chóng đến tất cả người dùng liên quan.
- Cập nhật phần mềm và vá lỗi bảo mật định kỳ mỗi 6 tháng một lần.

3.2.3 Yêu cầu chức năng không tương tác

Bên cạnh các yêu cầu chức năng mang tính tương tác trực tiếp với người dùng, hệ thống còn cần đáp ứng một số yêu cầu chức năng không tương tác. Đây là những chức năng diễn ra một cách tự động hoặc ngầm định trong quá trình vận hành hệ thống, nhằm đảm bảo tính chính xác của dữ liệu, cũng như nâng cao hiệu quả quản lý và trải nghiệm của người dùng:

- Hệ thống tự động gửi thông báo (qua email, hệ thống nội bộ) khi có thay đổi lịch hẹn, cập nhật từ mentor hoặc khi có thông tin quan trọng từ quản trị viên.
- Dữ liệu hệ thống (hồ sơ, lịch sử tư vấn, báo cáo) được lưu trữ an toàn và có cơ chế sao lưu định kỳ mỗi tháng để phòng ngừa sự cố.
- Tự động ghi lại hoạt động quan trọng trong hệ thống (đăng nhập, thay đổi hồ sơ, quản lý buổi hẹn, đánh giá) để phục vụ giám sát và kiểm toán.
- Tự động kiểm soát thời gian hoạt động của phiên đăng nhập (session), tự động hết hạn sau một khoảng thời gian người dùng không sử dụng dịch vụ.
- Hệ thống giám sát và ghi nhận sự cố, cảnh báo cho quản trị viên.

- Thực hiện cơ chế xử lý hàng đợi và giới hạn đăng nhập để đảm bảo hệ thống vận hành trơn tru khi có lượng truy cập cao.

3.3 Kiến trúc hệ thống

3.3.1 Mô hình tổng quát

Hệ thống được triển khai theo kiến trúc Client–Server ba lớp (3-tier) kinh điển, tách bạch trách nhiệm rõ ràng giữa các lớp:

- Presentation Layer (Frontend): React 19 + Vite + TypeScript + Tailwind-CSS. Vite đảm nhiệm dev server với HMR và bundle tối ưu cho production. Mã nguồn được tổ chức theo apis/, components/, layouts/, pages/, store/, types/. Lớp này chỉ chịu trách nhiệm rendering và gọi API qua HTTP, không chứa logic nghiệp vụ.
- Application Layer (Backend): Node.js + Express + TypeScript, cung cấp RESTful API. Tổ chức thư mục theo mô hình phân lớp service-oriented: controllers/ (HTTP entry), services/ (business logic), routes/ (định tuyến), middlewares/ (auth, error handling), validation/ (kiểm tra input). Kết nối DB qua thư viện mssql với cơ chế connection pool để tránh chi phí khởi tạo connection cho mỗi request – đây là điểm quan trọng cho throughput khi tải đồng thời cao. Logging dùng Winston có cấp độ.
- Data Layer (Database): MS SQL Server đảm nhiệm vai trò single source of truth. Một phần đáng kể logic nghiệp vụ được đẩy xuống lớp DB thay vì xử lý ở backend, gồm:
 - Stored Procedures (sp_SearchMentors, các thủ tục thống kê dashboard, các thủ tục CRUD slot có kiểm tra xung đột thời gian).
 - Triggers cho ràng buộc ngữ nghĩa (xác thực email trước khi kích hoạt account, cập nhật thuộc tính dẫn xuất total_reviews/total_stars/rating, cập nhật total_mentee từ số meeting ở trạng thái Completed).
 - Functions (tính rating trung bình, response time của mentor).

3.3.2 Mô tả hệ thống đề xuất

Hệ thống là một nền tảng kết nối giữa Mentor (người hướng dẫn) và Mentee (người được hướng dẫn), cho phép người dùng tìm kiếm, lựa chọn và tương tác với nhau dựa trên chuyên môn, kinh nghiệm và mục tiêu học tập. Hệ thống hướng đến việc hỗ trợ người dùng phát triển kỹ năng nghề nghiệp, học tập có định hướng và tạo ra mối quan hệ hướng dẫn lâu dài giữa hai bên.

Trong hệ thống có ba nhóm người dùng chính. Mentee là những cá nhân có nhu cầu học hỏi hoặc phát triển nghề nghiệp, thường là sinh viên. Mentor là những chuyên gia có kinh nghiệm trong các lĩnh vực cụ thể, cung cấp dịch vụ hướng dẫn, chia sẻ kỹ năng và kinh nghiệm. Quản trị viên chịu trách nhiệm quản lý thông tin người dùng, phê duyệt hồ sơ của Mentor, xử lý phản hồi và giám sát hoạt động chung của hệ thống.

Các chức năng chính của hệ thống bao gồm: tìm kiếm và lọc Mentor theo chuyên ngành, kỹ năng, từ khóa, hay công ty; xem thông tin chi tiết của từng Mentor như kinh nghiệm làm việc, gói mentoring, mức phí, tiểu sử và đánh giá từ Mentee khác; đăng ký gói mentoring phù hợp; quản lý và theo dõi lịch hẹn làm việc giữa Mentor và Mentee; cung cấp hệ thống trò chuyện trực tuyến để trao đổi, chia sẻ tài liệu; và tính năng đánh giá, nhận xét sau mỗi buổi mentoring nhằm đảm bảo chất lượng và uy tín của hệ thống.

Hệ thống quản lý **người dùng** xác định mỗi người dùng bằng một mã định danh duy nhất. Thông tin lưu trữ bao gồm họ tên, hình đại diện, mật khẩu, email, quốc gia, giới tính, múi giờ, vai trò, trạng thái tài khoản, thời gian tạo tài khoản, thời gian cập nhật gần nhất, mã OTP và thời gian hết hạn của mã OTP, mã khôi phục mật khẩu cùng thời gian hết hạn của mã này, mã Google, mã xác thực tài khoản qua email và loại tài khoản. Ngoài ra, người dùng còn có thể liên kết với nhiều tài khoản mạng xã hội, mỗi nền tảng sẽ có một đường dẫn liên kết riêng biệt.

Mỗi người dùng có thể **nhận** nhiều **thông báo**, và một thông báo có thể được gửi đến nhiều người dùng. Thực thể liên kết giữa người dùng và thông báo ghi nhận thời điểm thông báo được gửi. Mỗi thông báo có mã định danh, tiêu đề và nội dung.

Ngoài ra, hệ thống hỗ trợ chức năng nhắn tin giữa người dùng. Một người dùng có thể **gửi và nhận** nhiều **tin nhắn** từ những người dùng khác. Mỗi tin nhắn bao gồm

mã tin nhắn, thời gian gửi và nội dung.

Trong hệ thống, mỗi người dùng chỉ có thể đảm nhiệm một vai trò duy nhất tại một thời điểm, hoặc là **Mentee**, hoặc là **Mentor**, và không thể đồng thời thuộc cả hai nhóm. Đối với Mentee, hệ thống lưu trữ thông tin liên quan đến mục tiêu phát triển cá nhân mà họ mong muốn đạt được.

Trong khi đó, Mentor được quản lý với tập dữ liệu phong phú hơn nhằm phản ánh quá trình hoạt động chuyên môn của họ. Các thuộc tính dẫn xuất được hệ thống tự động tính toán bao gồm tổng số sao nhận được, tổng số Mentee đã hỗ trợ, tổng số lượt đánh giá cũng như điểm đánh giá trung bình, được xác định dựa trên tỷ lệ giữa tổng số sao và tổng số lượt đánh giá. Ngoài ra, Mentor còn có thông tin mô tả về các ngôn ngữ sử dụng, tiểu sử cá nhân, tiêu đề giới thiệu ngắn gọn, thời gian dự kiến phản hồi tin nhắn, bản CV đính kèm, cũng như các thông tin liên quan đến ngân hàng, bao gồm tên ngân hàng, mã số tài khoản, tên trên thẻ, chi nhánh ngân hàng và mã SWIFT.

Một Mentor có thể đồng thời **làm việc** với nhiều công ty, và tại mỗi công ty, họ đảm nhiệm một chức danh nghề nghiệp nhất định. Hệ thống quản lý **chức danh nghề nghiệp** như một thực thể độc lập, với mã số định danh và tên gọi cụ thể. **Công ty** cũng được mô hình hóa thành một thực thể riêng, với mã công ty và tên công ty tương ứng.

Ngoài ra, Mentor bắt buộc phải **sở hữu** ít nhất một **kỹ năng chuyên môn**. Kỹ năng được xem là thực thể có thể được chia sẻ giữa nhiều Mentor, mỗi kỹ năng được định danh bởi một mã kỹ năng và tên kỹ năng. Các kỹ năng này có thể **liên kết** với nhiều **lĩnh vực chuyên môn**, và tương tự, một lĩnh vực có thể bao gồm nhiều kỹ năng khác nhau. Thực thể lĩnh vực được hệ thống quản lý thông qua mã định danh và tên; và trong một cấu trúc phân cấp, một lĩnh vực có thể bao gồm các lĩnh vực con, tuy nhiên bản thân nó chỉ trực thuộc duy nhất một lĩnh vực cha.

Bên cạnh các thông tin cá nhân và chuyên môn, Mentor còn phải **thiết lập** các **phiên làm việc** của mình trong hệ thống. Phiên làm việc được hiểu như những khoảng thời gian rảnh mà Mentor chủ động sắp xếp để sẵn sàng tiếp nhận và hỗ trợ Mentee. Mỗi Mentor có thể tạo ra nhiều phiên làm việc khác nhau, nhằm phản ánh lịch trình thực tế và tính linh hoạt trong việc tư vấn. Mỗi phiên làm việc bao gồm thông tin về

ngày, thời điểm bắt đầu và thời điểm kết thúc, các giá trị này có tính duy nhất trong phạm vi từng Mentor để tránh trùng lặp lịch. Ngoài ra, mỗi phiên làm việc còn có một thuộc tính trạng thái, cho phép hệ thống phân biệt giữa các phiên đã có người đặt, phiên đang mở hoặc phiên đã bị hủy, qua đó hỗ trợ việc quản lý lịch hẹn hiệu quả hơn.

Một Mentee có thể **đánh giá** nhiều Mentor và một Mentor có thể nhận được đánh giá từ nhiều Mentee; hệ thống lưu trữ thông tin về số sao, nội dung, và thời gian gửi của đánh giá đó.

Bên cạnh việc thiết lập lịch làm việc, mỗi Mentor bắt buộc phải **khai báo** các **gói mentoring** mà họ cung cấp. Gói mentoring được xem như một sản phẩm dịch vụ mà Mentor cung cấp cho Mentee, với mục tiêu đa dạng hóa mô hình hỗ trợ và đáp ứng nhu cầu học tập cá nhân hóa. Hệ thống quản lý thông tin của mỗi gói mentoring bao gồm mã định danh, mô tả chi tiết, mức phí và loại hình gói.

Hiện tại, hệ thống hỗ trợ hai loại gói mentoring chính, được phân loại theo đặc thù cách thức triển khai. Loại thứ nhất là **gói mentoring theo phiên**, tập trung vào các buổi gặp gỡ tư vấn trực tiếp thông qua nền tảng, và có thuộc tính đặc trưng là thời lượng của phiên tư vấn. Loại thứ hai là **gói mentoring theo tháng**, cung cấp sự đồng hành liên tục trong một khoảng thời gian dài hơn, với các đặc tính bao gồm số cuộc gọi mỗi tháng, thời gian cuộc gọi và các tiện ích riêng biệt mà Mentee nhận được.

Mỗi gói mentoring **sử dụng** các phiên làm việc được Mentor thiết lập trước đó, đảm bảo tính nhất quán về thời gian và khả năng đáp ứng của Mentor. Một phiên làm việc chỉ thuộc về duy nhất một gói mentoring, nhằm đảm bảo rằng lịch trình hỗ trợ được phân bổ rõ ràng và không gây ra xung đột giữa các gói dịch vụ khác nhau.

Một Mentee có thể **đăng ký** nhiều gói mentoring khác nhau, và mỗi lần đăng ký được hệ thống ghi nhận dưới dạng một **đơn đăng ký** riêng biệt. Mỗi đơn đăng ký chứa các thông tin cơ bản gồm mã định danh và lời nhắn cá nhân mà Mentee muốn gửi đến Mentor, nhằm làm rõ nhu cầu hoặc mục tiêu cá nhân trong quá trình hướng dẫn.

Để tăng tính linh hoạt và khuyến khích sử dụng dịch vụ, hệ thống cho phép mỗi đơn đăng ký **áp dụng** một **mã giảm giá**, trong khi một mã giảm giá có thể được sử dụng cho nhiều đơn đăng ký khác nhau. Mỗi mã giảm giá được quản lý với các thuộc tính quan trọng như mã định danh, tên mã, loại giảm giá (giảm theo phần trăm hoặc

theo giá tiền cố định), giá trị giảm, khoảng thời gian áp dụng (ngày bắt đầu và ngày kết thúc), trạng thái hiệu lực của mã, số lần sử dụng tối đa cho phép và số lần đã được sử dụng thực tế.

Sau khi đơn đăng ký được xác nhận, hệ thống sẽ **xuất** ra một **hóa đơn** tương ứng. Mỗi hóa đơn được quản lý với mã định danh riêng, cùng các thông tin liên quan đến giao dịch trên Stripe, bao gồm mã định danh của phiên thanh toán, mã tài khoản khách hàng, email khách hàng, mã ý định thanh toán, mã giao dịch thực tế, mã giao dịch cân bằng, và liên kết đơn thanh toán. Các thuộc tính khác của hóa đơn bao gồm trạng thái thanh toán, đơn vị tiền tệ, phương thức thanh toán, số tiền trước thuế và tổng số tiền phải thanh toán. Khi Mentee tiến hành thanh toán hóa đơn, hệ thống sẽ lưu trữ thời gian thanh toán của Mentee. Mentee sẽ tiến hành **thanh toán** hóa đơn, hệ thống sẽ lưu trữ thời gian thanh toán hóa đơn của Mentee.

Từ hóa đơn đã thanh toán, hệ thống sẽ tạo ra một hoặc nhiều **buổi tư vấn**, tùy thuộc vào loại gói mentoring mà Mentee đã đăng ký. Mỗi buổi tư vấn có các thuộc tính bao gồm mã buổi, trạng thái hiện tại, địa điểm học tập và đường dẫn ôn tập. Mỗi buổi tư vấn phải được **liên kết** với một phiên làm việc cụ thể của Mentor, và phiên làm việc này đã được Mentor thiết lập trước đó. Cơ chế này đảm bảo rằng việc xếp lịch tư vấn luôn tuân thủ tính khả dụng của Mentor, đồng thời giúp hệ thống duy trì sự nhất quán trong quản lý thời gian và chất lượng dịch vụ.

3.3.3 Mô tả các ràng buộc ngữ nghĩa

Các ràng buộc ngữ nghĩa sau đây mô tả các quy tắc mà mô hình EERD không thể hiện được hoàn toàn:

- Ràng buộc trên thuộc tính:
 - Email của người dùng phải duy nhất và hợp lệ trong toàn hệ thống.
 - Mật khẩu của người dùng phải được hash.
 - Giới tính của người dùng chỉ nhận các giá trị hợp lệ: *nam* hoặc *nữ*.
 - Các liên kết mạng xã hội phải là URL hợp lệ.
 - Trạng thái của người dùng chỉ nhận một trong các giá trị: *active*, *inac-*

tive, banned, pending.

- Điểm đánh giá nằm trong khoảng từ 1 đến 5.
- Nội dung đánh giá không được để trống.
- Thời lượng của gói mentoring không vượt quá 2 tiếng.

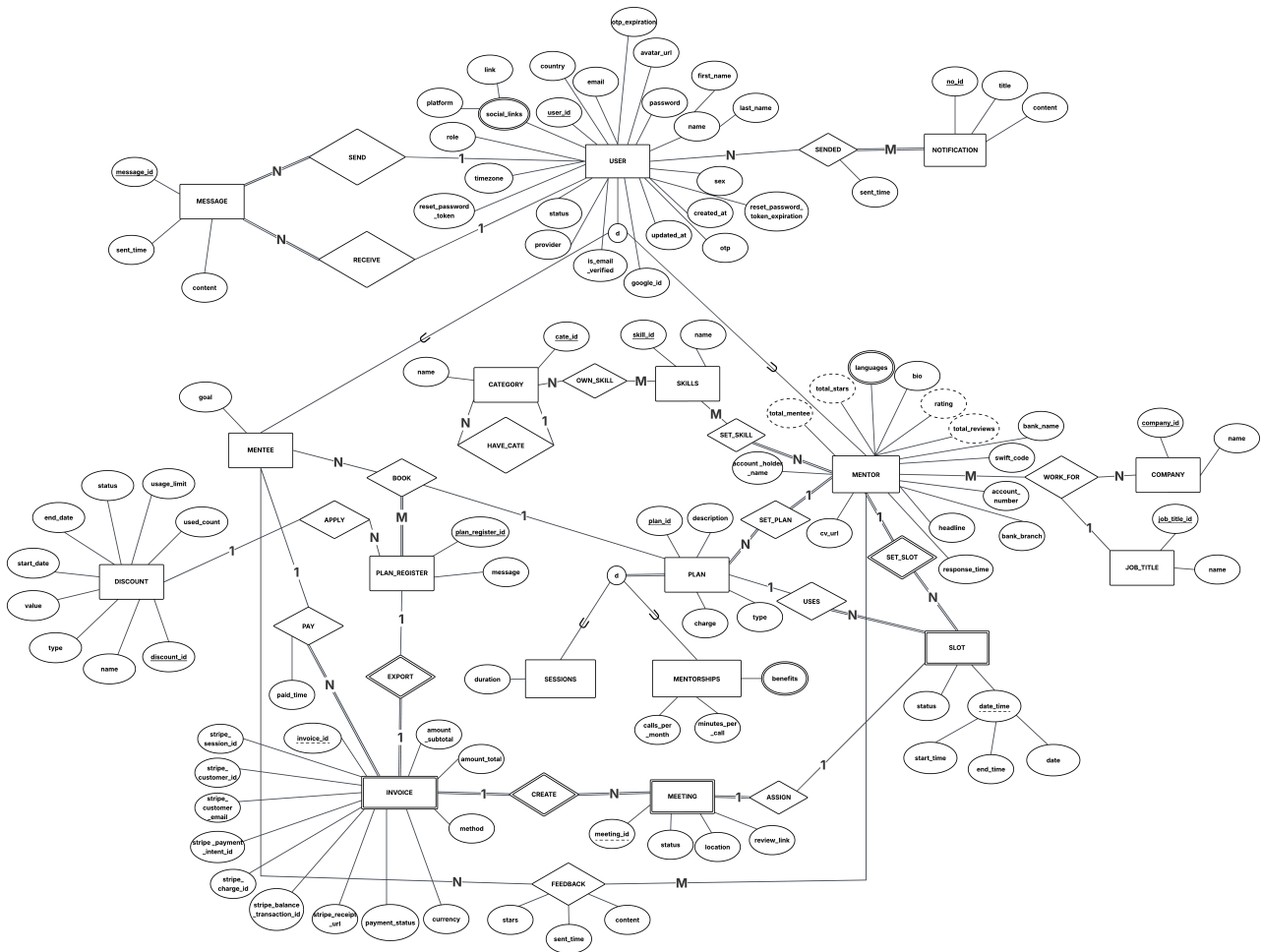
- Ràng buộc trên thực thể:

- Tài khoản mới tạo phải được xác minh qua email trước khi kích hoạt.
- Mỗi mentor phải có ít nhất một lĩnh vực chuyên môn trước khi được phép mở buổi tư vấn.

- Ràng buộc trên liên kết:

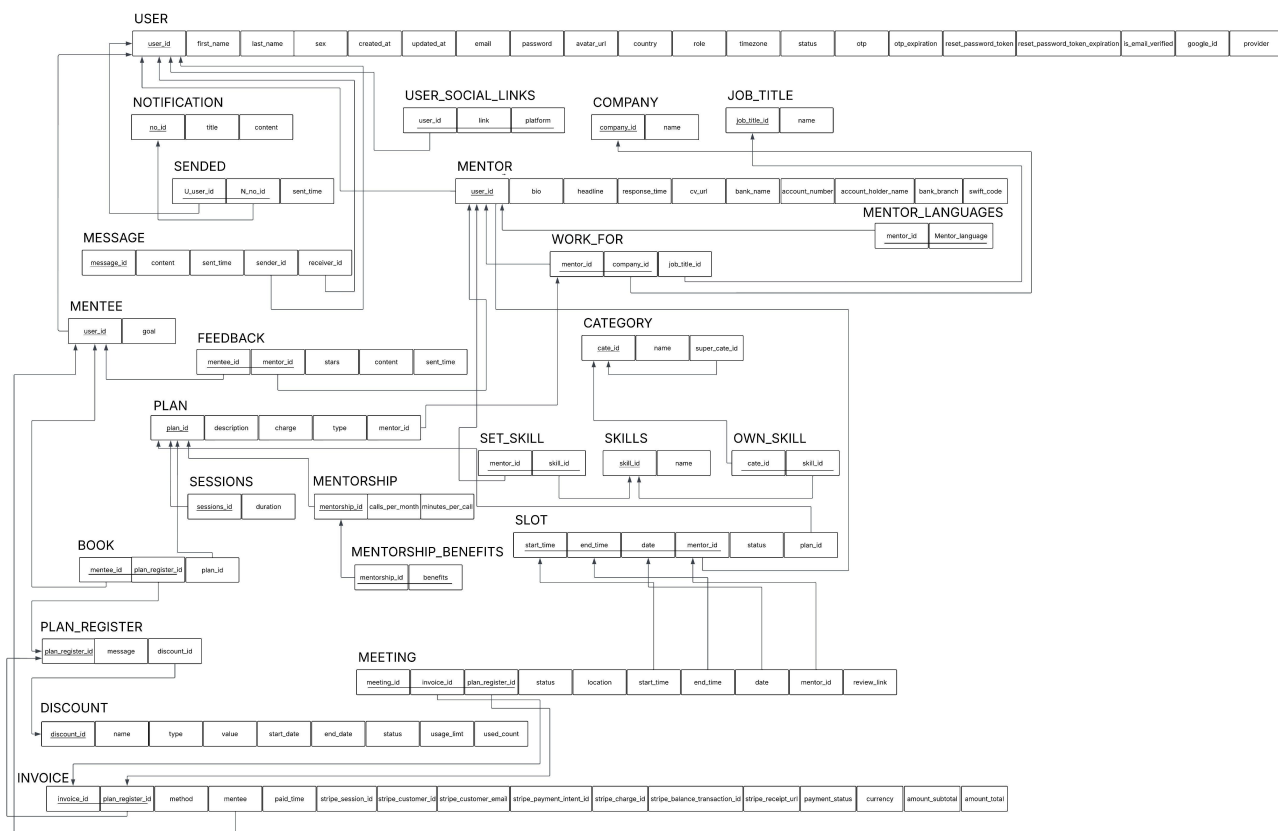
- Chỉ những Mentee đã tham gia và hoàn thành ít nhất một buổi tư vấn của Mentor mới có thể gửi đánh giá.
- Khi một phiên làm việc đã được đăng ký, trạng thái của nó phải chuyển thành "*đã được đăng ký*" và không cho Mentee khác đăng ký phiên đó nữa.
- Tin nhắn không thể bị chỉnh sửa sau khi gửi.

3.3.4 Thiết kế EERD



Hình 4: Mô hình EERD của hệ thống

3.3.5 Ảnh xạ lược đồ



Hình 5: Lược đồ CSDL quan hệ ánh xạ từ EERD đã thiết kế

4 So sánh MS SQL Server và Cassandra

4.1 Data storage & management

4.1.1 Khái niệm chung về data storage và data management

Data Storage (Lưu trữ dữ liệu) là quá trình tổ chức, cấu trúc và lưu giữ dữ liệu dưới các định dạng phù hợp trên các thiết bị hoặc hệ thống lưu trữ. Data Storage tập trung vào việc thiết kế mô hình dữ liệu, định nghĩa schema, cách phân tán và biểu diễn dữ liệu nhằm đảm bảo tính sẵn sàng, hiệu quả truy xuất và khả năng mở rộng của hệ thống.

Data Management (Quản trị dữ liệu) là tập hợp các cơ chế và chức năng mà hệ quản trị cơ sở dữ liệu cung cấp để quản lý, thao tác, bảo vệ và khai thác dữ liệu đã được lưu trữ. Data Management bao gồm các hoạt động như truy vấn dữ liệu, đảm bảo tính nhất quán, hỗ trợ transaction, kiểm soát đồng thời, lập chỉ mục và bảo mật dữ liệu, nhằm nâng cao hiệu quả vận hành và hỗ trợ ra quyết định dựa trên dữ liệu.

4.1.2 Đặc trưng cơ chế data storage và data management trong MS SQL Server

MS SQL Server là một hệ quản trị cơ sở dữ liệu sử dụng mô hình dữ liệu quan hệ. Với các dữ liệu được tổ chức dưới dạng bảng quan hệ 2 chiều với các schema cố định được định nghĩa rõ ràng từ đầu, các bảng này được liên kết logic với nhau qua các khóa chính và khóa ngoại.

SQL Server tập trung vào mối quan hệ giữa các bảng, nó hỗ trợ đầy đủ các lệnh JOIN và các transaction theo nguyên tắc ACID, nhờ đó lập trình viên có thể dễ dàng kết hợp và tương tác dữ liệu của nhiều bảng chỉ qua 1 câu lệnh SQL duy nhất.

Về mặt lưu trữ vật lý, MS SQL Server sử dụng cấu trúc B-Tree làm nền tảng cho chỉ mục cụm. Với cấu trúc này, dữ liệu được tổ chức thành các trang sắp xếp theo thứ tự logic của khóa chính trên toàn bộ tập dữ liệu. Khi thực hiện thao tác ghi, hệ thống phải định vị đúng vị trí trong cây B-Tree, ghi đè trực tiếp lên trang dữ liệu hiện có, đồng thời cập nhật transaction log để đảm bảo tính ACID. Cơ chế này tối ưu cho các

truy vấn đọc ngẫu nhiên và truy vấn phạm vi, nhưng đồng thời làm giảm tốc độ ghi do phải thực hiện nhiều thao tác kiểm tra và cập nhật đồng bộ.

Với những đặc điểm trên, SQL Server mang lại lợi thế lớn trong việc quản lý dữ liệu có cấu trúc rõ ràng, đặc biệt là các hệ thống yêu cầu truy vấn phức tạp. Hệ thống hỗ trợ tốt việc chuẩn hóa dữ liệu, giảm thiểu dư thừa và đảm bảo tính toàn vẹn tham chiếu giữa các bảng. Tuy nhiên, SQL Server gặp hạn chế đáng kể về khả năng mở rộng ngang khi khối lượng dữ liệu tăng lớn do schema cứng nhắc.

4.1.3 Đặc trưng cơ chế data storage và data management trong Cassandra

Cassandra là hệ quản trị cơ sở dữ liệu thuộc loại Wide-Column Store trong họ NoSQL. Về mặt lưu trữ dữ liệu, Cassandra tổ chức dữ liệu dưới dạng các hàng có số lượng cột động không giới hạn, dữ liệu được phân tán và sắp xếp dựa trên khóa chính bao gồm khóa phân vùng hỗ trợ quyết định phân tán dữ liệu và khóa phân cụm hỗ trợ sắp xếp dữ liệu bên trong 1 phân vùng. Ngoài ra, Cassandra còn hỗ trợ chỉ mục thứ cấp tạo chỉ mục trên các cột không phải khóa chính. Khác với MS SQL Server, Cassandra không yêu cầu schema cố định từ đầu, và hỗ trợ thêm cột động ngay trong thời gian thực thi.

Về mặt quản trị dữ liệu, Cassandra có những hạn chế rõ rệt khi hệ thống không hỗ trợ lệnh JOIN và không cung cấp đầy đủ các transaction ACID. Dẫn đến nếu cần kết hợp dữ liệu từ nhiều bảng hoặc từ nhiều phân vùng, lập trình viên buộc phải thực hiện nhiều truy vấn riêng lẻ và tự xử lý phép JOIN ở tầng ứng dụng. Để hỗ trợ truy vấn, Cassandra còn hỗ trợ chỉ mục thứ cấp tạo chỉ mục trên các cột không phải khóa chính. Tuy nhiên, chỉ mục thứ cấp có hiệu suất hạn chế, và dễ gây tình trạng hotspot, khiến một hoặc một vài node trong phân cụm phải chịu tải rất cao trong khi các node khác lại ít tải hoặc gần như không làm việc, làm phá vỡ tính cân bằng tải, vì lẽ đó nên Cassandra không được khuyến khích sử dụng trên bảng chứa nhiều dữ liệu.

Về mặt lưu trữ vật lý, Apache Cassandra sử dụng cấu trúc LSM-Tree, luôn ghi thêm dữ liệu mới theo cơ chế Append-only. Mỗi thao tác ghi được ghi tuần tự vào Commit Log và MemTable. Khi MemTable đầy, toàn bộ dữ liệu trong đó được chuyển xuống đĩa thành một file SSTable lưu trữ bất biến, không thể sửa đổi sau khi tạo. Theo

thời gian, nhiều SSTable nhỏ tích lũy trên đĩa sẽ được hệ thống tự động gộp lại, giúp giải phóng dung lượng và cải thiện hiệu suất đọc. Nhờ cơ chế này, Cassandra tránh được chi phí định vị vị trí ngẫu nhiên trên đĩa, từ đó đạt tốc độ ghi rất cao nhưng thao tác đọc có thể chậm hơn do dữ liệu của một bản ghi có thể nằm rải rác trên nhiều SSTable khác nhau. Để khắc phục điều này, Cassandra sử dụng Key Cache để lưu lại vị trí của các bản ghi thường xuyên truy cập, và Bloom Filter để nhanh chóng loại bỏ các SSTable không chứa dữ liệu cần tìm, giúp giảm đáng kể số lần truy cập đĩa không cần thiết.

Có thể thấy, Cassandra nổi bật với tính khả dụng cao, sự linh hoạt, tốc độ đọc/ghi lớn, cùng khả năng mở rộng ngang gần như tuyến tính. Nhưng đổi lại, Cassandra chấp nhận dữ liệu dư thừa, sử dụng mô hình Eventual consistency cho phép tồn tại các trạng thái không đồng bộ tạm thời trong hệ thống. Điểm yếu lớn nhất của hệ quản trị này là thiếu hỗ trợ JOIN và transaction ACID đầy đủ, đòi hỏi người thiết kế phải có kỹ năng trong mô hình hóa dữ liệu theo hướng Query-Driven ngay từ bước đầu thiết kế schema.

4.1.4 Kịch bản so sánh thực tế giữa SQL Server và Cassandra

4.1.4.1 Kịch bản 1: So sánh cơ chế sắp xếp dữ liệu

Với MS SQL Server, hệ thống sử dụng mô hình chuẩn hóa dữ liệu. Thông tin về mentor không được lưu trữ tập trung tại một bảng duy nhất mà phân tán tại nhiều bảng chuyên biệt để giảm dư thừa. Khi cần truy xuất, ta có thể sử dụng các phép JOIN tại thời điểm truy vấn để tổng hợp dữ liệu từ nhiều bảng khác nhau.

```
1 USE mentoria;  
2 EXEC dbo.sp_SearchMentors @Page = 1, @Limit = 10;
```

user_id	first_name	last_name	email	avatar_url	country	status	timezone
1	John	Doe	john.doe@example.com	https://i.pravatar.cc/150?img=1	United States	Active	America/New_York
2	Sarah	Johnson	sarah.johnson@example.com	https://i.pravatar.cc/150?img=5	United Kingdom	Active	Europe/London
3	Michael	Chen	michael.chen@example.com	https://i.pravatar.cc/150?img=12	Singapore	Active	Asia/Singapore
4	Emily	Rodriguez	emily.rodriguez@example.com	https://i.pravatar.cc/150?img=9	Spain	Active	Europe/Madrid
5	David	Kim	david.kim@example.com	https://i.pravatar.cc/150?img=15	South Korea	Active	Asia/Seoul
6	Lisa	Anderson	lisa.anderson@example.com	https://i.pravatar.cc/150?img=10	Canada	Active	America/Toronto
7	James	Martinez	james.martinez@example.com	https://i.pravatar.cc/150?img=13	Mexico	Active	America/Mexico_City
8	Anna	Kowalski	anna.kowalski@example.com	https://i.pravatar.cc/150?img=16	Poland	Active	Europe/Warsaw
9	Robert	Thompson	robert.thompson@example.com	https://i.pravatar.cc/150?img=17	Australia	Active	Australia/Sydney
10	Maria	Garcia	maria.garcia@example.com	https://i.pravatar.cc/150?img=23	Argentina	Active	America/Argentina/Buenos_Aires

Hình 6: Kết quả truy vấn 10 dữ liệu đầu tiên của MS SQL Server

Với Cassandra, hệ thống vận hành theo triết lý phi chuẩn hóa và thiết kế Schema dựa trên truy vấn. Các bảng dữ liệu được thiết kế dưới dạng một bảng phẳng chứa đầy đủ các thông tin cần thiết đã được tổng hợp và tính toán sẵn từ đầu, giúp tối ưu hóa hiệu suất đọc/ghi ở quy mô lớn. Khi cần truy xuất, ta chỉ cần gọi đến 1 bảng duy nhất đã được thiết kế để chứa toàn bộ dữ liệu cần thiết cho truy vấn đó.

```
1 USE mentoriadbms;  
2 SELECT * FROM mentor_profiles LIMIT 10;
```

mentor_id	account_holder_name	account_number	avatar_url	bank_branch	bank_name
a1000000-0000-0000-0000-000000000014	Laura Muller	4455667788	https://i.pravatar.cc/150?img=29	Berlin Alexanderplatz	Deutsche Bank
a1000000-0000-0000-0000-000000000025	Mohammed Ahmed	5566778899	https://i.pravatar.cc/150?img=46	Dubai Marina	Emirates NBD
a1000000-0000-0000-0000-000000000006	Lisa Anderson	6789012345	https://i.pravatar.cc/150?img=10	Toronto Downtown	Royal Bank of Canada
a1000000-0000-0000-0000-000000000015	Daniel Santos	5566778899	https://i.pravatar.cc/150?img=26	Sao Paulo Paulista	Banco do Brasil
a1000000-0000-0000-0000-000000000022	Isabella Lopez	2233445566	https://i.pravatar.cc/150?img=42	Bogota Zona T	Bancolombia
a1000000-0000-0000-0000-000000000030	Sophia Nguyen	0011223344	https://i.pravatar.cc/150?img=53	Ho Chi Minh District 1	Vietcombank
a1000000-0000-0000-0000-000000000004	Emily Rodriguez	4567890123	https://i.pravatar.cc/150?img=9	Madrid Centro	Santander Bank
a1000000-0000-0000-0000-000000000016	Emma Wilson	6677889900	https://i.pravatar.cc/150?img=32	Auckland Queen Street	ANZ Bank
a1000000-0000-0000-0000-000000000007	James Martinez	7890123456	https://i.pravatar.cc/150?img=13	Mexico City Reforma	BBVA Bancomer
a1000000-0000-0000-0000-000000000021	Patrick O'Brien	1122334455	https://i.pravatar.cc/150?img=39	Dublin College Green	Bank of Ireland

Hình 7: Kết quả truy vấn 10 dữ liệu đầu tiên của Apache Cassandra

Kết quả test case cho thấy cả hai hệ thống đều trả về 10 mentor, tuy nhiên kết quả lại khác nhau, kết quả của test case này là minh chứng cho sự khác nhau về cách sắp xếp dữ liệu của 2 hệ quản trị cơ sở dữ liệu MS SQL Server và Apache Cassandra.

MS SQL Server lưu dữ liệu theo thứ tự vật lý của chỉ mục cụm, mặc định là khóa chính. Vì lẽ đó khi yêu cầu lấy ra 10 dòng đầu, hệ thống sẽ tự hiểu và đưa ra 10 dòng dữ liệu tuần tự được sắp xếp sẵn theo khóa chính.

Cassandra sắp xếp thứ tự vật lý của dữ liệu trong nội bộ 1 phân vùng dữ liệu theo khóa phân cụm. Tuy nhiên các phân vùng sẽ được sắp xếp theo mã băm nên thứ tự 10 dòng trả về phụ thuộc vào thứ tự token nội bộ của các phân vùng trên ring, không có ý nghĩa logic với người dùng. Để lấy được 10 dòng đầu tiên theo khóa phân cụm, ta phải chỉ ra mình muốn lấy 10 dòng dữ liệu đầu tiên trong khóa phân vùng nào.

Có thể thấy cách sắp xếp dữ liệu của 2 hệ thống này hoàn toàn không giống nhau. Cách sắp xếp dữ liệu của MS SQL Server mang tính toàn cục, đảm bảo tính nhất quán về mặt thứ tự trên toàn bộ tập dữ liệu, vị trí lưu trữ vật lý phản ánh trực tiếp thứ tự logic của dữ liệu. Cách sắp xếp của Cassandra mang tính cục bộ, các phân vùng tồn tại một cách rời rạc và không có thứ tự logic với nhau.

4.1.4.2 Kịch bản 2: So sánh hiệu suất INSERT hàng loạt

Với MS SQL Server, ta có thể tạo dữ liệu giả lập trực tiếp bằng SQL Script, hàm ROW_NUMBER() kết hợp CROSS JOIN giúp sinh dữ liệu theo chuỗi tăng dần một cách hiệu quả, giảm tải cho CPU.

```
1 USE mentoria;
2 INSERT INTO companies(cname)
3 SELECT TOP 500000
4     'Company_New_' + CAST(ROW_NUMBER() OVER (ORDER BY (SELECT NULL))
5     AS VARCHAR)
6 FROM sys.all_objects a
7 CROSS JOIN sys.all_objects b;
```



```
[2026-05-01 15:23:24] mentoria> USE mentoria;
                                INSERT INTO companies (cname)
                                SELECT TOP 500000
                                    'Company_New_' + CAST(ROW_NUMBER() OVER (ORDER BY (SELECT NULL)) AS VARCHAR)
                                FROM sys.all_objects a
                                CROSS JOIN sys.all_objects b;
[2026-05-01 15:23:29] [S0001][5701] Changed database context to 'mentoria'.
[2026-05-01 15:23:29] 500,000 rows affected in 5 s 855 ms
```

Hình 8: Thời gian thực thi khi chèn 500.000 dòng vào bảng companies trên MS SQL Server

Với Apache Cassandra, ta tạo dữ liệu giả lập và đo tốc độ bằng công cụ benchmark cassandra-stress kết hợp tệp cấu hình yaml.

```
1 keyspace: mentoriadbms
2 table: companies
3 table_definition:
4     CREATE TABLE companies (
5         company_id uuid PRIMARY KEY,
6         cname text
7     );
8 columnspec:
9     - name: company_id
10       size: fixed(16)
11       population: seq(31..500030)
12     - name: cname
```

```
13     size: uniform(13..18)
14     population: seq(31..500030)
15 insert:
16     partitions: fixed(500)
17     batchtype: UNLOGGED
18 queries:
19     read1:
20         cql: select * from users where companies = ? and company_id = ?
21         fields: samerow
```

Lệnh chạy benchmark được thực thi qua Docker như sau:

```
1 docker run --rm -v "${PWD}:/data" --network host cassandra:latest /opt
  /cassandra/tools/bin/cassandra-stress user profile=/data/server/src
  /database/cassandra_stress.yaml n=1000 "ops(insert=1)" no-warmup cl
  =ONE -rate threads=1000
```

```
Results:
Op rate           :      4 op/s [insert: 4 op/s]
Partition rate    :    1,885 pk/s [insert: 1,885 pk/s]
Row rate          :    1,885 row/s [insert: 1,885 row/s]
Latency mean      :    60.2 ms [insert: 60.2 ms]
Latency median    :    59.8 ms [insert: 59.8 ms]
Latency 95th percentile :    60.7 ms [insert: 60.7 ms]
Latency 99th percentile :    60.7 ms [insert: 60.7 ms]
Latency 99.9th percentile :    60.7 ms [insert: 60.7 ms]
Latency max       :    60.7 ms [insert: 60.7 ms]
Total partitions  :      1,000 [insert: 1,000]
Total errors      :           0 [insert: 0]
Total GC count    : 0
Total GC memory   : 0 B
Total GC time     :    0.0 seconds
Avg GC time       :    NaN ms
StdDev GC time    :    0.0 ms
Total operation time : 00:00:00
```

Hình 9: Thời gian thực thi khi chèn 500.000 dòng vào bảng companies trên Apache Cassandra

Kết quả thực nghiệm cho thấy MS SQL Server hoàn thành việc chèn 500.000 dòng trong 5 giây 855ms khi thực thi tuần tự. Trong khi đó, Cassandra với 1.000 luồng song song hoàn thành cùng khối lượng dữ liệu trong thời gian dưới 1 giây. *Tuy nhiên đây không phải là so sánh tương đương, sự chênh lệch này phản ánh bản chất kiến trúc của từng hệ thống hơn là năng lực tuyệt đối của từng công cụ đo.*

SSMS thực thi tuần tự từng câu lệnh INSERT, mỗi bản ghi phải tuân thủ nghiêm ngặt tính ACID, hệ thống phải kiểm tra toàn bộ ràng buộc, cập nhật B-Tree index và ghi vào transaction log trước khi xác nhận thành công. Kiến trúc này đảm bảo tính nhất quán dữ liệu cao, phù hợp với các hệ thống đòi hỏi độ tin cậy và tính toàn vẹn lớn, nhưng đồng thời giới hạn khả năng ghi song song trên node đơn. *Trong thực tế khi cần đổ dữ liệu lớn vào MS SQL Server người ta có nhiều công cụ khác như BULK Insert, Bulk Copy Program,... thay vì sử dụng các câu lệnh INSERT thông thường như thế này vì mỗi dòng đều phải thực hiện kiểm tra ràng buộc, ghi log và xác nhận giao dịch, dẫn đến giảm tốc độ thực thi.*

Trong khi đó, Apache Cassandra chạy song song 1000 luồng để thêm vào các batch có kích thước lớn. Hệ quản trị cơ sở dữ liệu này được thiết kế tối ưu cho khối lượng ghi lớn với kiến trúc write-optimized thông qua cơ chế append-only, MemTable và Commit Log. Dữ liệu được ghi tuần tự vào commit log và bộ nhớ đệm mà không yêu cầu định vị vị trí trên đĩa. Nhờ đó, Cassandra cho phép hàng nghìn luồng ghi đồng thời mà ít xảy ra xung đột. Đây là lý do cassandra-stress với 1.000 threads là cách đo phù hợp và tự nhiên với kiến trúc của Cassandra. *Trong thực tế, cách nạp 1 batch với size lớn và nhiều luồng song song cùng chạy trong Cassandra thế này thường không được khuyến khích, tuy tốc độ nhanh hơn nhưng hiệu suất không ổn định, đặc biệt là khi áp dụng cho hệ thống lớn.*

4.1.4.3 Kịch bản 3: So sánh kiến trúc lưu trữ vật lý

MS SQL Server đại diện cho kiến trúc B-Tree với mô hình lưu trữ tập trung. Toàn bộ dữ liệu được tổ chức trong hai loại file chính, gồm file dữ liệu và file nhật ký. File dữ liệu chứa các trang dữ liệu và index, file nhật ký ghi lại toàn bộ transaction. Khi cập nhật, hệ thống ghi đè trực tiếp lên trang dữ liệu hiện có sau khi định vị đúng vị trí

trong B-Tree.

```
1 USE mentoria;  
2 GO  
3 SELECT  
4     name AS [Tên_Logic],  
5     type_desc AS [Loại_File],  
6     physical_name AS [Đường_dẫn_File_Vật_lý],  
7     (size * 8.0 / 1024) AS [Dung_lượng_MB]  
8 FROM sys.database_files;
```

Tên_Logic	Loại_File	Đường_dẫn_File_Vật_lý	Dung_lượng_MB
mentoria	ROWS	D:\sql\MSSQL16.SQLEXPRESS\MSSQL\DATA\mentoria.mdf	136.000000
mentoria_log	LOG	D:\sql\MSSQL16.SQLEXPRESS\MSSQL\DATA\mentoria_log.ldf	648.000000

Hình 10: Cấu trúc file lưu trữ vật lý của SQL Server trong hệ thống Mentoria

Apache Cassandra đại diện cho kiến trúc LSM-Tree, với mô hình lưu trữ phân tán. Thay vì ghi đè, Cassandra luôn ghi mới dữ liệu được tiếp nhận vào MemTable trong bộ nhớ, sau đó flush xuống đĩa thành các quá trình SSTable bất biến. Theo thời gian, các SSTable được gộp lại để tối ưu dung lượng.

Lệnh chạy nodetool được thực thi qua Docker như sau:

```
1 docker exec -it mentoria-cassandra nodetool info
```

```
ID : 72c8359f-7a31-4366-b58a-e2c89d504512  
Gossip active : true  
Native Transport active: true  
Load : 239.69 MiB  
Generation No : 1777647157  
Uptime (seconds) : 5145  
Heap Memory (MB) : 215.21 / 1897.44  
Off Heap Memory (MB) : 2.07  
Data Center : datacenter1  
Rack : rack1  
Exceptions : 0  
Key Cache : entries 112, size 10.77 KiB, capacity 94 MiB, 376 hits, 485 requests, 0.775 recent hit rate, 14400 save period in seconds  
Row Cache : entries 0, size 0 bytes, capacity 0 bytes, 0 hits, 0 requests, NaN recent hit rate, 0 save period in seconds  
Counter Cache : entries 92, size 11.5 KiB, capacity 47 MiB, 0 hits, 0 requests, NaN recent hit rate, 7200 save period in seconds  
Network Cache : size 0 bytes, overflow size: 0 bytes, capacity 118 MiB  
Percent Repaired : 0.0%  
Token : (invoke with -T/--tokens to see all 16 tokens)
```

Hình 11: Thông tin node Cassandra và trạng thái bộ nhớ theo mô hình LSM-Tree

Apache Cassandra không quản lý dữ liệu như một khối thống nhất, có thể thấy qua trạng thái 'gossip active' và biến 'token', hệ thống liên tục giao tiếp với các token

khi hoạt động. Ngoài ra, hệ thống còn hỗ trợ bộ nhớ đệm tầng thấp Key Cache và vùng nhớ ngoài tiến trình Off-heap memory để hỗ trợ việc tìm kiếm dữ liệu trên các cấu trúc SSTables bất biến thay vì quét tuần tự, bù đắp cho nhược điểm đọc chậm của cấu trúc LSM-Tree. Dung lượng lưu trữ thực tế 239.56 MiB không chỉ chứa dữ liệu thô mà còn bao gồm các thành phần hỗ trợ như Bloom Filter và hệ thống chỉ mục phân tán, giúp tối ưu hóa hiệu suất đọc mà không cần quét toàn bộ tệp tin dữ liệu.

MS SQL Server tập trung vào việc quản lý các page có kích thước cố định trong file mdf. Cấu trúc này được tổ chức theo mô hình B-Tree, tối ưu cho các truy vấn quan hệ phức tạp và đảm bảo tính toàn vẹn dữ liệu tại điểm tập trung.

Kết quả cho thấy sự khác biệt căn bản trong triết lý lưu trữ của hai hệ thống. SQL Server tổ chức dữ liệu với số lượng file cố định và tập trung, dung lượng tăng trưởng có kiểm soát, phản ánh khả năng mở rộng theo chiều dọc (Vertical Scaling) trên các hệ thống máy chủ đơn lẻ có cấu hình mạnh mẽ. Ngược lại, Cassandra phá vỡ khái niệm "file database" truyền thống, phân tán dữ liệu trên nhiều nút mạng. Kiến trúc này hỗ trợ khả năng mở rộng theo chiều ngang (Horizontal Scaling), tăng độ khả dụng.

4.1.5 Đánh giá và quyết định

Qua phân tích lý thuyết, thiết kế và kết quả thực nghiệm giữa MS SQL Server và Cassandra, nhóm chúng em nhận thấy hai hệ quản trị cơ sở dữ liệu này mang những đặc tính hoàn toàn khác biệt.

MS SQL Server tổ chức dữ liệu theo mô hình quan hệ, duy trì thứ tự logic toàn cục dựa trên chỉ mục cụm. Điều này giúp đảm bảo tính nhất quán dữ liệu tuyệt đối và khả năng truy vấn linh hoạt trên nhiều bảng thông qua các phép JOIN phức tạp. Tuy nhiên, việc phải tuân thủ nghiêm ngặt tính ACID và kiểm tra ràng buộc ngay khi ghi khiến hiệu suất lệnh bị giới hạn trên một node đơn lẻ.

Cassandra ưu tiên hiệu năng và khả năng mở rộng quy mô lớn. Sử dụng kiến trúc LSM-Tree và cơ chế chỉ ghi Append-only giúp hệ thống đạt tốc độ xử lý vượt trội khi thực hiện chèn dữ liệu hàng loạt nhờ tối ưu hóa việc ghi vào bộ nhớ đệm và commit log. Dữ liệu được thiết kế phi chuẩn hóa, chấp nhận dư thừa để tối ưu cho các truy vấn cụ thể. Khác với SQL Server, Cassandra sắp xếp dữ liệu theo cơ chế phân tán, không duy

trì thứ tự logic toàn cục nhằm đảm bảo tính khả dụng cao và khả năng mở rộng linh hoạt trên nhiều nút mạng.

So sánh tổng quát cho thấy không có hệ quản trị cơ sở dữ liệu nào là tối ưu tuyệt đối. MS SQL Server là lựa chọn lý tưởng cho các ứng dụng đòi hỏi tính toàn vẹn dữ liệu cao, quan hệ thực thể phức tạp và cấu trúc truy vấn thay đổi thường xuyên. Trong khi đó, Cassandra chiếm ưu thế rõ rệt trong các bài toán Big Data, yêu cầu tốc độ ghi cực nhanh, khả năng chịu lỗi cao và có mô hình truy vấn dữ liệu cố định. Việc lựa chọn hệ thống nào phụ thuộc vào sự đánh đổi giữa tính nhất quán dữ liệu và khả năng mở rộng của từng bài toán cụ thể.

4.2 Indexing

4.2.1 Khái niệm chung về Indexing

Lập chỉ mục (Indexing) trong cơ sở dữ liệu là một kỹ thuật được sử dụng để tăng tốc các thao tác truy xuất dữ liệu bằng cách giảm thiểu số lần truy cập đĩa cần thiết để định vị các bản ghi. Thay vì hệ thống phải thực hiện quét toàn bộ dữ liệu vật lý trên ổ cứng với độ phức tạp $O(n)$, Index cung cấp một cấu trúc tra cứu rút gọn kèm theo các con trỏ trực tiếp đến vị trí lưu trữ vật lý của bản ghi. Sự đánh đổi lớn nhất của Indexing là không gian lưu trữ bổ sung và sự suy giảm hiệu suất trong các thao tác ghi do hệ thống phải cập nhật đồng thời cả dữ liệu gốc và cấu trúc chỉ mục.

4.2.2 Đặc trưng cơ chế Indexing trong MS SQL Server

MS SQL Server đại diện cho mô hình cơ sở dữ liệu quan hệ truyền thống, sử dụng cấu trúc dữ liệu B+ Tree (Balanced) làm nền tảng lưu trữ chỉ mục (đối với Rowstore).

4.2.2.1 Tổng quan các loại Index trong MS SQL Server

Để đáp ứng đa dạng các kịch bản truy vấn, hệ thống cung cấp phong phú các loại chỉ mục:

- **Clustered Index (Chỉ mục cụm):**

- **Bản chất:** Quyết định thứ tự sắp xếp và cấu trúc lưu trữ vật lý của bảng. Dữ liệu thực tế nằm ở chính các node lá.
- **Cấu trúc & Cơ chế:** Bảng có Clustered Index được gọi là *Clustered table*. Nếu không có chỉ mục này, bảng được gọi là *Heap* (dữ liệu lưu trữ tự do không thứ tự).
- **Đặc điểm & Giới hạn:** Chỉ có thể có **duy nhất 1** Clustered Index trên mỗi bảng (do dữ liệu vật lý chỉ có thể được sắp xếp theo một thứ tự duy nhất tại một thời điểm).
- **Cách thức hình thành & Mở rộng:** Thường được SQL Server tự động tạo ra mặc định khi người dùng thiết lập ràng buộc khóa chính (PRIMARY KEY).

- **Nonclustered Index (Chỉ mục phi cụm):**

- **Bản chất:** Có cấu trúc tách biệt hoàn toàn với dữ liệu vật lý. Node lá không chứa dữ liệu gốc mà chỉ chứa khóa chỉ mục và một con trỏ (*row locator*).
- **Cấu trúc & Cơ chế:** Tùy cấu trúc bảng, con trỏ sẽ trỏ trực tiếp vào vị trí vật lý (đối với bảng Heap) hoặc trỏ vào khóa của Clustered Index (đối với Clustered table).
- **Đặc điểm & Giới hạn:** Không giới hạn như Clustered, một bảng có thể tạo **nhiều** Nonclustered Index để phục vụ các kịch bản tra cứu khác nhau.
- **Cách thức hình thành & Mở rộng:** Có thể thêm các cột phụ (*included columns*) vào cấp độ lá để cho phép truy xuất dữ liệu ngay tại chỉ mục mà không cần về bảng gốc (*fully covered queries*).

- **Unique Index (Chỉ mục duy nhất):**

- **Bản chất:** Đảm bảo tính toàn vẹn dữ liệu bằng cách tuyệt đối không cho phép bất kỳ giá trị trùng lặp nào tồn tại trong khóa chỉ mục.
- **Cấu trúc & Cơ chế:** Cung cấp quyền kiểm soát linh hoạt (IGNORE_DUP_KEY)

để bỏ qua dòng trùng lặp khi INSERT thay vì hủy (rollback). Lưu ý, hệ thống coi NULL là một giá trị thực, nên cột chỉ được phép chứa tối đa **một** giá trị NULL.

- **Đặc điểm & Giới hạn:** Nó không phải là một cấu trúc vật lý riêng biệt, mà là một "đặc tính" có thể áp đặt lên cả Clustered hoặc Nonclustered Index để bắt buộc sự duy nhất.
- **Cách thức hình thành & Mở rộng:** Tự động sinh ra khi tạo PRIMARY KEY hoặc UNIQUE. Nếu tạo độc lập bằng lệnh, nó hỗ trợ tính năng mạnh mẽ hơn Constraint như tạo chỉ mục có điều kiện (WHERE) để lách giới hạn NULL.

- **Columnstore Index (Chỉ mục lưu trữ theo cột):**

- **Bản chất:** Lưu trữ và quản lý dữ liệu theo định dạng cột (column-wise) thay vì dạng dòng truyền thống (row-wise). Đây là tiêu chuẩn tối ưu hàng đầu cho Data Warehouse (Kho dữ liệu) và các truy vấn phân tích.
- **Cấu trúc & Cơ chế:** Bảng được chia thành các *Rowgroup*. Mỗi cột trong Rowgroup được nén thành một *Column segment*. Sử dụng vùng đệm *Deltastore* (dạng B-tree) để giữ tạm các dòng dữ liệu mới chèn trước khi nén chuyển vào Columnstore. Truy vấn dùng cơ chế *Batch mode* (xử lý dữ liệu theo lô) giúp tăng tốc độ.
- **Đặc điểm & Giới hạn:** Khả năng nén dữ liệu cực cao (giảm 10 lần dung lượng) và giảm thiểu I/O tối đa. Phát huy sức mạnh vượt trội khi quét (*scan*) lượng dữ liệu khổng lồ, nhưng **không** phù hợp để tìm kiếm chính xác một giá trị đơn lẻ (*seek*) như Rowstore.
- **Cách thức hình thành & Mở rộng:** Bao gồm *Clustered Columnstore* (đóng vai trò là lưu trữ vật lý chính của toàn bảng) và *Nonclustered Columnstore* (chỉ mục phụ). Điểm ưu việt là có thể kết hợp Nonclustered Columnstore trên bảng Rowstore thông thường để thực hiện phân tích dữ liệu thời gian thực (Operational Analytics) ngay trên hệ thống giao dịch (OLTP).

- **Filtered Index:** Một Nonclustered index được tối ưu hóa, sử dụng điều kiện lọc để chỉ lập chỉ mục cho một tập con dữ liệu, giúp tiết kiệm dung lượng lưu trữ.
- **Index with included columns:** Mở rộng của Nonclustered index, cho phép đính kèm thêm các cột không phải khóa vào cấp lá để truy vấn có thể lấy ngay dữ liệu mà không cần vòng lại bảng gốc.
- **Các chỉ mục chuyên dụng khác:** Hash Index cho bảng in-memory, Spatial Index cho dữ liệu không gian, XML và Full-text Index.

4.2.2.2 Đặc điểm cốt lõi của các cơ chế Indexing trong MS SQL Server

- **Kiến trúc phân tầng rõ ràng:** Sự tách biệt giữa lưu trữ vật lý (Clustered) và cấu trúc tra cứu logic (Non-clustered) giúp MS SQL Server duy trì nền tảng dữ liệu ổn định trong khi vẫn có thể mở rộng nhiều hướng truy vấn khác nhau.
- **Tính linh hoạt cao:** Cho phép kết hợp nhiều Non-Clustered Index, Included Columns, hoặc Filtered Index để phục vụ các truy vấn lọc động phức tạp mà không cần can thiệp hay thay đổi cấu trúc dữ liệu gốc.
- **Hiệu suất:** Tối ưu hóa tuyệt vời cho các thao tác đọc quét dải và tìm kiếm chính xác với độ phức tạp $O(\log_m n)$ (với m là bậc của cây và n là số lượng phần tử). Tuy nhiên, chi phí ghi cao do hiện tượng phân mảnh và chi phí tổ chức, tái cân bằng lại cây B+ Tree khi dữ liệu thay đổi liên tục.

4.2.3 Đặc trưng cơ chế Indexing trong Apache Cassandra

Khác biệt hoàn toàn với SQL Server, Cassandra là một cơ sở dữ liệu NoSQL phân tán, thiết kế dựa trên kiến trúc LSM-Tree (Log-Structured Merge-Tree), tối ưu tuyệt đối cho tốc độ ghi dữ liệu $O(1)$ qua Memtable và SSTable.

4.2.3.1 Tổng quan các loại Index trong Apache Cassandra

Để phục vụ việc tra cứu trong môi trường phân tán, Cassandra cung cấp các cơ chế:

- **Primary Index (Chỉ mục chính):**

- **Bản chất:** Tương đương với khái niệm Primary Key, là cấu trúc gốc vừa dùng để định danh duy nhất một dòng, vừa quyết định cách dữ liệu được phân phối và lưu trữ vật lý trên toàn hệ thống cụm (cluster).
- **Cấu trúc & Cơ chế:** Hoạt động dựa trên sự kết hợp của 2 thành phần: *Partition Key* (dùng hàm băm để phân bổ dữ liệu về các node cụ thể) và *Clustering Columns* (sắp xếp thứ tự dữ liệu cục bộ bên trong từng node).
- **Đặc điểm & Giới hạn:** Điểm dị biệt là không có lỗi trùng khóa (nếu INSERT trùng sẽ tự động thành lệnh UPSERT ghi đè dữ liệu). Rất tối ưu cho truy vấn, nhưng có thể gây "nghẽn cổ chai" (*hotspot*) cục bộ nếu thiết kế phân vùng gom quá nhiều dữ liệu vào một node.
- **Cách thức hình thành & Mở rộng:** Khai báo bằng cú pháp PRIMARY KEY khi tạo bảng. Trong Cassandra, chỉ mục này bắt buộc phải được thiết kế "ngược" dựa trên các kịch bản câu lệnh truy vấn có sẵn (*Query-driven modeling*), khác hoàn toàn với cách thiết kế chuẩn hóa của cơ sở dữ liệu quan hệ (SQL).

- **Secondary Index (Chỉ mục phụ):**

- **Bản chất:** Là cơ chế lập chỉ mục phụ nguyên bản của Apache Cassandra, cho phép người dùng truy vấn dữ liệu thông qua các cột bình thường mà không cần sử dụng đến khóa phân vùng (partition key).
- **Cấu trúc & Cơ chế:** Chỉ mục này được xây dựng và duy trì cục bộ (locally built) trên từng node riêng biệt trong cụm (cluster). Quá trình khởi tạo và cập nhật diễn ra ngầm tự động, không gây chặn (block) các thao tác đọc/ghi dữ liệu.

- **Đặc điểm & Giới hạn:** Hiện nay 2i bị đánh giá là có hiệu suất kém và độ trễ cao (được khuyến nghị thay thế bằng SAI - Storage-Attached Index). Truy vấn 2i trên diện rộng thường bắt buộc phải dùng thêm chỉ thị ALLOW FILTERING. Ngoài ra, 2i không thể được sửa đổi (ALTER) mà phải xóa đi tạo lại, và không áp dụng được cho kiểu dữ liệu counter.
- **Cách thức hình thành & Mở rộng:** Khai báo bằng lệnh CREATE INDEX. Đặc biệt, nó hỗ trợ lập chỉ mục rất sâu trên các cấu trúc dữ liệu dạng bộ sưu tập (Collections): có thể index Set/List, index Map (theo KEYS, VALUES, hoặc ENTRIES), và thậm chí index toàn bộ nội dung của một collection đã đóng băng (FULL(FROZEN)).

- **Storage-Attached Indexing - SAI (Chỉ mục đính kèm lưu trữ):**

- **Bản chất:** Là giải pháp lập chỉ mục phân tán toàn cầu, có khả năng mở rộng cực cao, sinh ra để khắc phục triệt để các yếu điểm của 2i. Nhờ tích hợp sâu vào bộ lưu trữ cốt lõi của Cassandra, SAI hỗ trợ mạnh mẽ các truy vấn nâng cao và cả tìm kiếm vector cho các ứng dụng AI.
- **Cấu trúc & Cơ chế:** Không tạo ra các bảng ẩn công kênh, SAI lập chỉ mục trực tiếp lên *Memtables* (trên RAM) và *SSTables* (trên đĩa cứng) ngay tại thời điểm dữ liệu được ghi (*Write path*). Khi đọc (*Read path*), *SAI Coordinator* sẽ tự động phân tích và chọn ra chỉ mục có tính chọn lọc cao nhất, kết hợp cùng bộ khung *Token Flow* để duyệt và hợp nhất kết quả cực kỳ tối ưu.
- **Đặc điểm & Giới hạn:** Mang lại hiệu năng vượt trội so với mọi phương pháp Index khác trong Cassandra. Nó giảm thiểu tối đa dung lượng đĩa (TCO), tương thích hoàn toàn với tính năng truyền dữ liệu không cần sao chép (*Zero-copy streaming*), và không hề bị giảm hiệu suất khi xử lý các cột thường xuyên bị xóa (nhờ cơ chế *post-filtering*).
- **Cách thức hình thành & Mở rộng:** Được khởi tạo thông qua lệnh CREATE CUSTOM INDEX ... USING 'sai'. Đặc biệt ưu việt ở chỗ: nó cho phép sử dụng đồng thời **nhiều** chỉ mục SAI trong cùng một câu truy

vẫn duy nhất, hỗ trợ đa dạng toán tử từ khoảng giá trị số ($<$, $>$), đến chuỗi (LIKE), và cả bộ sưu tập (CONTAINS).

- **SASI (SSTable Attached Secondary Index):** Một dạng chỉ mục tùy chỉnh thế hệ cũ (hỗ trợ tìm kiếm văn bản), hiện tại đang bị thay thế dần bởi SAI.

4.2.3.2 Đặc điểm cốt lõi của các cơ chế Indexing trong Apache Cassandra

- **Primary Indexing (Partition Key & Clustering Key):** Là phương thức tra cứu chính. Cassandra sử dụng hàm băm lên Partition Key để phân phối và xác định chính xác dữ liệu nằm ở node nào trong cụm máy chủ. Dữ liệu sau đó được sắp xếp cục bộ trên node đó bằng Clustering Columns.
- **Secondary Indexes:** Do là các bảng ẩn cục bộ, khi hệ thống nhận được một truy vấn bằng 2i mà không có Partition Key đi kèm, bộ điều phối buộc phải phát đi truy vấn đến toàn bộ các node (hiện tượng Scatter-Gather), gây thất cổ chai và quá tải mạng, CPU nghiêm trọng.
- **Storage-Attached Indexing (SAI):** Cơ chế index hiện đại gắn trực tiếp vòng đời chỉ mục vào vòng đời của SSTable. Dù giải quyết được bài toán lưu trữ và cho phép linh hoạt hơn 2i, việc lọc dữ liệu diện rộng không kèm Partition Key vẫn đi ngược lại nguyên lý thiết kế phân tán, không mang lại hiệu suất cao cho ad-hoc queries toàn hệ thống.
- **Query-First Design:** Cơ chế lập chỉ mục trong Cassandra bị ràng buộc khắt khe bởi nguyên tắc "phải định nghĩa trước câu truy vấn rồi mới thiết kế bảng". Hệ thống cực kỳ hạn chế với các kịch bản truy vấn lọc đa chiều phát sinh ngẫu nhiên.

4.2.4 Kịch bản so sánh thực tế giữa SQL Server và Cassandra

4.2.4.1 Kịch bản 1: Truy xuất bằng (=) với khóa chính

Mục tiêu: Đánh giá tốc độ khi truy xuất 1 dòng duy nhất dựa trên khóa chính.

SQL Server:

```
1 SELECT * INTO #T1 FROM BankTransactions WHERE TransactionID = 1048548;
```

Cassandra:

```
1 SELECT COUNT(*) FROM bank_transactions WHERE transaction_id = '
  T1048548';
```

Kết quả:

- SQL Server:

```
=====
KỊCH BẢN 1: POINT LOOKUP (CLUSTERED INDEX)
=====

SQL Server Execution Times:
  CPU time = 5 ms,  elapsed time = 11 ms.
```

Hình 12: Kết quả thực thi Kịch bản 1 với SQL Server

- Cassandra:

```
=====
KỊCH BẢN 1: Point Lookup = 4.45 ms
-----
```

Hình 13: Kết quả thực thi Kịch bản 1 với Cassandra

Đánh giá: Với tập dữ liệu trên 1 triệu bản ghi, Cassandra xử lý Point Lookup nhanh hơn đáng kể (4.45 ms) so với mức 11 ms của SQL Server. SQL Server sử dụng Clustered Index tổ chức dưới dạng cấu trúc cây B-Tree, tốn một lượng thời gian nhất định để duyệt từ node gốc đến node lá để lấy dữ liệu. Trong khi đó, Cassandra dựa vào Partition Key kết hợp hàm băm để định vị chính xác vị trí dữ liệu trên kiến trúc phân tán của mình. Cơ chế này đem lại khả năng truy xuất với độ phức tạp tiệm cận $O(1)$, cực kỳ tối ưu cho các thao tác đọc đúng một dòng.

4.2.4.2 Kịch bản 2: Truy xuất trên cột chưa đánh index

Mục tiêu: Quan sát hành vi của DBMS khi người dùng cố tình tìm kiếm trên một cột không được tối ưu (CustomerID).

SQL Server:

```
1 SELECT * INTO #T2 FROM BankTransactions WHERE CustomerID = 'C5674116';
```

Cassandra:

```
1 SELECT COUNT(*) FROM bank_transactions WHERE customer_id = 'C5674116'  
ALLOW FILTERING;
```

Kết quả:

- SQL Server:

```
=====
KỊCH BẢN 2: FULL TABLE SCAN (NO INDEX)
=====

SQL Server Execution Times:
  CPU time = 7 ms,  elapsed time = 12 ms.
```

Hình 14: Kết quả thực thi Kịch bản 2 với SQL Server

- Cassandra:

```
-----
KỊCH BẢN 2: Table Scan = 12.68 ms
-----
```

Hình 15: Kết quả thực thi Kịch bản 2 với Cassandra

Đánh giá: Khi không có chỉ mục, cả hai hệ quản trị đều bị ép phải quét diện rộng. SQL Server mất 12 ms cho thao tác Table Scan duyệt tuần tự qua các bản ghi. Cassandra tốn 12.68 ms khi dùng chỉ thị `ALLOW FILTERING`. Mặc dù thời gian trả về ở thử nghiệm này tương đồng, nhưng về mặt kiến trúc, `ALLOW FILTERING` trên Cassandra bắt buộc quét toàn bộ các phân vùng. Với lượng dữ liệu khổng lồ, hiệu suất của Cassandra trong trường hợp này sẽ xuống dốc không phanh do phải đọc qua nhiều node, vốn là rào cản chí mạng trong hệ thống phân tán.

4.2.4.3 Kịch bản 3: Đánh chỉ mục phụ trên cột thường

Mục tiêu: Cải thiện tốc độ truy vấn cho kịch bản 2 bằng cách đánh chỉ mục.

SQL Server:

```
1 IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'idx_customer')
2 CREATE NONCLUSTERED INDEX idx_customer ON BankTransactions(CustomerID)
3 ;
4 SELECT * INTO #T3 FROM BankTransactions WHERE CustomerID = 'C5674116';
```

Cassandra:

```
1 CREATE INDEX IF NOT EXISTS idx_customer_cas ON bank_transactions(
  customer_id);
2 SELECT COUNT(*) FROM bank_transactions WHERE customer_id = 'C5674116';
```

Kết quả:

- SQL Server:

=====

KỊCH BẢN 3: NONCLUSTERED INDEX (SECONDARY INDEX)

=====

SQL Server Execution Times:
CPU time = 6 ms, elapsed time = 9 ms.

Hình 16: Kết quả thực thi Kịch bản 3 với SQL Server

- Cassandra:

KỊCH BẢN 3: Secondary Index (2i) = 6.83 ms

Hình 17: Kết quả thực thi Kịch bản 3 với Cassandra

Đánh giá: Khi thiết lập chỉ mục phụ, SQL Server giảm thời gian truy vấn xuống 9 ms bằng cách sử dụng Nonclustered Index chứa con trỏ dẫn về Clustered Index để lấy dữ liệu. Cassandra lại cho thấy sự tối ưu nhỉnh hơn (6.83 ms) khi dùng Secondary Index (2i). Index này xây dựng một cấu trúc dữ liệu cục bộ ngay trên từng node, loại bỏ được việc phải quét toàn bộ các mảnh dữ liệu (SSTables) như kịch bản trước, giải quyết hiệu quả truy vấn lọc theo một giá trị trên cột thông thường.

4.2.4.4 Kịch bản 4: Truy vấn theo khoảng giá trị

Mục tiêu: Lấy các giao dịch có số tiền lớn hơn 50.000.

SQL Server:

```
1 IF NOT EXISTS (SELECT * FROM sys.indexes WHERE name = 'idx_amount')
2 CREATE NONCLUSTERED INDEX idx_amount ON BankTransactions(
    TransactionAmount);
3
4 SELECT * INTO #T4 FROM BankTransactions WHERE TransactionAmount >
    50000;
```

Cassandra:

```
1 CREATE CUSTOM INDEX IF NOT EXISTS idx_amount_sai ON bank_transactions(
    transaction_amount) USING 'sai';
2 SELECT COUNT(*) FROM bank_transactions WHERE transaction_amount >
    50000;
```

Kết quả:

- SQL Server:

=====

KỊCH BẢN 4: RANGE QUERY VOI INDEX

=====

SQL Server Execution Times:

CPU time = 128 ms, elapsed time = 274 ms.

Hình 18: Kết quả thực thi Kịch bản 4 với SQL Server

- Cassandra:

KỊCH BẢN 4: Range Query (SAI) = 1548.13 ms

Hình 19: Kết quả thực thi Kịch bản 4 với Cassandra

Đánh giá: Đặc tính cốt lõi của mỗi loại Database lộ rõ ở kịch bản Range Query. SQL Server mất 274 ms nhờ vào cấu trúc cây B-Tree với các node lá được liên kết vòng.

Dữ liệu được sắp xếp có thứ tự cho phép SQL Server quét ngang các khoảng giá trị vô cùng trơn tru. Ngược lại, dù đã dùng Storage Attached Index (SAI), Cassandra vẫn mất tới 1548.13 ms. Cấu trúc dữ liệu LSM-Tree cùng phương pháp băm dữ liệu rải rác trên nhiều node khiến Cassandra tốn kém chi phí để gom tụ lại các bản ghi nằm trong một dải giá trị, làm cho truy vấn tốn thời gian hơn gấp nhiều lần.

4.2.4.5 Kịch bản 5: Truy vấn có Group by

Mục tiêu: Khả năng báo cáo thống kê.

SQL Server:

```
1 SELECT CustLocation, COUNT(*) as Count, SUM(TransactionAmount) as  
   Total  
2 INTO #T5  
3 FROM BankTransactions GROUP BY CustLocation;
```

Cassandra:

```
1 DROP MATERIALIZED VIEW IF EXISTS transactions_by_location;  
2  
3 CREATE MATERIALIZED VIEW transactions_by_location AS  
4 SELECT cust_location, transaction_id, transaction_amount  
5 FROM bank_transactions  
6 WHERE cust_location IS NOT NULL  
7    AND transaction_id IS NOT NULL  
8 PRIMARY KEY (cust_location, transaction_id);  
9  
10 SELECT cust_location, SUM(transaction_amount)  
11 FROM transactions_by_location  
12 GROUP BY cust_location LIMIT 10;
```

Kết quả:

- SQL Server:

```
=====
KỊCH BẢN 5: ANALYTICAL QUERY (GROUP BY)
=====
```

```
SQL Server Execution Times:
  CPU time = 652 ms,  elapsed time = 357 ms.
```

Hình 20: Kết quả thực thi Kịch bản 5 với SQL Server

- Cassandra:

```
-----
KỊCH BẢN 5a: Thời gian KHOI TAO View (Chỉ phí an) = 294.79 ms
-----
KỊCH BẢN 5b: Thời gian TRUY VAN qua View = 4.71 ms
-----
```

Hình 21: Kết quả thực thi Kịch bản 5 với Cassandra

Đánh giá: Thực hiện các thao tác GROUP BY trên cơ sở dữ liệu lưu theo hàng như SQL Server tốn khá nhiều chi phí duyệt, mất tổng cộng 357 ms. Cassandra dùng phương pháp Materialized View - đòi hỏi một chi phí khởi tạo và tính toán ẩn cực kỳ lớn lúc bắt đầu tạo view (lên tới 294.79 ms). Tuy nhiên, điểm ăn tiền là sau khi view được xây dựng xong, các truy vấn sau đó đọc thẳng dữ liệu đã được tính sẵn, kéo thời gian trả về xuống cực kỳ nhanh, chỉ còn 4.71 ms.

4.2.4.6 Kịch bản 6: Truy vấn có Group by với ColumnStore Index trong SQL Server

Mục tiêu: Minh chứng tốc độ truy vấn khi dùng ColumnStore Index.

SQL Server:

```
1 -- Tao bang moi
2 IF OBJECT_ID('BankTransactions_Columnar', 'U') IS NOT NULL
3     DROP TABLE BankTransactions_Columnar;
4
5 SELECT * INTO BankTransactions_Columnar FROM BankTransactions;
6
```

```
7 -- Danh chi muc column store
8 CREATE CLUSTERED COLUMNSTORE INDEX CCI_Bank_Transactions
9     ON BankTransactions_Columnar;
10 GO
11
12 -- Xoa cache
13 CHECKPOINT;
14 DBCC DROPCLEANBUFFERS WITH NO_INFOMSGS;
15 DBCC FREEPROCCACHE WITH NO_INFOMSGS;
16
17 -- Truy van voi group by
18 SET STATISTICS TIME ON;
19 SELECT CustLocation, COUNT(*) as Count, SUM(TransactionAmount) as
    Total
20 INTO #T6
21 FROM BankTransactions_Columnar GROUP BY CustLocation;
22 SET STATISTICS TIME OFF;
23 GO
```

Kết quả:

- SQL Server:

```
=====
KICH BAN 6: COLUMNSTORE INDEX (ANALYSIS OPTIMIZATION)
=====

SQL Server Execution Times:
    CPU time = 79 ms,  elapsed time = 96 ms.
```

Hình 22: Kết quả thực thi Kịch bản 6 với SQL Server

Đánh giá: Đây là lợi thế đặc trưng của SQL Server khi chuyển sang cơ chế đánh chỉ mục chuyên phân tích: ColumnStore Index. Thay vì lưu theo hàng, dữ liệu được tổ chức theo từng cột độc lập. Điều này giúp hệ thống chỉ nạp chính xác các cột cần thiết (CustLocation, TransactionAmount) vào bộ nhớ RAM, bỏ qua hàng loạt các thông tin dư thừa. Nhờ vậy, thao tác tính toán SUM và GROUP BY được nén I/O tối đa, kéo tốc

độ truy vấn giảm đột phá từ 357 ms xuống chỉ còn 96 ms, cực kỳ mạnh mẽ đối với các truy vấn kho dữ liệu và OLAP.

4.2.5 Đánh giá và quyết định

Dựa trên các đặc trưng kiến trúc nêu trên, MS SQL Server là lựa chọn tối ưu và vượt trội hơn hẳn so với Apache Cassandra đối với dự án phát triển nền tảng kết nối Mentor. Các luận điểm chứng minh bao gồm:

- **Bản chất truy vấn của hệ thống là Ad-hoc Filtering:** Tính năng cốt lõi của dự án là cho phép Mentee tìm kiếm Mentor dựa trên nhiều bộ lọc tùy ý và kết hợp linh hoạt. Cassandra sẽ thất bại ở bài toán này vì Mentee không thể biết trước Mentor_ID (Partition Key). Việc dùng Secondary Index để lọc các tiêu chí này sẽ ép Cassandra phải rà soát toàn bộ hệ thống phân tán, gây tắc nghẽn nghiêm trọng. Để khắc phục, lập trình viên sẽ phải nhân bản dữ liệu ra vô số bảng khác nhau, dẫn đến phình to dữ liệu. MS SQL Server giải quyết bài toán này một cách triệt để thông qua sức mạnh của Non-Clustered Indexes kết hợp với Included Columns, cho phép truy xuất kết quả tìm kiếm đa chiều tính bằng mili-giây.
- **Đảm bảo tính toàn vẹn Giao dịch:** Quá trình Mentee đặt lịch hoặc thanh toán yêu cầu tính nhất quán dữ liệu tuyệt đối. Cơ chế khóa và quản lý giao dịch của MS SQL Server đảm bảo không xảy ra tình trạng xung đột, ví dụ như hai người cùng đặt một khung giờ của một Mentor. Cassandra với cơ chế Eventual Consistency không được thiết kế cho loại nghiệp vụ nhạy cảm này.

Kết luận: Kiến trúc B-Tree linh hoạt kết hợp với môi trường giao dịch của MS SQL Server đáp ứng hoàn hảo tính chất dữ liệu quan hệ sâu và nhu cầu truy vấn đa chiều động của nền tảng kết nối Mentor.

4.3 Query processing

4.3.1 Khái niệm chung về xử lý truy vấn

Xử lý truy vấn là toàn bộ chuỗi hoạt động mà một hệ quản trị cơ sở dữ liệu (DBMS) thực thi nhằm chuyển đổi một câu lệnh SQL cấp cao thành kết quả dữ liệu cụ thể. Thay vì thực thi trực tiếp câu lệnh SQL, DBMS triển khai một pipeline gồm nhiều giai đoạn kế tiếp nhau, bao gồm phân tích, biến đổi và tối ưu hóa, nhằm đảm bảo hiệu năng và tính đúng đắn của kết quả.

Theo Elmasri và Navathe, một truy vấn SQL khi được đưa vào hệ thống sẽ trải qua bốn giai đoạn chính. Trước hết, ở giai đoạn phân tích cú pháp và dịch, câu lệnh SQL được kiểm tra tính hợp lệ và chuyển đổi sang biểu diễn nội bộ dưới dạng đại số quan hệ. Tiếp theo, trong giai đoạn tối ưu hóa, hệ thống sinh ra nhiều kế hoạch thực thi khả dĩ và lựa chọn phương án có chi phí ước tính thấp nhất. Kế hoạch này sau đó được cụ thể hóa ở giai đoạn sinh kế hoạch thực thi, dưới dạng một cây toán tử kèm theo các chiến lược truy cập dữ liệu tương ứng. Cuối cùng, kế hoạch được thực thi để truy xuất dữ liệu và trả kết quả cho người dùng.

Trong quá trình thực thi này, cách thức đánh giá các biểu thức đại số quan hệ đóng vai trò quan trọng đối với hiệu năng tổng thể. Các DBMS hiện đại thường kết hợp hai chiến lược chính. Với materialized evaluation, kết quả trung gian của mỗi toán tử được ghi xuống bộ nhớ thứ cấp trước khi chuyển sang bước tiếp theo, đảm bảo tính ổn định nhưng phát sinh chi phí I/O đáng kể. Ngược lại, pipelined evaluation cho phép truyền trực tiếp các dòng dữ liệu giữa các toán tử mà không cần vật liệu hóa toàn bộ kết quả trung gian, từ đó giảm độ trễ và nhu cầu bộ nhớ, đặc biệt hiệu quả đối với các truy vấn phức tạp.

Việc lựa chọn chiến lược thực thi phù hợp phụ thuộc trực tiếp vào cơ chế ước tính chi phí của hệ thống. Chi phí này được tính toán dựa trên nhiều yếu tố, bao gồm chi phí truy cập bộ nhớ thứ cấp (số khối đĩa cần đọc/ghi), chi phí xử lý CPU và dung lượng bộ nhớ cần thiết cho các cấu trúc trung gian. Các thông số thống kê như số bản ghi, số khối, hệ số chặn, số giá trị phân biệt và độ chọn lọc của thuộc tính được lưu trữ trong System Catalog đóng vai trò là đầu vào quan trọng cho các mô hình ước tính chi phí

này.

Từ nền tảng ước tính chi phí, DBMS áp dụng hai hướng tiếp cận tối ưu hóa chính. Tối ưu hóa dựa trên heuristic sử dụng các quy tắc biến đổi đại số nhằm rút gọn không gian tìm kiếm, chẳng hạn như đẩy các phép chọn và chiếu xuống gần các toán hạng gốc để giảm kích thước dữ liệu trung gian trước khi thực hiện các phép kết nối. Trong khi đó, tối ưu hóa dựa trên chi phí thực hiện việc liệt kê và đánh giá nhiều kế hoạch thực thi khác nhau, từ đó lựa chọn phương án tối ưu; đây là cơ chế cốt lõi được triển khai trong các hệ quản trị quan hệ như MS SQL Server.

Tuy nhiên, cách tiếp cận này không phải là phổ quát cho mọi hệ cơ sở dữ liệu. Đối với Cassandra – một hệ quản trị cơ sở dữ liệu NoSQL phân tán – khái niệm xử lý truy vấn được định nghĩa theo cách tiếp cận khác biệt so với các hệ quản trị quan hệ truyền thống. Cassandra không cung cấp bộ tối ưu hóa truy vấn theo nghĩa cổ điển. Thay vào đó, trách nhiệm tối ưu hóa được dịch chuyển sang giai đoạn thiết kế lược đồ dữ liệu. Cụ thể, lập trình viên cần xác định trước các mẫu truy vấn và thiết kế khóa phân vùng cùng khóa phân cụm sao cho mỗi truy vấn trong môi trường vận hành chỉ cần truy cập vào một phân vùng duy nhất. Cách tiếp cận này giúp loại bỏ hoàn toàn nhu cầu thực hiện các thao tác phân tán đa phân vùng, từ đó tối ưu hóa hiệu năng và độ trễ của hệ thống.

4.3.2 Đặc trưng cơ chế xử lý truy vấn trong MS SQL Server

MS SQL Server triển khai một pipeline xử lý truy vấn toàn diện, được điều phối bởi bộ tối ưu hóa truy vấn – một trong những thành phần cốt lõi và phức tạp nhất của hệ thống.

Giai đoạn phân tích và dịch: Khi một câu lệnh SQL được gửi đến hệ thống, thành phần Parser thực hiện kiểm tra tính hợp lệ về cú pháp và ngữ nghĩa. Sau đó, Algebrizer chuyển đổi câu lệnh này thành một cây đại số quan hệ nội bộ.

Mỗi khối truy vấn, bao gồm mệnh đề **SELECT-FROM-WHERE** và các thành phần tùy chọn như **GROUP BY**, **HAVING**, được xử lý như một đơn vị tối ưu hóa độc lập. Các truy vấn lồng được nhận diện như các query block riêng biệt và có thể được biến đổi thành các phép kết (JOIN) tương đương, qua đó mở rộng không gian tối ưu hóa.

Giai đoạn tối ưu hóa dựa trên chi phí: Bộ tối ưu hóa truy vấn của SQL Server là một bộ tối ưu hóa dựa trên chi phí. Hệ thống duy trì các số liệu thống kê được cập nhật định kỳ về phân bố dữ liệu trong từng cột và chỉ mục, cho phép ước tính độ chọn lọc của các vị từ trong mệnh đề **WHERE**.

Dựa trên các thông tin này, optimizer sinh ra nhiều kế hoạch thực thi khả dĩ. Mỗi kế hoạch là một tổ hợp của:

- Phương thức truy cập dữ liệu: Clustered Index Seek, Index Scan, Table Scan.
- Thuật toán kết: Nested Loop Join, Merge Join, Hash Match Join.
- Thứ tự thực hiện các phép kết.

Kế hoạch có tổng chi phí ước tính nhỏ nhất sẽ được lựa chọn để thực thi.

Chi phí truy cập và các chiến lược thực thi: Đối với các truy vấn **SELECT** đơn giản trên khóa chính, SQL Server thường sử dụng Clustered Index Seek với chi phí truy cập xấp xỉ:

$$C_{S3a} = x + 1$$

trong đó x là số cấp của cây B^+ -tree.

Đối với các truy vấn theo khoảng giá trị trên cột đã được lập chỉ mục, hệ thống sử dụng Index Range Scan, thực hiện duyệt tuần tự các node lá của cây B^+ -tree. Đây là một lợi thế cấu trúc quan trọng của mô hình Rowstore so với LSM-Tree trong các hệ NoSQL như Cassandra.

Trong các phép kết phức tạp, SQL Server lựa chọn thuật toán phù hợp dựa trên đặc điểm dữ liệu:

- **Nested Loop Join:** phù hợp khi một trong hai quan hệ có kích thước nhỏ hoặc có index hỗ trợ. Chi phí có thể được ước lượng như sau:

$$C_{J1} = b_R + b_R \cdot b_S + C_{\text{output}}$$

- **Merge Join:** hiệu quả khi cả hai quan hệ đã được sắp xếp theo thuộc tính kết.
- **Hash Match Join:** được sử dụng khi xử lý các tập dữ liệu lớn không có

index phù hợp, với chi phí I/O tương đối cân bằng.

Plan Caching: Sau khi một kế hoạch thực thi được sinh ra, SQL Server lưu trữ kế hoạch này trong Plan Cache. Các lần thực thi tiếp theo của cùng một stored procedure hoặc parameterized query có thể tái sử dụng kế hoạch đã biên dịch, từ đó loại bỏ hoàn toàn chi phí của giai đoạn tối ưu hóa. Cơ chế này giải thích tại sao các stored procedures (ví dụ như `sp_SearchMentors`, `sp_DashboardStatistics`) thường đạt hiệu năng vượt trội so với các truy vấn ad-hoc trong môi trường có tải cao.

Pipelining và thực thi song song: SQL Server hỗ trợ thực thi song song bằng cách phân chia dữ liệu thành nhiều luồng xử lý đồng thời. Đồng thời, cơ chế pipelined evaluation được áp dụng xuyên suốt trong cây thực thi: dữ liệu đầu ra của một toán tử (ví dụ Index Scan) được truyền trực tiếp sang toán tử kế tiếp (ví dụ Hash Join) mà không cần vật liệu hóa toàn bộ kết quả trung gian. Cách tiếp cận này giúp giảm đáng kể chi phí bộ nhớ và cải thiện độ trễ phản hồi, đặc biệt đối với các truy vấn có độ phức tạp cao.

4.3.3 Đặc trưng cơ chế xử lý truy vấn trong Apache Cassandra

Apache Cassandra tiếp cận bài toán xử lý truy vấn theo một triết lý hoàn toàn khác so với các hệ quản trị cơ sở dữ liệu quan hệ truyền thống. Sự khác biệt này xuất phát từ bản chất của một hệ thống phân tán ưu tiên tính sẵn sàng và khả năng chịu lỗi phân tán, chấp nhận đánh đổi một phần tính nhất quán để đạt được khả năng mở rộng và độ ổn định cao.

Trung tâm của quá trình xử lý truy vấn trong Cassandra là nút điều phối (Coordinator). Khi ứng dụng gửi một câu lệnh truy vấn đến bất kỳ nút nào trong cụm, nút đó sẽ tạm thời đóng vai trò điều phối. Nó sử dụng khóa phân vùng để tính toán giá trị băm, từ đó xác định chính xác dữ liệu nằm ở nút nào trong hệ thống.

Nếu truy vấn có đầy đủ khóa phân vùng, nút điều phối chỉ cần gửi yêu cầu đến một nhóm nhỏ các nút lưu bản sao dữ liệu tương ứng. Đây được gọi là truy vấn trên một phân vùng duy nhất, và là trường hợp tối ưu nhất với chi phí gần như hằng số. Toàn bộ Cassandra được thiết kế xoay quanh việc đảm bảo các truy vấn thực tế đều rơi vào tình huống này.

Ngược lại, nếu truy vấn không chỉ rõ khóa phân vùng hoặc yêu cầu lọc dữ liệu không phù hợp với cấu trúc lưu trữ, hệ thống sẽ không thể tối ưu. Do Cassandra không có bộ tối ưu truy vấn như trong SQL Server, nó không thể tự sắp xếp lại điều kiện lọc, cũng không hỗ trợ phép nối bảng (JOIN). Khi đó, nút điều phối buộc phải gửi yêu cầu đến toàn bộ các nút trong cụm. Mỗi nút sẽ tự quét dữ liệu của mình rồi trả kết quả về để tổng hợp. Cách làm này khiến chi phí tăng theo số lượng nút, đồng thời gây tắc nghẽn mạng và tiêu tốn tài nguyên xử lý khi hệ thống lớn.

Một điểm khác biệt quan trọng nữa nằm ở cách Cassandra xử lý ghi và đọc dữ liệu. Đối với ghi dữ liệu, hệ thống được tối ưu rất mạnh: mỗi thao tác ghi chỉ đơn giản là ghi thêm vào cuối nhật ký và lưu vào bộ nhớ tạm, không có cập nhật trực tiếp trên dữ liệu cũ. Nhờ vậy, việc ghi diễn ra rất nhanh và ổn định.

Tuy nhiên, điều này khiến việc đọc dữ liệu trở nên phức tạp hơn. Dữ liệu có thể nằm rải rác trong nhiều tệp khác nhau trên đĩa (gọi là SSTable). Khi đọc, Cassandra phải kiểm tra nhiều tệp, loại bỏ những tệp chắc chắn không chứa dữ liệu nhờ bộ lọc Bloom, sau đó ghép các kết quả lại với nhau và xử lý các bản ghi đã bị đánh dấu xóa. Nếu dữ liệu chưa được gom lại, việc đọc sẽ chậm hơn đáng kể do phải xử lý quá nhiều tệp.

Do không hỗ trợ các truy vấn phức tạp như nối bảng hay gom nhóm linh hoạt, Cassandra buộc người thiết kế phải thay đổi cách tổ chức dữ liệu. Thay vì lưu dữ liệu theo dạng chuẩn hóa như trong SQL, Cassandra thường sử dụng phi chuẩn hóa dữ liệu, tức là chấp nhận lưu trùng lặp dữ liệu để phục vụ các truy vấn cụ thể.

Ngoài ra, hệ thống còn hỗ trợ bảng nhìn vật lý hóa (materialized view), cho phép tạo sẵn một phiên bản dữ liệu đã được sắp xếp lại theo một khóa phân vùng khác. Việc xây dựng các bảng này tốn nhiều chi phí ghi và đồng bộ, nhưng đổi lại, truy vấn đọc trên đó sẽ rất nhanh vì chỉ cần truy cập một phân vùng duy nhất.

Tóm lại, Cassandra đánh đổi tính linh hoạt trong truy vấn để đạt được hiệu năng cao và khả năng mở rộng tuyến tính. Khác với SQL Server – nơi hệ thống tự động tối ưu truy vấn khi chạy – Cassandra yêu cầu việc tối ưu phải được thực hiện ngay từ lúc thiết kế dữ liệu. Vì vậy, hiệu năng của hệ thống phụ thuộc rất lớn vào cách lựa chọn khóa và tổ chức schema của lập trình viên.

4.3.4 Kịch bản so sánh thực tế giữa SQL Server và Cassandra

4.3.4.1 Kịch bản 1: Hiệu ứng plan cache

SQL Server có cơ chế lưu và tái sử dụng kế hoạch thực thi truy vấn. Vì vậy, ở lần chạy đầu tiên, câu truy vấn phải đi qua toàn bộ quy trình gồm phân tích cú pháp, kiểm tra ngữ nghĩa, tối ưu truy vấn, biên dịch và thực thi. Những lần chạy sau có thể dùng lại kế hoạch đã được biên dịch sẵn nên giảm đáng kể chi phí xử lý. Trong khi đó, Cassandra không có cơ chế biên dịch truy vấn như vậy; mỗi truy vấn chủ yếu chỉ tốn chi phí định tuyến trước khi được xử lý.

SQL Server:

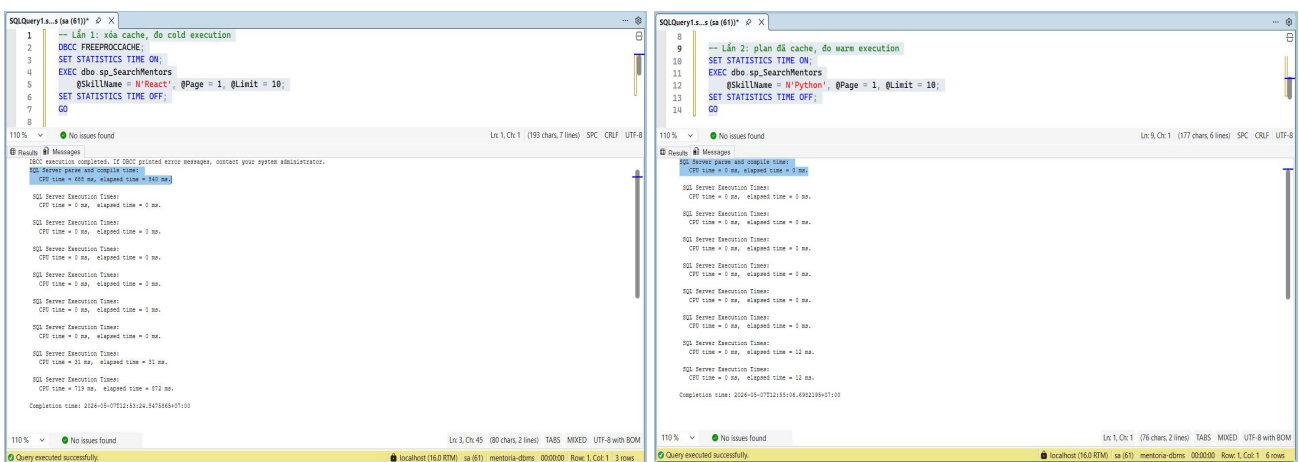
```
1 -- Lần 1: xóa cache, đo cold execution
2 DBCC FREEPROCCACHE;
3 SET STATISTICS TIME ON;
4 EXEC dbo.sp_SearchMentors
5     @SkillName = N'React', @Page = 1, @Limit = 10;
6 SET STATISTICS TIME OFF;
7 GO
8
9 -- Lần 2: plan đã cache, đo warm execution
10 SET STATISTICS TIME ON;
11 EXEC dbo.sp_SearchMentors
12     @SkillName = N'Python', @Page = 1, @Limit = 10;
13 SET STATISTICS TIME OFF;
14 GO
```

Cassandra:

```
1 -- Lần 1
2 TRACING ON;
3 SELECT mentor_id, first_name, last_name, rating
4 FROM mentors_by_skill
5 WHERE skill_name = 'React'
6 LIMIT 10;
7 TRACING OFF;
8
```

```
9 -- Lần 2: cùng cấu trúc query, param khác - Cassandra vẫn parse lại (
    không có plan cache)
10 TRACING ON;
11 SELECT mentor_id, first_name, last_name, rating
12 FROM mentors_by_skill
13 WHERE skill_name = 'Python'
14 LIMIT 10;
15 TRACING OFF;
```

Kết quả:

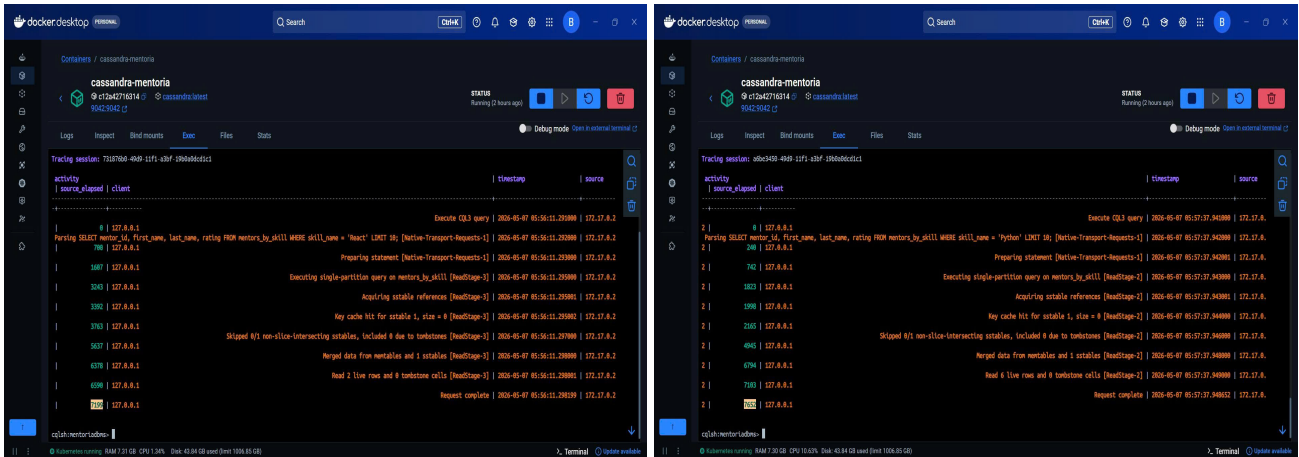


(a) Chạy lần 1

(b) Chạy lần 2

Hình 23: Kết quả thực thi Kịch bản 1 với SQL Server

Kết quả đo cho thấy ở lần thực thi đầu tiên, SQL Server phát sinh thời gian biên dịch truy vấn (compile time > 0), trong khi ở những lần gọi sau, thời gian này gần như bằng 0 nhờ cơ chế tái sử dụng execution plan từ Plan Cache. Đây là bằng chứng trực tiếp cho cơ chế cache kế hoạch thực thi của SQL Server.



(a) Chạy lần 1

(b) Chạy lần 2

Hình 24: Kết quả thực thi Kịch bản 1 với Cassandra

Ngược lại, Cassandra không tồn tại khái niệm biên dịch truy vấn theo kiểu đó. Coordinator chỉ thực hiện định tuyến request dựa trên token của partition key, không xây dựng query tree, không có optimizer và cũng không tạo execution plan phức tạp.

Sự khác biệt này phản ánh rõ tư duy thiết kế của hai hệ quản trị: SQL Server chấp nhận trả chi phí tối ưu và biên dịch một lần để đổi lấy hiệu năng tốt hơn ở các lần thực thi tiếp theo, còn Cassandra loại bỏ hoàn toàn lớp xử lý đó để giảm overhead, nhưng đồng thời cũng không hưởng được lợi ích từ việc tối ưu truy vấn.

4.3.4.2 Kịch bản 2: Pipeline biên dịch và thực thi truy vấn

Ở góc độ kỹ thuật, đây là một kịch bản mang tính định tính nhằm so sánh trực tiếp toàn bộ pipeline xử lý của một truy vấn nghiệp vụ phức tạp trên hai hệ thống. Mục tiêu không nằm ở việc đo thời gian thực thi theo mili giây, mà ở việc phân tích số lượng bước xử lý, khả năng tối ưu hóa truy vấn, cũng như điểm mà mỗi hệ quản trị thành công hoặc gặp giới hạn kiến trúc.

Một truy vấn nghiệp vụ thực tế từ hệ thống Mentoria được chọn làm đối tượng so sánh và biểu diễn trên cả SQL Server lẫn Cassandra: "Tìm tất cả Mentor có kỹ năng React, rating từ 4.0 trở lên, thuộc quốc gia United States, đồng thời đã có ít nhất một phiên mentoring ở trạng thái Completed trong vòng 365 ngày gần nhất; kết quả được sắp xếp theo doanh thu giảm dần."

Kịch bản này cho phép quan sát rõ sự khác biệt giữa mô hình xử lý truy vấn quan

hệ của SQL Server - nơi truy vấn được phân tích, tối ưu và xây dựng execution plan
- với Cassandra, nơi truy vấn phải được thiết kế xoay quanh partition key và mô hình truy cập dữ liệu được xác định trước.

SQL Server:

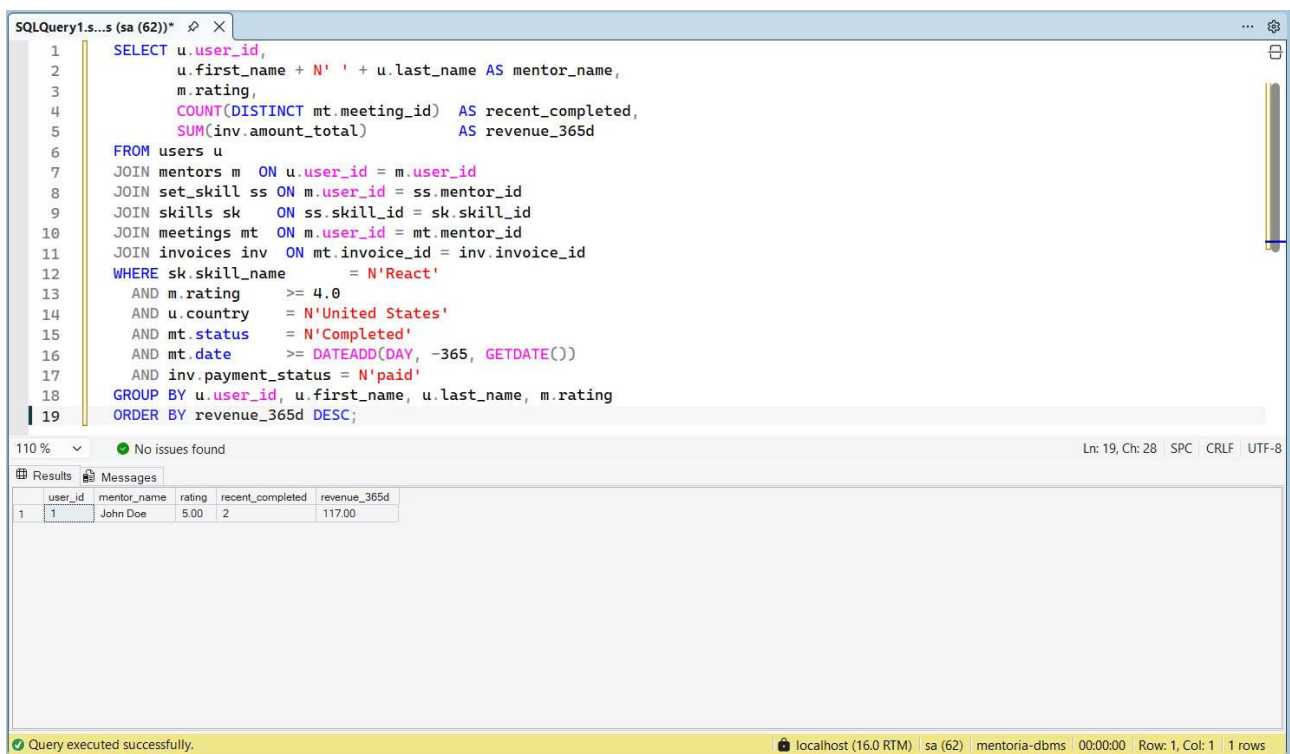
```
1 SELECT u.user_id,  
2       u.first_name + N' ' + u.last_name AS mentor_name,  
3       m.rating,  
4       COUNT(DISTINCT mt.meeting_id) AS recent_completed,  
5       SUM(inv.amount_total) AS revenue_365d  
6 FROM users u  
7 JOIN mentors m ON u.user_id = m.user_id  
8 JOIN set_skill ss ON m.user_id = ss.mentor_id  
9 JOIN skills sk ON ss.skill_id = sk.skill_id  
10 JOIN meetings mt ON m.user_id = mt.mentor_id  
11 JOIN invoices inv ON mt.invoice_id = inv.invoice_id  
12 WHERE sk.skill_name = N'React'  
13        AND m.rating >= 4.0  
14        AND u.country = N'United States'  
15        AND mt.status = N'Completed'  
16        AND mt.date >= DATEADD(DAY, -365, GETDATE())  
17        AND inv.payment_status = N'paid'  
18 GROUP BY u.user_id, u.first_name, u.last_name, m.rating  
19 ORDER BY revenue_365d DESC;
```

Cassandra:

```
1 -- Bước 1: Lọc mentor theo skill và rating  
2 SELECT mentor_id, first_name, last_name, rating, country, status  
3 FROM mentors_by_skill  
4 WHERE skill_name = 'React'  
5        AND rating >= 4.0  
6 LIMIT 100;  
7  
8 -- Bước 2: Với mỗi mentor_id (đã lọc country + status ở app), lấy các  
9        meeting Completed trong 365 ngày  
9 SELECT meeting_id, meeting_date, status, amount_paid
```

```
10 FROM meetings_by_mentor
11 WHERE mentor_id = 00000000-0000-0000-0000-000000000001 -- placeholder
    value
12 AND meeting_date >= '2025-01-07'
13 AND status = 'Completed'
14 ALLOW FILTERING;
15
16 -- Bước 3: Đọc doanh thu từ bảng pre-aggregated thay cho SUM(inv.
    amount_total) trong SQL Server
17 SELECT mentor_id, mentor_name, total_revenue, completed_meetings
18 FROM revenue_by_mentor
19 WHERE bucket = 'all'
20 ORDER BY total_revenue DESC
21 LIMIT 50;
```

Kết quả:



The screenshot shows the SQL Server Enterprise Manager interface. The top pane displays a SQL query that joins several tables: users, mentors, set_skill, skills, meetings, and invoices. The query filters for users with a rating of 4.0 or higher, located in the United States, who have completed meetings and whose invoices are paid. The results are grouped by user information and ordered by revenue. The bottom pane shows the results of the query, which is a single row for user ID 1, John Doe, with a rating of 5.00, 2 recent completed meetings, and a revenue of 117.00.

user_id	mentor_name	rating	recent_completed	revenue_365d
1	John Doe	5.00	2	117.00

Hình 25: Kết quả thực thi Kịch bản 2 với SQL Server

Trên SQL Server, truy vấn nghiệp vụ được biểu diễn trọn vẹn trong một câu lệnh SQL duy nhất. Query optimizer phân tích toàn bộ điều kiện lọc, lựa chọn thứ tự JOIN

tối ưu, xây dựng execution plan và thực thi trong một pipeline thống nhất. Tính đúng đắn của kết quả, bao gồm việc loại bỏ các bản ghi không thỏa mãn điều kiện giao được đảm bảo hoàn toàn bởi database engine.

Trên Cassandra, cùng truy vấn nghiệp vụ đó nhưng không thể biểu diễn trong một câu lệnh CQL. Do thiếu cơ chế JOIN và GROUP BY cross-partition, truy vấn buộc phải được phân rã thành nhiều bước độc lập, mỗi bước tương ứng với một access pattern đã được thiết kế sẵn trong schema. Tính đúng đắn của kết quả tổng hợp không còn là trách nhiệm của database nữa mà là tầng ứng dụng phải đảm nhận việc intersection các tập kết quả, lọc điều kiện trên những cột không thuộc primary key, và loại bỏ các bản ghi thừa trước khi trả về client.

Điều này phản ánh triết lý thiết kế đối lập giữa hai hệ thống: SQL Server tối ưu hóa tại thời điểm thực thi thông qua query optimizer; Cassandra chuyển gánh nặng tối ưu hóa về thời điểm thiết kế schema, đổi lấy khả năng mở rộng ngang tuyến tính. Hệ quả thực tiễn là Cassandra phù hợp với các hệ thống có access pattern xác định và khối lượng ghi lớn, trong khi SQL Server phù hợp hơn với các truy vấn nghiệp vụ phức tạp, đa chiều và thay đổi theo thời gian.

4.3.4.3 Kịch bản 3: Tối ưu hóa truy vấn bằng lọc sớm dữ liệu

Mục tiêu của kịch bản này là quan sát tác động của các kỹ thuật tối ưu hóa dựa trên heuristic lên số lượng logical reads khi thay đổi mức độ chọn lọc của điều kiện lọc trong stored procedure sp_DashboardStatistics. Trong SQL Server, query optimizer có khả năng tự động đẩy các predicate xuống càng sớm càng tốt trong execution plan (predicate pushdown), nhằm giảm kích thước của các tập dữ liệu trung gian trước khi thực hiện các phép JOIN hoặc aggregate. Nhờ đó, số lượng dữ liệu mà các operator phải xử lý được cắt giảm đáng kể.

Ngược lại, Cassandra không sở hữu query optimizer theo kiểu cost-based hoặc heuristic-based như SQL Server. Hệ thống không tự động tái cấu trúc truy vấn hay quyết định chiến lược lọc dữ liệu tối ưu trong thời gian chạy. Mọi quyết định liên quan đến filtering, partitioning hay pre-aggregation đều phải được lập trình viên xác định từ trước ngay tại giai đoạn thiết kế schema và access pattern.

Chỉ số được sử dụng trong kịch bản này không phải elapsed time mà là logical reads. Đây là metric phản ánh trực tiếp số lượng data pages mà storage engine buộc phải đọc qua trong quá trình thực thi truy vấn, từ đó cho thấy kích thước thực tế của các tập trung gian mà execution engine phải xử lý.

SQL Server:

```
1 DBCC FREEPROCCACHE;
2 GO
3
4 -- Lần 1: Date range 12 tháng - filter rộng, invoices scan lớn nhất
5 DECLARE @S1 DATETIME = DATEADD(MONTH, -12, GETDATE()), @E1 DATETIME =
    GETDATE();
6 SET STATISTICS IO ON;
7 EXEC dbo.sp_DashboardStatistics
8     @GroupBy      = N'mentor',
9     @StartDate    = @S1,
10    @EndDate       = @E1
11 WITH RECOMPILE;
12 SET STATISTICS IO OFF;
13 GO
14
15 -- Lần 2: Date range 3 tháng - invoices scan nhỏ hơn, logical reads giảm rõ rệt
16 DECLARE @S2 DATETIME = DATEADD(MONTH, -3, GETDATE()), @E2 DATETIME =
    GETDATE();
17 SET STATISTICS IO ON;
18 EXEC dbo.sp_DashboardStatistics
19     @GroupBy      = N'mentor',
20     @StartDate    = @S2,
21     @EndDate       = @E2
22 WITH RECOMPILE;
23 SET STATISTICS IO OFF;
24 GO
```

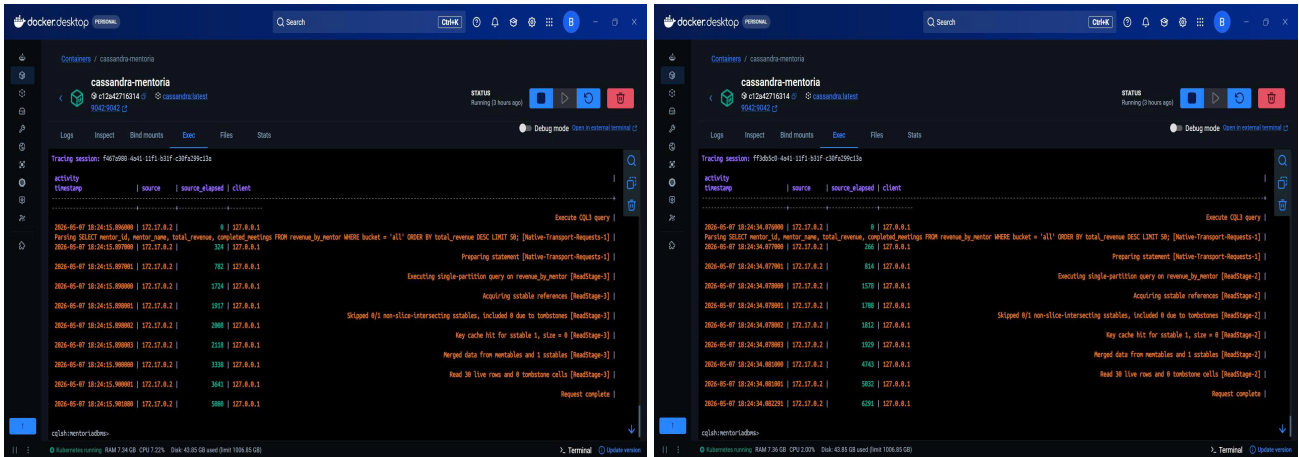
Cassandra:

```
1 -- Lần 1 - Tương đương SQL Server lần 1 (12 tháng): đọc toàn bộ
```

Kết quả:



Hình 26: Kết quả thực thi Kịch bản 3 với SQL Server



(a) Chạy lần 1

(b) Chạy lần 2

Hình 27: Kết quả thực thi Kịch bản 3 với Cassandra

Kết quả thực nghiệm cho thấy SQL Server có khả năng tự động điều chỉnh execution plan dựa trên độ chọn lọc của điều kiện thời gian tại thời điểm thực thi. Khi phạm vi thời gian được thu hẹp, query optimizer có thể đẩy predicate xuống sớm hơn trong cây thực thi, từ đó làm giảm kích thước các tập trung gian và kéo theo số lượng logical reads giảm tương ứng. Điều này phản ánh khả năng tối ưu hóa động của SQL Server dựa trên thống kê dữ liệu và cost estimation tại runtime.

Ngược lại, Cassandra không sở hữu query optimizer theo mô hình cost-based hay heuristic-based như SQL Server. Hai truy vấn với phạm vi thời gian khác nhau về bản chất vẫn đọc cùng một lượng dữ liệu nếu schema và partition strategy không thay đổi. Điều kiện lọc theo thời gian trong nhiều trường hợp không giúp giảm lượng dữ liệu phải quét qua tại runtime, bởi Cassandra không tự động tái cấu trúc execution strategy dựa trên predicate.

Để đạt được hiệu quả tương đương với cơ chế predicate pushdown của SQL Server, Cassandra yêu cầu việc tối ưu phải được thực hiện ngay từ giai đoạn thiết kế dữ liệu. Lập trình viên cần chủ động "nhúng" chiến lược lọc vào schema thông qua partition key, clustering column hoặc pre-aggregated table. Ví dụ, dữ liệu doanh thu có thể được tổ chức theo từng tháng để các truy vấn theo khoảng thời gian chỉ cần quét qua một phần dữ liệu đã được phân vùng sẵn. Đây có thể xem như một dạng "predicate pushdown thủ công", nơi quyết định tối ưu hóa được thực hiện tại design time thay vì được query optimizer tự động xử lý tại runtime.

4.3.5 Đánh giá và quyết định

Dựa trên các phân tích về pipeline xử lý truy vấn, cơ chế tối ưu hóa và các kịch bản thực nghiệm ở trên, MS SQL Server cho thấy mức độ phù hợp vượt trội hơn đáng kể so với Apache Cassandra đối với bài toán xử lý dữ liệu của nền tảng Mentoria.

Trước hết, đặc trưng truy vấn của hệ thống Mentoria mang bản chất quan hệ và đa chiều rất rõ rệt. Các chức năng cốt lõi như tìm kiếm Mentor theo nhiều tiêu chí kết hợp, thống kê dashboard, lọc dữ liệu theo thời gian, tính toán doanh thu hay phân tích lịch sử phiên mentoring đều yêu cầu khả năng xử lý ad-hoc query linh hoạt. Đây chính là nhóm workload mà SQL Server được tối ưu mạnh nhất thông qua cost-based optimizer, execution plan generation, statistics estimation và hệ thống indexing phong phú. Trong các kịch bản thực nghiệm, SQL Server liên tục cho thấy khả năng tự động tối ưu execution plan tại runtime thông qua các kỹ thuật như predicate pushdown, index seek selection, join reordering hay plan caching. Những cơ chế này giúp hệ thống giảm đáng kể logical reads, kích thước tập trung gian và chi phí thực thi mà không yêu cầu lập trình viên can thiệp thủ công.

Ngược lại, Cassandra tiếp cận xử lý truy vấn theo triết lý hoàn toàn khác. Hệ thống không cung cấp query optimizer theo nghĩa truyền thống, không xây dựng execution plan phức tạp và cũng không hỗ trợ tốt cho các truy vấn phát sinh động ngoài access pattern đã được định nghĩa trước. Trong Cassandra, hiệu năng truy vấn phụ thuộc trực tiếp vào cách thiết kế partition key, clustering column và chiến lược phân vùng dữ liệu ngay từ thời điểm xây dựng schema. Điều này đồng nghĩa với việc các tối ưu như filtering, aggregation hay range scan không được quyết định tự động tại runtime mà phải được "đóng cứng" vào data model từ trước. Đối với các workload có tính linh hoạt cao như Mentoria, cách tiếp cận này dẫn đến hiện tượng nhân bản dữ liệu trên nhiều bảng chuyên biệt, làm tăng độ phức tạp trong thiết kế, đồng bộ dữ liệu và bảo trì hệ thống.

Ngoài ra, các truy vấn nghiệp vụ của Mentoria thường xuyên yêu cầu JOIN, GROUP BY, ORDER BY và aggregate trên nhiều chiều dữ liệu khác nhau. Đây là những thao tác được SQL Server hỗ trợ hiệu quả thông qua relational algebra engine

và các chiến lược tối ưu hóa chi phí. Trong khi đó, Cassandra chỉ phát huy hiệu quả tối đa khi truy vấn bám sát partition key và access pattern cố định. Những truy vấn lọc đa chiều hoặc cross-partition thường dẫn đến scatter-gather, full cluster scan hoặc yêu cầu ALLOW FILTERING - vốn đi ngược lại triết lý thiết kế phân tán của Cassandra và gây suy giảm hiệu năng nghiêm trọng khi dữ liệu tăng trưởng lớn.

Từ góc độ kiến trúc tổng thể, Cassandra vẫn là lựa chọn rất mạnh cho các hệ thống write-heavy quy mô cực lớn như telemetry, event logging, IoT hay messaging feed - nơi access pattern gần như cố định và throughput là ưu tiên hàng đầu. Tuy nhiên, đối với một nền tảng hướng nghiệp và kết nối Mentor như Mentoria, nhu cầu trọng tâm lại nằm ở khả năng truy vấn linh hoạt, tối ưu hóa động, đảm bảo tính nhất quán giao dịch và hỗ trợ phân tích dữ liệu đa chiều. Đây chính là thế mạnh cốt lõi của SQL Server.

Vì vậy, xét trên cả phương diện query processing, khả năng tối ưu hóa runtime, độ linh hoạt truy vấn và tính phù hợp với đặc thù nghiệp vụ, MS SQL Server là lựa chọn phù hợp và hiệu quả hơn Apache Cassandra cho hệ thống Mentoria.

4.4 Data Backup and Recovery

4.4.1 Khái niệm chung về data backup và recovery

Trong các hệ quản trị cơ sở dữ liệu (DBMS), chiến lược sao lưu và phục hồi không chỉ đơn thuần là việc sao chép dữ liệu, mà là một quy trình đảm bảo tính bền vững (Durability) theo tiêu chuẩn ACID. Đây là lớp phòng thủ cuối cùng chống lại sự cố mất mát dữ liệu do lỗi phần cứng, phần mềm hoặc thao tác sai sót từ con người. Nhóm tập trung đánh giá hiệu quả qua hai chỉ số cốt lõi:

- **Recovery Point Objective (RPO):** Xác định lượng dữ liệu tối đa mà hệ thống chấp nhận bị mất, tính theo đơn vị thời gian. Với chiến lược sao lưu định kỳ hàng ngày, RPO có thể lên đến 24 giờ.
- **Recovery Time Objective (RTO):** Xác định khoảng thời gian tối đa để đưa hệ thống hoạt động trở lại sau khi xảy ra sự cố. RTO phụ thuộc trực tiếp vào dung lượng dữ liệu sao lưu và hiệu suất của hạ tầng phục hồi.

4.4.2 Đặc trưng cơ chế backup và recovery trong MS SQL Server

MS SQL Server đại diện cho mô hình quản trị dữ liệu tập trung, đảm bảo an toàn thông qua nhật ký giao dịch nghiêm ngặt và kiến trúc tệp tin vật lý đặc thù (.mdf và .ldf).

Full recovery model và quản lý log chain:

- **Cơ chế WAL:** Mọi giao dịch phát sinh đều được ghi lại vào Transaction Log thông qua cơ chế Write-Ahead Logging (WAL) trước khi thay đổi được áp dụng chính thức. Điều này cho phép thực hiện phục hồi dữ liệu đến từng thời điểm cụ thể (Point-in-time Recovery).
- **Duy trì Log Chain:** Trong chế độ Full Recovery, Transaction Log sẽ phình to liên tục nếu không được sao lưu. Nhóm thực hiện duy trì chuỗi Log (*Log Chain*) liên tục; việc sao lưu Log thường xuyên không chỉ giúp tối ưu RPO mà còn giúp "cắt tỉa" (*Truncate*) tệp nhật ký, giải phóng không gian đĩa cứng.
- **Tail-Log Backup:** Đây là kỹ thuật cứu cánh cuối cùng được nhóm nghiên cứu. Trong trường hợp tệp dữ liệu (.mdf) bị hỏng nhưng tệp Log vẫn khả dụng, việc sao lưu phần "đuôi" nhật ký (*Tail*) cho phép khôi phục dữ liệu đến sát thời điểm xảy ra sự cố, đưa RPO về mức gần bằng 0.

Cơ chế sao lưu vật lý và tối ưu hóa:

- **Full Backup:** Sử dụng bản sao lưu toàn phần tạo ra tệp .bak. Quá trình này đảm bảo tính nhất quán dữ liệu thông qua các điểm kiểm tra (*checkpoint*) để đồng bộ dữ liệu giữa bộ nhớ đệm và đĩa vật lý.
- **Backup Compression:** Để tối ưu RTO, nhóm áp dụng thuật toán nén bản sao lưu. Việc nạp một tệp .bak đã nén giúp giảm thiểu lưu lượng I/O, từ đó rút ngắn đáng kể thời gian Restore so với việc sử dụng các tệp sao lưu thô nguyên bản.

4.4.3 Đặc trưng cơ chế backup và recovery trong Apache Cassandra

Trái ngược với SQL Server, Cassandra ưu tiên tính sẵn sàng cao thông qua kiến trúc không máy chủ gốc (Masterless) và cơ chế lưu trữ phân tán.

Tính bền vững qua CommitLog và Memtable:

- **Write Path Durability:** Khi một thao tác ghi được thực thi, Cassandra đồng thời ghi dữ liệu vào *CommitLog* (trên đĩa) và *Memtable* (trên RAM). CommitLog đóng vai trò là cơ chế phục hồi thảm họa tức thì; nếu nút bị sập đột ngột, hệ thống sẽ đọc lại CommitLog để tái tạo lại dữ liệu trong Memtable khi khởi động lại.
- **SSTables và Snapshots:** Vì dữ liệu trong SSTables là bất biến (*Immutable*), việc sao lưu qua lệnh `nodetool snapshot` chỉ đơn giản là tạo các liên kết cứng (*hard-links*), giúp quá trình sao lưu diễn ra tức thời (*Zero-overhead*) mà không ảnh hưởng đến hiệu năng truy vấn hiện hành.

Nhân bản và tự chữa lành (Self-healing):

- **Anti-Entropy Repair:** Cassandra sử dụng cấu trúc cây băm *Merkle Trees* để so sánh dữ liệu giữa các nút mạng mà không cần gửi toàn bộ dữ liệu qua băng thông. Cơ chế này giúp phát hiện và đồng bộ nhanh chóng các bản sao bị sai lệch (*Data inconsistency*).
- **Hinted Handoff:** Khi một nút không khả dụng, các nút khác sẽ lưu giữ các "gợi ý" (*hints*) về dữ liệu cần ghi. Ngay khi nút đó hoạt động trở lại, các dữ liệu này sẽ được đẩy về để đảm bảo tính nhất quán cuối cùng (*Eventual Consistency*).
- **Incremental Backups:** Ngoài Snapshot, Cassandra hỗ trợ sao lưu lũy tiến. Mỗi khi một SSTable mới được đẩy xuống đĩa (*Flushed*), một bản sao lưu sẽ được tạo ra, giúp tiết kiệm dung lượng lưu trữ so với việc chỉ Snapshot toàn bộ database.

4.4.4 Kịch bản so sánh thực tế giữa SQL Server và Cassandra

Để đánh giá trực diện hiệu năng của hai hệ thống, nhóm thực hiện các kịch bản mô phỏng thảm họa và đối chiếu kết quả xử lý.

4.4.4.1 Kịch bản 1: Phục hồi sau thảm họa hệ thống

MS SQL Server: Nhóm thực hiện tạo bản sao lưu vật lý Database Mentorina ra tệp `mentorina.bak` (dung lượng thực tế ghi nhận khoảng 7.6 MB) và tiến hành phục hồi sau khi giả lập xóa hoàn toàn cơ sở dữ liệu. Kết quả cho thấy dữ liệu được khôi phục toàn vẹn 100% nhưng hệ thống ghi nhận trạng thái gián đoạn hoàn toàn (Downtime 100%) trong suốt quá trình Restore.

Apache Cassandra: Nhóm thiết lập cụm Cluster gồm 02 nút (Seed Node và Secondary Node) thông qua Docker với giới hạn bộ nhớ từ 512M đến 1G và thực hiện sập đột ngột nút chính.

So sánh: SQL Server yêu cầu thời gian dừng hệ thống để nạp lại dữ liệu (RTO phụ thuộc tốc độ I/O), trong khi Cassandra đạt trạng thái **Zero Downtime** do dữ liệu vẫn khả dụng trên nút phụ còn lại nhờ hệ số nhân bản $RF = 2$.

4.4.4.2 Kịch bản 2: Tính linh hoạt trong xử lý sự cố dữ liệu cục bộ

MS SQL Server: Giả lập tình huống xóa nhầm dữ liệu trên các bảng trung tâm sau khi đã tắt các ràng buộc khóa ngoại. SQL Server thể hiện ưu thế nhờ Transaction Log, cho phép phục hồi chính xác trạng thái dữ liệu về ngay trước thời điểm xảy ra lệnh xóa.

Apache Cassandra: Thực nghiệm cho thấy khi một nút quay trở lại hoạt động sau khi sập, cơ chế Hinted Handoff tự động đẩy các phần dữ liệu bị thiếu sang nút đó mà không cần can thiệp quản trị thủ công.

So sánh: SQL Server mạnh về khả năng phục hồi dữ liệu chính xác theo thời điểm (Point-in-time), trong khi Cassandra mạnh về khả năng duy trì tính sẵn sàng liên tục thông qua cơ chế tự phục hồi nhất quán.

4.4.5 Đánh giá và quyết định

Phân tích sự đánh đổi:

- **Tài nguyên tiêu thụ:** Qua thực nghiệm, cụm Cassandra tiêu tốn lượng RAM (JVM Heap) gấp khoảng 3 lần so với một instance SQL Server có cùng

quy mô dữ liệu để quản lý bộ nhớ đệm và duy trì cụm nút.

- **Độ phức tạp vận hành:** SQL Server đơn giản và dễ quản trị tập trung. Cassandra đòi hỏi kỹ thuật chuyên môn cao cho các tiến trình định kỳ như dọn dẹp dữ liệu cũ (Compaction) và duy trì nhất quán cuối cùng.

So sánh tổng kết:

Tiêu chí	MS SQL Server	Apache Cassandra
Mô hình	Tập trung (Centralized)	Phân tán (Distributed)
Khả năng phục hồi	Khôi phục về thời điểm (Point-in-time)	Tự chữa lành thông qua nhân bản
Tính sẵn sàng	Gián đoạn (Downtime) khi Restore	Rất cao (Không Downtime khi sập nút)
Hạ tầng phù hợp	Monolithic Architecture	Microservices Architecture

Bảng 2: So sánh cơ chế Backup và Recovery giữa hai DBMS

Dựa trên kết quả thực nghiệm và mục tiêu dự án Mentoria, nhóm quyết định lựa chọn **MS SQL Server** vì:

- **Sự tương thích kiến trúc:** Phù hợp tuyệt đối với mô hình **Monolithic** hiện tại của dự án, giúp tối ưu chi phí hạ tầng và giảm độ phức tạp trong triển khai.
- **An toàn dữ liệu:** Với quy mô hiện tại, cơ chế sao lưu tệp vật lý **.bak** và Transaction Log đáp ứng tốt các chỉ số RPO/RTO mà không cần đầu tư hạ tầng phân tán tốn kém.

Apache Cassandra được nhìn nhận là giải pháp mở rộng tiềm năng khi hệ thống chuyển đổi sang kiến trúc **Microservices** trong tương lai.

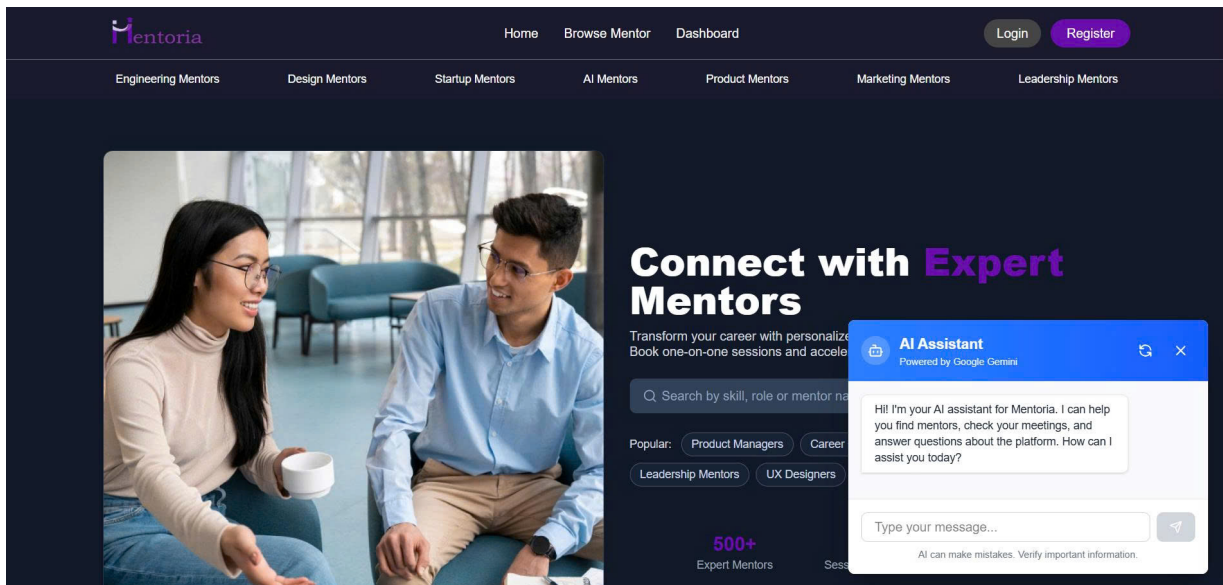
5 Hiện thực hệ thống

Mã nguồn của dự án được lưu trữ tại: <https://github.com/BaoDuong254/mentoria-dbms>.

5.1 Giao diện dành cho Guest

Guest là nhóm người dùng chưa đăng nhập vào hệ thống. Theo yêu cầu chức năng đã trình bày ở mục 3.2.1, Guest được phép truy cập một phần nội dung công khai của nền tảng nhằm khám phá dịch vụ trước khi quyết định đăng ký tài khoản. Cụ thể, Guest có thể xem trang chủ giới thiệu, duyệt danh sách mentor, xem hồ sơ chi tiết của từng mentor, tương tác với trợ lý AI, cũng như đăng ký tài khoản với vai trò Mentee hoặc Mentor. Các thao tác liên quan đến đặt lịch, thanh toán hay đánh giá đều bị hạn chế và yêu cầu xác thực đăng nhập.

5.1.1 Trang chủ (Landing Page)



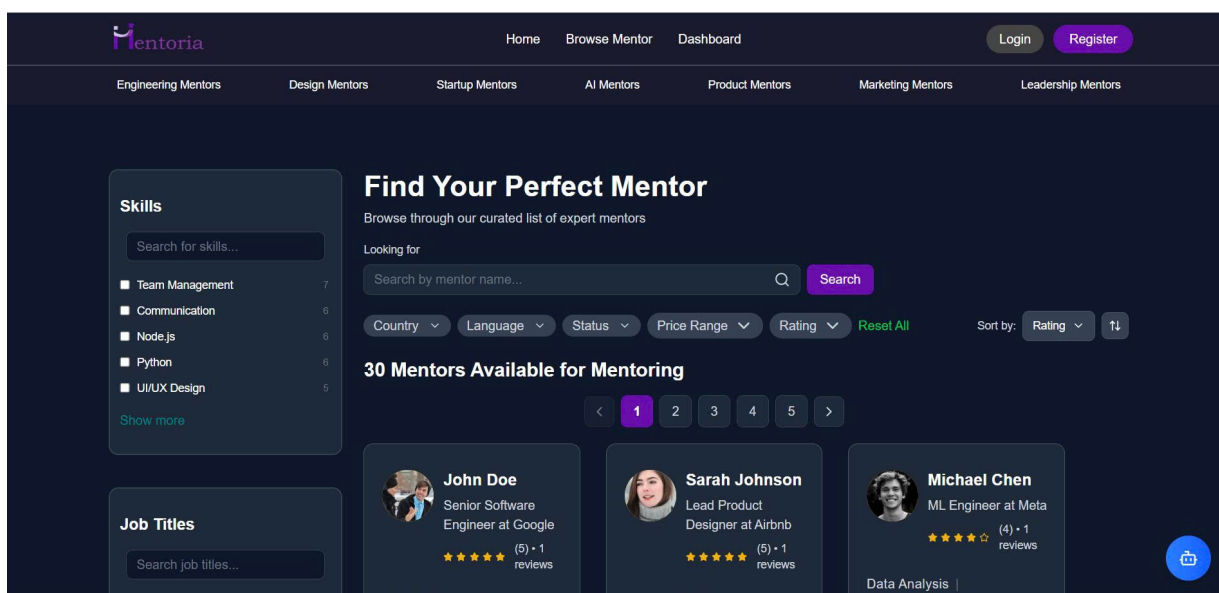
Hình 28: Giao diện trang chủ với trợ lý AI tích hợp

Trang chủ là điểm tiếp cận đầu tiên của Guest với hệ thống Mentoria. Giao diện được thiết kế với thông điệp chính “*Connect with Expert Mentors*”, kèm theo thanh tìm kiếm nhanh theo kỹ năng, vai trò hoặc tên mentor và các nhóm chuyên môn phổ biến (Product Managers, Career Coaches, Software Engineers, Leadership Mentors, UX

Designers). Phần thanh điều hướng trên cùng cung cấp các lối tắt đến *Home*, *Browse Mentor*, *Dashboard*, cùng các nút *Login* và *Register* ở góc phải.

Một thành phần đáng chú ý là **trợ lý AI Assistant** (powered by Google Gemini) được tích hợp dưới dạng widget chat ở góc dưới phải màn hình. Trợ lý này hỗ trợ Guest tìm kiếm mentor phù hợp, trả lời các câu hỏi tổng quan về nền tảng và hướng dẫn các luồng nghiệp vụ cơ bản, qua đó giảm rào cản tiếp cận cho người dùng mới.

5.1.2 Trang duyệt Mentor (Browse Mentor)



Hình 29: Giao diện duyệt và lọc danh sách mentor

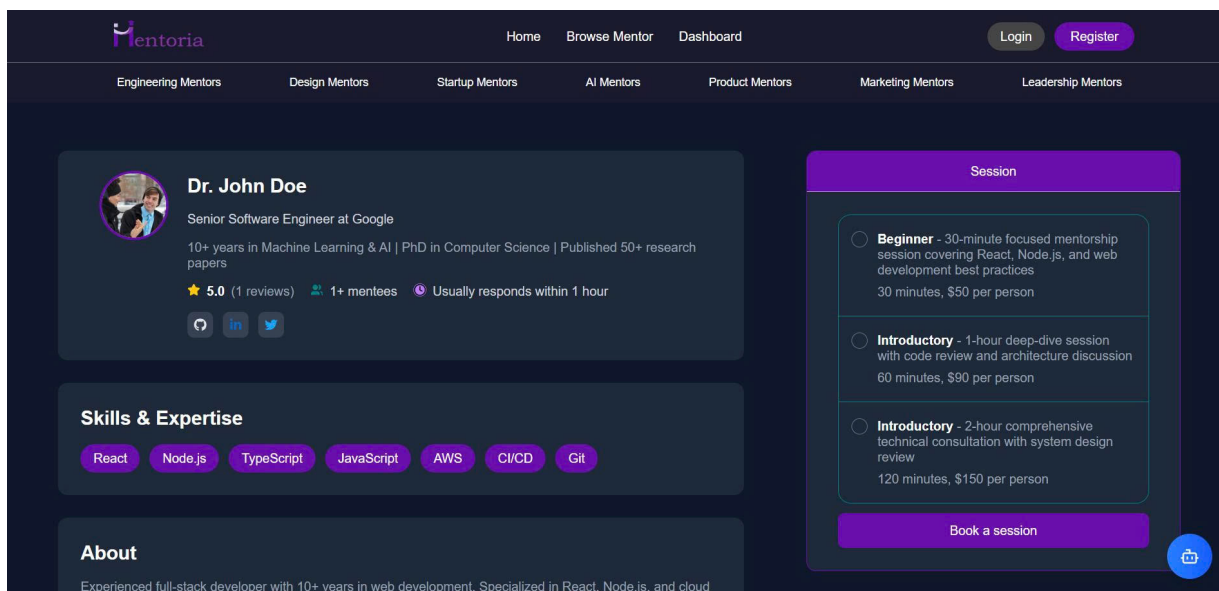
Trang *Browse Mentor* cho phép Guest duyệt danh sách mentor được lưu trữ trong cơ sở dữ liệu mà không cần đăng ký. Giao diện được tổ chức theo hai khu vực chính:

- **Bảng lọc bên trái:** Hỗ trợ lọc theo kỹ năng (Skills) như Team Management, Communication, Node.js, Python, UI/UX Design, kèm số lượng mentor tương ứng cho mỗi mục (dựa trên truy vấn COUNT từ CSDL). Bên dưới còn có bộ lọc theo Job Titles và Companies.
- **Khu vực chính:** Hiển thị thanh tìm kiếm theo tên mentor, kèm các bộ lọc nâng cao (*Country*, *Language*, *Status*, *Price Range*, *Rating*) và nút *Reset All*. Các mentor được hiển thị dạng card chứa ảnh đại diện, họ tên, chức danh,

công ty đang làm việc, số sao đánh giá và số lượt review. Hệ thống hỗ trợ phân trang và sắp xếp theo *Rating*.

Toàn bộ chức năng tìm kiếm và lọc đa chiều này được thực thi thông qua stored procedure `dbo.sp_SearchMentors`. Thủ tục này đã bao hàm đầy đủ ba loại query phức tạp nhất theo yêu cầu của đề bài: **Query có điều kiện ghép** (mệnh đề `WHERE` kết hợp hàng loạt điều kiện `AND/OR` trên nhiều cột như `role`, `first_name`, `last_name`, `country`, `status` cùng các điều kiện về rating và price range), **Query có JOIN** (tổ hợp `INNER JOIN` kết hợp `LEFT JOIN` trên các bảng `users`, `mentors`, `set_skill`, `skills`, `categories`, `work_for`, `companies`, `job_title`, `plans`) và **Query có Subquery** (correlated subquery dạng `(SELECT MIN(plan_charge) ...)` để tính `lowest_plan_price`, cùng các subquery dạng `EXISTS` để kiểm tra điều kiện lọc theo company, job title, skill, category, language).

5.1.3 Trang hồ sơ Mentor



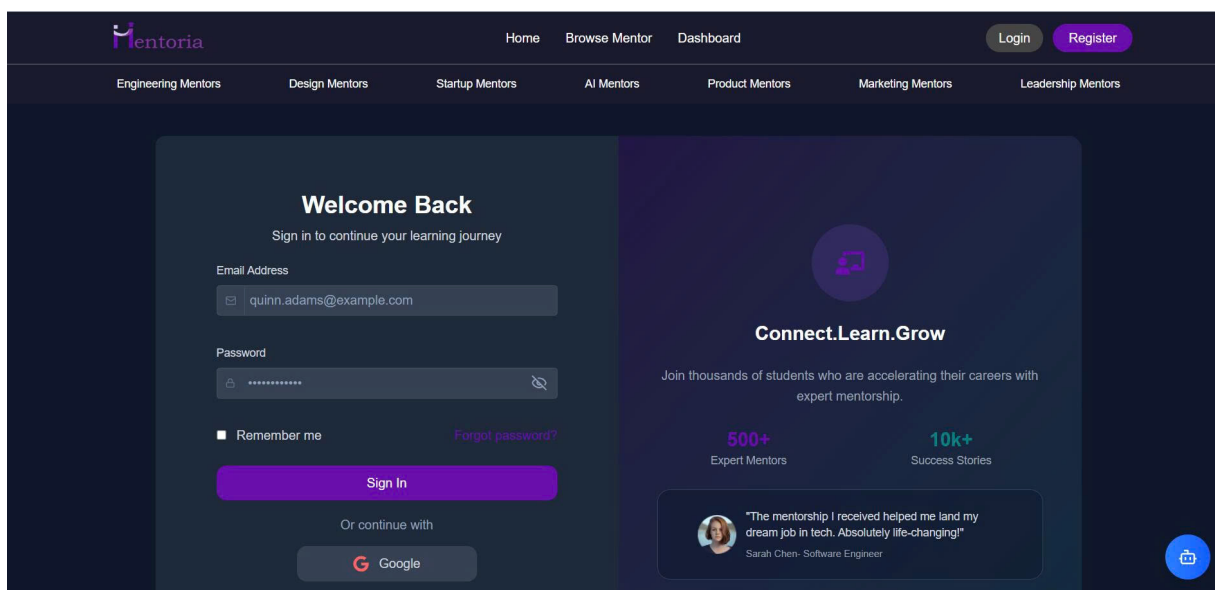
Hình 30: Giao diện trang hồ sơ chi tiết của Mentor

Khi Guest chọn một mentor cụ thể từ trang *Browse Mentor*, hệ thống điều hướng đến trang hồ sơ chi tiết. Giao diện trình bày đầy đủ thông tin chuyên môn của mentor, bao gồm: tên (Dr. John Doe), chức danh và công ty (*Senior Software Engineer at Google*), tiểu sử ngắn (kinh nghiệm 10+ năm trong Machine Learning & AI, học vị

PhD, các công trình đã công bố), điểm đánh giá trung bình kèm số lượt review, số mentee đã hỗ trợ, thời gian phản hồi trung bình và các liên kết mạng xã hội (GitHub, LinkedIn, Twitter).

Phần *Skills & Expertise* liệt kê các kỹ năng (React, Node.js, TypeScript, JavaScript, AWS, CI/CD, Git), được lấy từ bảng quan hệ N-N giữa Mentor và Skill trong CSDL. Bên phải là khu vực *Session* hiển thị các gói mentoring mà mentor cung cấp, với ba mức độ điển hình: Beginner (30 phút, \$50), Introductory (60 phút, \$90) và Introductory mở rộng (120 phút, \$150). Tuy nhiên, nút **Book a session** chỉ kích hoạt khi người dùng đã đăng nhập; với Guest, hệ thống sẽ điều hướng sang trang đăng nhập khi nhấn vào.

5.1.4 Trang đăng nhập



Hình 31: Giao diện trang đăng nhập

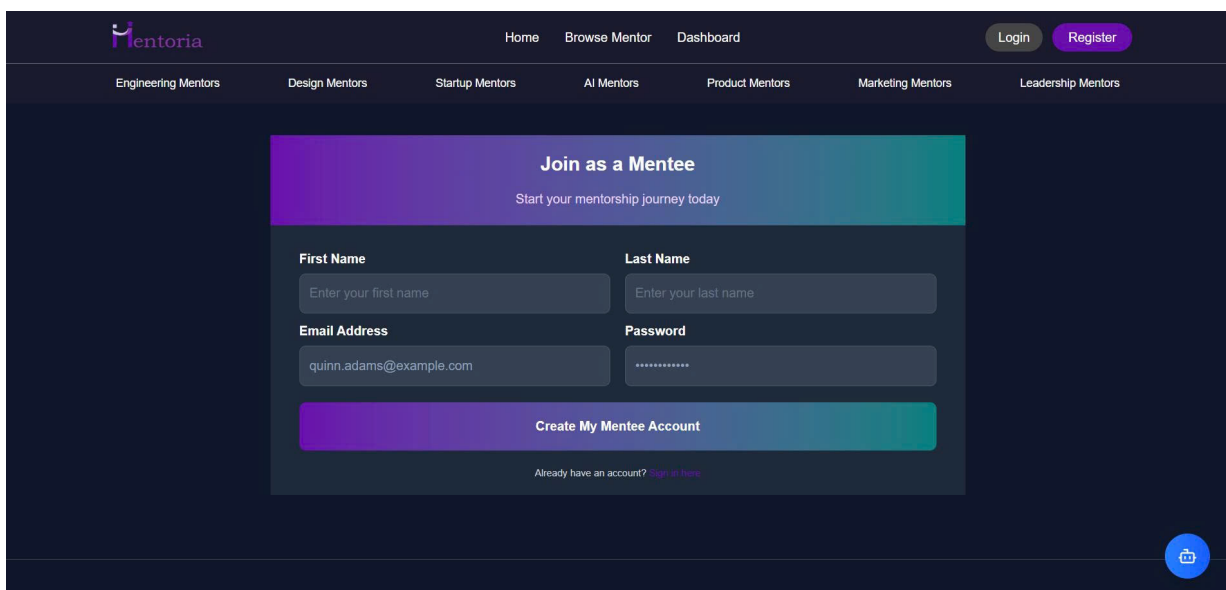
Trang đăng nhập được thiết kế tối giản với thông điệp “Welcome Back”, gồm các trường *Email Address* và *Password*, kèm tùy chọn *Remember me* và liên kết *Forgot password?* cho phép khôi phục mật khẩu qua email (các tính năng này nhóm đang phát triển).

Bên phải giao diện là khu vực giới thiệu nền tảng với các con số minh chứng độ uy tín (500+ Expert Mentors, 10k+ Success Stories) và testimonial từ người dùng thực tế, giúp tăng độ tin cậy đối với người dùng mới.

Mặc dù trang đăng nhập trông đơn giản về mặt giao diện, luồng xử lý phía backend lại tương đối phong phú và là minh chứng tốt cho việc hệ thống đáp ứng đầy đủ các loại query-update theo yêu cầu của đề bài. Khi Mentee/Mentor nhập email và mật khẩu rồi nhấn *Sign In*, frontend gọi API POST `/api/auth/login` đến backend, kích hoạt một chuỗi thao tác CSDL trong file `auth.service.ts`:

- **Query với điều kiện đơn:** Truy vấn `SELECT 1 FROM users WHERE email = @email` để xác định người dùng có tồn tại trong hệ thống hay không.
- **UPDATE:** Sau khi xác thực mật khẩu (so sánh hash) thành công, hệ thống thực thi câu lệnh `UPDATE users SET otp = ..., otp_expires = ... WHERE user_id = @userId (auth.service.ts:199)` để sinh mã OTP xác thực hai bước, đồng thời cập nhật thời gian đăng nhập gần nhất.
- **INSERT (luồng đăng ký liên quan):** Trang đăng nhập có liên kết *“Don’t have an account?”* điều hướng tới luồng đăng ký, trong đó `auth.service.ts:68` thực thi `INSERT INTO users` để tạo bản ghi người dùng mới với trạng thái `pending` chờ xác minh email.

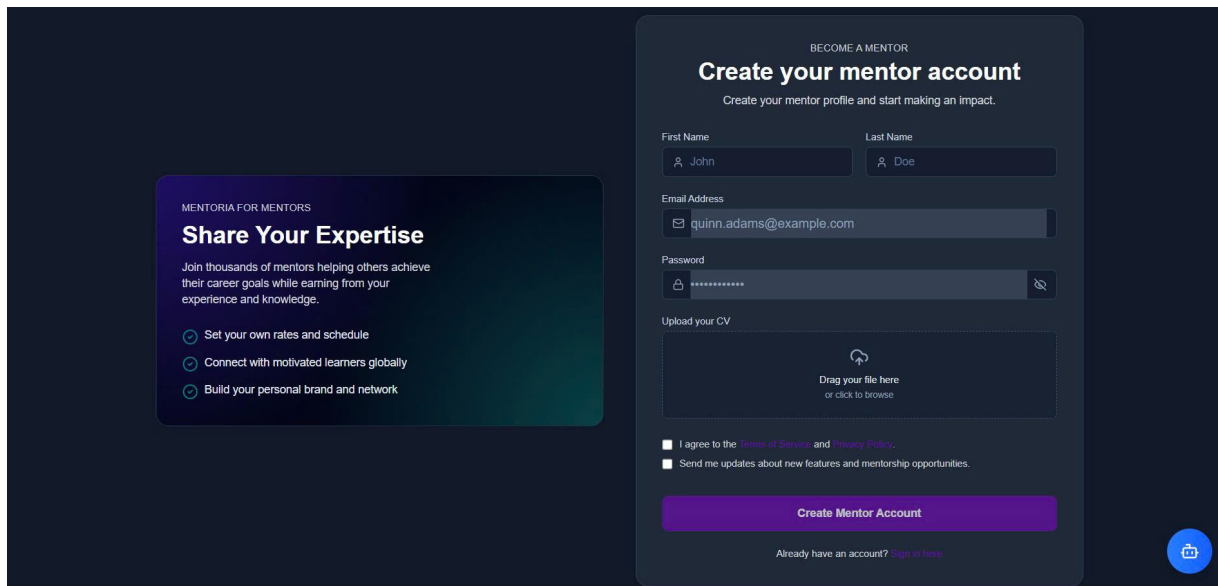
5.1.5 Trang đăng ký tài khoản Mentee



Hình 32: Giao diện đăng ký tài khoản Mentee

Đối với Guest muốn tham gia hệ thống với vai trò Mentee, giao diện đăng ký được thiết kế đơn giản nhằm giảm tối đa độ ma sát (friction) trong bước onboarding. Form chỉ yêu cầu các trường tối thiểu: *First Name*, *Last Name*, *Email Address* và *Password*. Sau khi nhấn *Create My Mentee Account*, hệ thống sẽ tạo bản ghi trong bảng **users** với **role** = 'Mentee' và **status** = 'pending', đồng thời gửi mã OTP đến email người dùng để xác thực, tuân thủ ràng buộc ngữ nghĩa “*Tài khoản mới tạo phải được xác minh qua email trước khi kích hoạt*” đã trình bày ở mục 3.4.3. Tài khoản chỉ chuyển sang trạng thái **active** sau khi xác thực thành công.

5.1.6 Trang đăng ký tài khoản Mentor



Hình 33: Giao diện đăng ký tài khoản Mentor

So với Mentee, quy trình đăng ký Mentor có yêu cầu thẩm định cao hơn do liên quan trực tiếp đến chất lượng dịch vụ. Ngoài các trường thông tin cơ bản (*First Name*, *Last Name*, *Email Address*, *Password*), Guest bắt buộc phải tải lên CV thông qua khu vực *Upload your CV* (drag-and-drop hoặc duyệt file). Người dùng cũng phải đồng ý với *Terms of Service* và *Privacy Policy* trước khi nhấn *Create Mentor Account*.

Sau khi nộp hồ sơ, tài khoản Mentor được tạo với trạng thái **pending** và xếp vào hàng chờ thẩm định. Quản trị viên (Admin) sẽ xem xét CV cùng các thông tin khai báo, sau đó chấp nhận hoặc từ chối hồ sơ – đây chính là chức năng “*chấp nhận hồ sơ*”

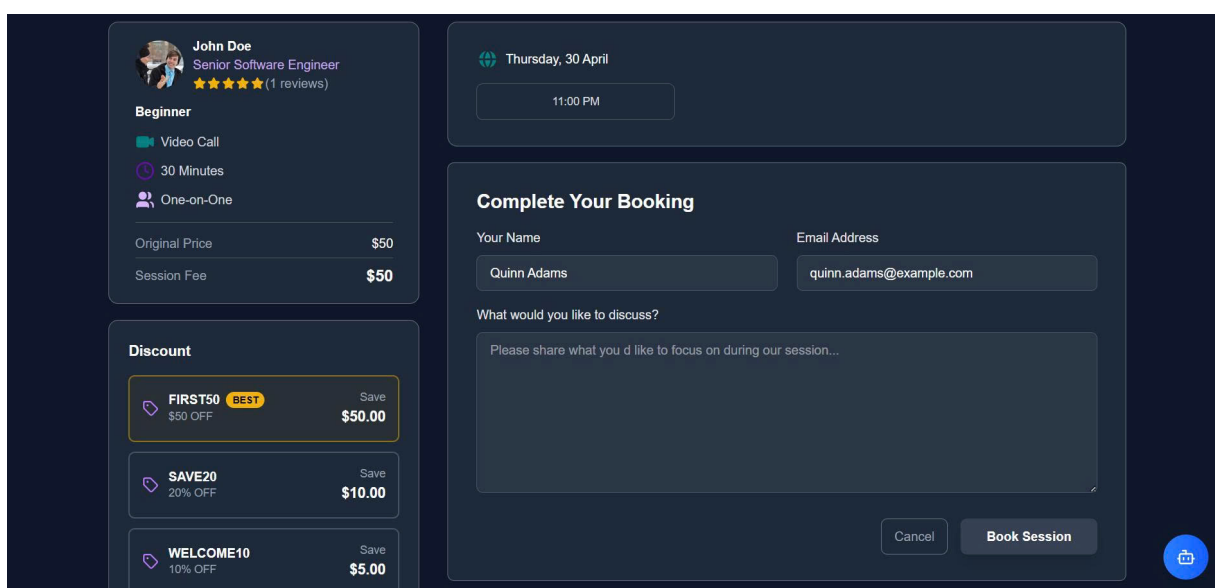
ứng tuyển trở thành mentor” đã được liệt kê trong yêu cầu chức năng của Admin (mục 3.2.1). Chỉ khi hồ sơ được phê duyệt, Mentor mới có thể đăng nhập, cập nhật thông tin chuyên môn (kỹ năng, lĩnh vực, công ty, gói mentoring, lịch làm việc) và bắt đầu nhận booking từ Mentee.

Khu vực bên trái giao diện (*Mentoria for Mentors – Share Your Expertise*) đóng vai trò marketing, nhấn mạnh ba lợi ích cốt lõi: tự thiết lập mức phí và lịch làm việc, kết nối với học viên toàn cầu, và xây dựng thương hiệu cá nhân.

5.2 Giao diện dành cho Mentee

Mentee là nhóm người dùng đã đăng ký tài khoản và xác thực email thành công với vai trò người được hướng dẫn. Sau khi đăng nhập, Mentee được mở khóa toàn bộ luồng nghiệp vụ cốt lõi của hệ thống: đặt lịch buổi tư vấn với mentor, áp dụng mã giảm giá, thanh toán qua cổng Stripe và quản lý các phiên mentoring đã đặt thông qua dashboard cá nhân. Phần này tập trung trình bày ba giao diện then chốt trong luồng đặt lịch của Mentee, theo đúng trình tự nghiệp vụ thực tế: đặt lịch → thanh toán → theo dõi.

5.2.1 Trang đặt lịch với Mentor



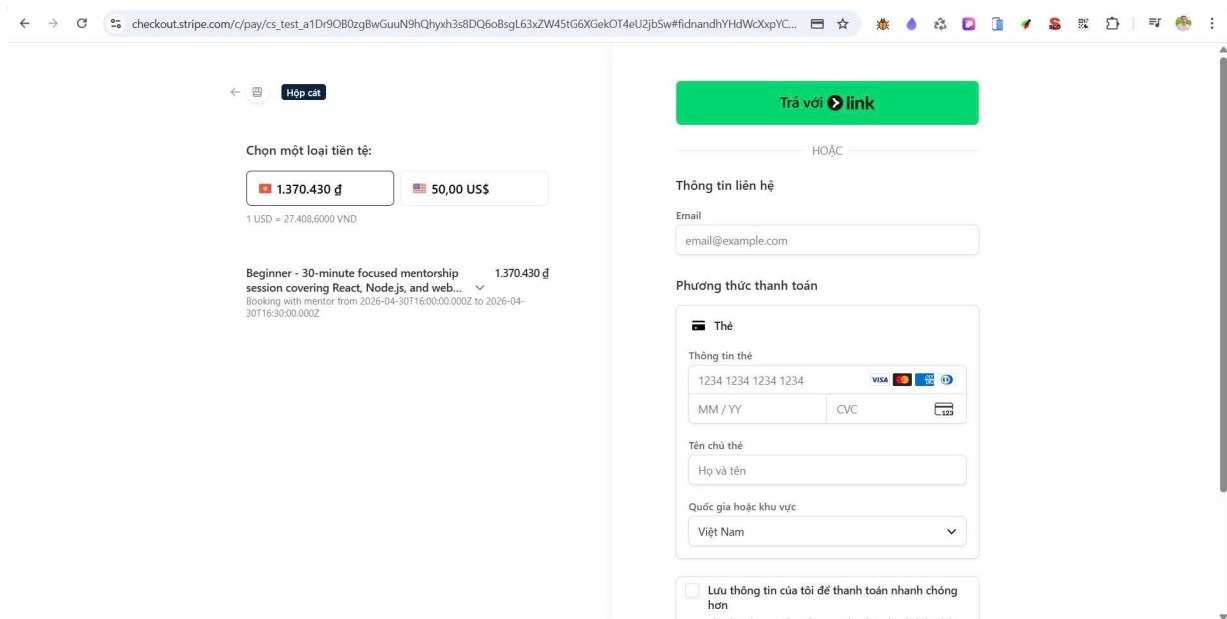
Hình 34: Giao diện đặt lịch buổi tư vấn với Mentor

Sau khi Mentee chọn một gói mentoring trên trang hồ sơ Mentor và nhấn *Book a session*, hệ thống điều hướng đến trang *Complete Your Booking*. Giao diện được tổ chức thành ba khu vực rõ ràng nhằm hỗ trợ Mentee đưa ra quyết định đặt lịch một cách minh bạch:

- **Tóm tắt thông tin Mentor và gói dịch vụ (cột trái):** Hiển thị ảnh đại diện, họ tên (*John Doe*), chức danh (*Senior Software Engineer*), điểm đánh giá trung bình, tên gói (*Beginner*) cùng các thuộc tính của gói: hình thức (Video Call), thời lượng (30 Minutes), kiểu tư vấn (One-on-One). Phần *Original Price* và *Session Fee* được tách biệt nhằm hiển thị rõ giá gốc và giá sau khi áp dụng mã giảm giá (nếu có).
- **Khu vực mã giảm giá (Discount):** Liệt kê toàn bộ các mã giảm giá hợp lệ tại thời điểm đặt lịch, được truy vấn từ bảng `discounts` với các điều kiện lọc: mã đang ở trạng thái `active`, nằm trong khoảng thời gian áp dụng (`start_date ≤ NOW() ≤ end_date`) và số lần đã sử dụng chưa vượt quá giới hạn (`used_count < max_uses`). Mỗi mã hiển thị tên mã (*FIRST50*, *SAVE20*, *WELCOME10*), loại giảm (theo phần trăm hoặc số tiền cố định) và số tiền tiết kiệm tương ứng. Hệ thống tự động đánh dấu *BEST* cho mã có giá trị giảm cao nhất nhằm hỗ trợ Mentee chọn nhanh.
- **Khu vực hoàn tất đặt lịch (cột phải):** Hiển thị ngày và khung giờ đã chọn (*Thursday, 30 April – 11:00 PM*), kèm form nhập tên, email và lời nhắn cá nhân (*What would you like to discuss?*), giúp Mentor nắm được mục tiêu của buổi tư vấn trước khi diễn ra.

Khi Mentee nhấn *Book Session*, hệ thống thực hiện một loạt thao tác nguyên tử trong phạm vi một transaction trên SQL Server: tạo bản ghi đơn đăng ký, liên kết với mã giảm giá đã chọn, tăng `used_count` của mã giảm giá, khóa phiên làm việc tương ứng (chuyển trạng thái slot từ *open* sang *pending*) và sinh hóa đơn ở trạng thái *unpaid*. Việc bọc các thao tác này trong transaction là bắt buộc để tránh tình trạng race condition khi hai Mentee cùng đặt một khung giờ – đây là một trong các điểm yếu cố hữu của Cassandra do thiếu hỗ trợ ACID đầy đủ.

5.2.2 Trang thanh toán qua Stripe



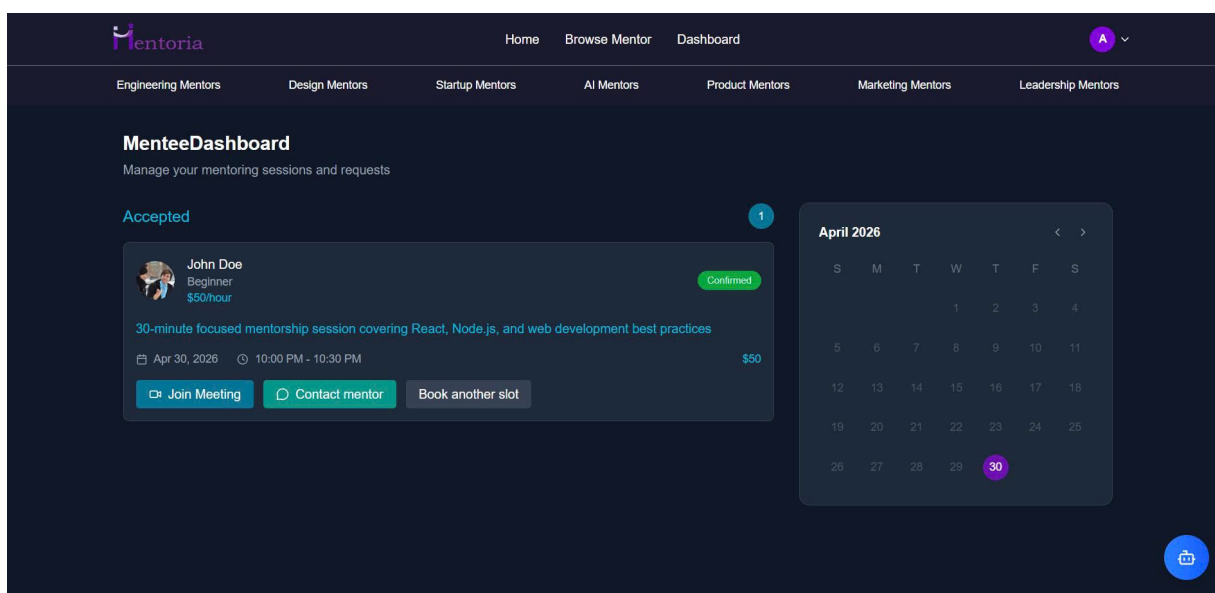
Hình 35: Giao diện thanh toán tích hợp với Stripe Checkout

Ngay sau khi tạo hóa đơn thành công, hệ thống điều hướng Mentee đến trang thanh toán của Stripe thông qua cơ chế *Stripe Checkout Session*. Đây là quyết định kiến trúc quan trọng: thay vì tự xây dựng module thanh toán, hệ thống ủy quyền hoàn toàn việc xử lý dữ liệu thẻ cho Stripe – một bên thứ ba đã đạt chứng chỉ PCI DSS Level 1. Cách tiếp cận này mang lại ba lợi ích đáng kể:

- **Giảm phạm vi tuân thủ PCI:** Hệ thống không bao giờ tiếp xúc trực tiếp với số thẻ tín dụng, do đó không phải triển khai và duy trì các yêu cầu kỹ thuật phức tạp của PCI DSS.
- **Hỗ trợ đa tiền tệ và đa phương thức thanh toán:** Stripe tự động chuyển đổi tỷ giá theo thời gian thực (trong ví dụ ở Hình 35, \$50 USD tương đương 1.370.430 VND với tỷ giá 1 USD = 27.408,6 VND) và hỗ trợ nhiều hình thức thanh toán: thẻ Visa/Mastercard/AMEX, Stripe Link và các phương thức nội địa.
- **Webhook callback đảm bảo tính nhất quán:** Sau khi Mentee thanh toán thành công, Stripe gửi webhook `checkout.session.completed` về backend.

Backend cập nhật trạng thái hóa đơn từ *unpaid* sang *paid*, lưu các trường *stripe_payment_intent_id*, *stripe_charge_id*, *stripe_balance_transaction_id* đã mô tả trong EERD, đồng thời tự động sinh các bản ghi buổi tư vấn (*meetings*) tương ứng với loại gói mentoring và chuyển trạng thái slot sang *booked*.

5.2.3 Dashboard quản lý của Mentee



Hình 36: Giao diện Dashboard quản lý phiên mentoring của Mentee

Sau khi đặt lịch và thanh toán thành công, Mentee có thể truy cập *Mentee Dashboard* để theo dõi và quản lý toàn bộ các buổi tư vấn của mình. Giao diện được chia thành hai khu vực chính:

- **Danh sách buổi tư vấn (cột trái):** Các buổi được phân loại theo trạng thái (*Accepted*, *Pending*, *Completed*, *Cancelled*), với badge số lượng tương ứng cho từng nhóm. Mỗi card hiển thị thông tin Mentor (ảnh, tên, gói đã đăng ký, đơn giá), tiêu đề buổi, ngày-giờ, tổng phí và trạng thái xác nhận (*Confirmed* dạng badge xanh). Ba nút thao tác tương ứng với các luồng nghiệp vụ chính:
 - *Join Meeting*: điều hướng tới link phòng họp đã được lưu trong trường *meeting_url* của thực thể buổi tư vấn.
 - *Contact mentor*: mở giao diện chat trực tiếp với Mentor (tính năng này

nhóm đang phát triển).

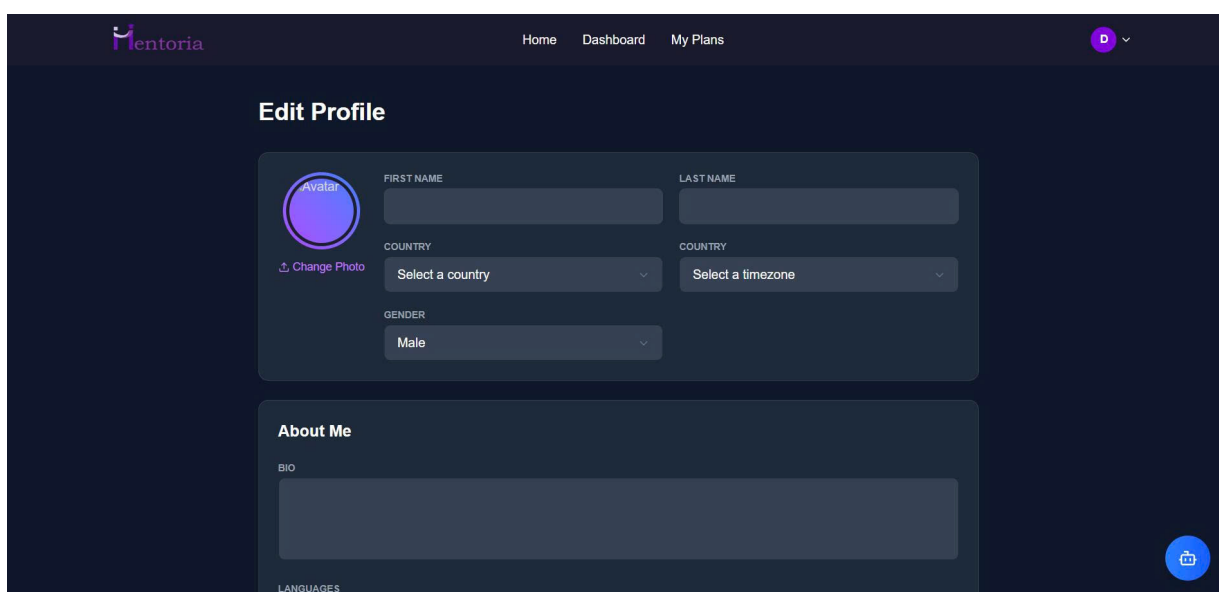
- *Book another slot*: điều hướng nhanh về trang hồ sơ Mentor để đặt thêm buổi tư vấn.

- **Calendar view (cột phải)**: Hiển thị tháng hiện tại với các ngày có buổi tư vấn được đánh dấu nổi bật (ngày 30/04 trong ví dụ). Cách trình bày này giúp Mentee nhanh chóng hình dung lịch học của mình, đặc biệt hữu ích khi đăng ký nhiều gói cùng lúc.

5.3 Giao diện dành cho Mentor

Mentor là nhóm người dùng cung cấp dịch vụ tư vấn chuyên môn cho Mentee. So với Mentee, vai trò Mentor có yêu cầu vận hành phức tạp hơn đáng kể: phải duy trì hồ sơ chuyên nghiệp đầy đủ, thiết lập các gói mentoring cùng mức phí, quản lý lịch khả dụng và xử lý các buổi tư vấn đã được đặt. Phần này trình bày ba giao diện then chốt hỗ trợ Mentor vận hành dịch vụ của mình: trang chỉnh sửa hồ sơ, trang quản lý gói mentoring và dashboard điều phối lịch hẹn.

5.3.1 Trang chỉnh sửa hồ sơ Mentor

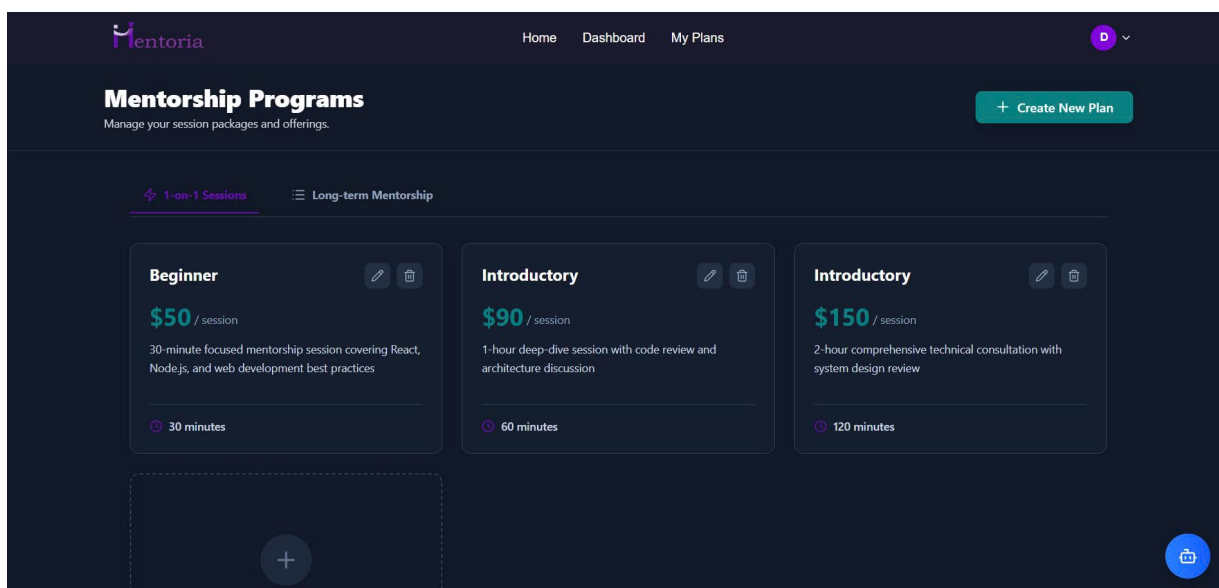


Hình 37: Giao diện chỉnh sửa hồ sơ chuyên môn của Mentor

Trang *Edit Profile* là điểm khởi đầu bắt buộc đối với mọi Mentor sau khi hồ sơ ứng tuyển được Admin phê duyệt. Theo ràng buộc ngữ nghĩa đã trình bày ở mục 3.4.3 (“*Mỗi mentor phải có ít nhất một lĩnh vực chuyên môn trước khi được phép mở buổi tư vấn*”), Mentor không thể tạo gói dịch vụ hoặc nhận booking cho đến khi hoàn tất các trường thông tin tối thiểu. Giao diện được tổ chức theo cụm thông tin nhằm giảm tải nhận thức cho người dùng:

- **Cụm thông tin định danh:** Bao gồm ảnh đại diện (*Change Photo*), họ và tên, quốc gia, múi giờ và giới tính. Trường *Country* và *Timezone* đặc biệt quan trọng đối với hệ thống mentoring quốc tế: do Mentor và Mentee có thể ở các múi giờ khác nhau, hệ thống phải lưu *timezone* ở dạng IANA (ví dụ *America/New_York*, *Asia/Ho_Chi_Minh*) thay vì offset cố định, nhằm xử lý chính xác các trường hợp daylight saving time. Khi hiển thị slot lịch cho Mentee, backend sẽ tự động chuyển đổi sang timezone của người xem.
- **Cụm About Me:** Gồm *Bio* (tiểu sử cá nhân), *Languages* (ngôn ngữ sử dụng) – đây là các trường tự do văn bản giúp Mentor giới thiệu bản thân.
- **Cụm thông tin chuyên môn (phần dưới của trang, không hiển thị trong ảnh):** Gồm Skills, Companies Job Titles,... Đây là các thực thể có quan hệ N-N với Mentor, được lưu trữ trong các bảng trung gian như *mentor_skills*, *work_for*. Khi Mentor cập nhật, hệ thống thực hiện thao tác *diff-based update*: so sánh tập kỹ năng cũ và mới, chỉ INSERT các quan hệ mới và DELETE các quan hệ đã bị bỏ, thay vì xóa toàn bộ rồi chèn lại – cách tiếp cận này giảm áp lực ghi và tránh kích hoạt cascade trigger không cần thiết.

5.3.2 Trang quản lý gói mentoring (My Plans)

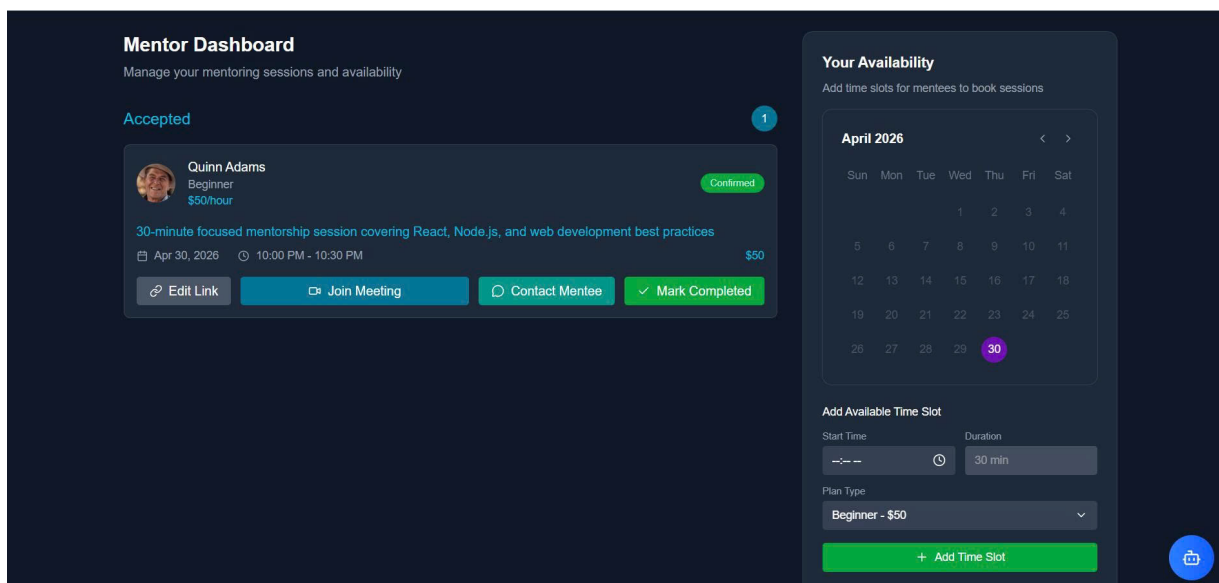


Hình 38: Giao diện quản lý các gói mentoring

Trang *Mentorship Programs* là nơi Mentor định nghĩa các sản phẩm dịch vụ mà mentor cung cấp ra thị trường. Theo thiết kế EERD ở mục 3.4.2, hệ thống phân chia gói mentoring thành hai loại chính, được phản ánh trực tiếp qua hai tab điều hướng trên giao diện:

- **Tab *1-on-1 Sessions* (gói theo phiên):** Tập trung vào các buổi gặp gỡ tư vấn đơn lẻ, với thuộc tính đặc trưng là *thời lượng* của phiên. Trong ví dụ ở Hình 38, Mentor đang cung cấp ba mức độ: *Beginner* (30 phút, \$50), *Introductory* (60 phút, \$90) và *Introductory mở rộng* (120 phút, \$150). Mỗi card hiển thị tên gói, đơn giá, mô tả chi tiết và thời lượng, kèm hai nút thao tác sửa và xóa. Card cuối cùng (dấu cộng) là placeholder để tạo gói mới.
- **Tab *Long-term Mentorship* (gói theo tháng):** Cung cấp sự đồng hành dài hạn với các thuộc tính đặc thù như số cuộc gọi mỗi tháng, thời gian mỗi cuộc gọi và các tiện ích đi kèm (Q&A không giới hạn, hỗ trợ thực hành, code review).

5.3.3 Dashboard điều phối của Mentor



Hình 39: Giao diện Dashboard quản lý phiên và lịch khả dụng của Mentor

Mentor Dashboard là giao diện vận hành chính, kết hợp hai luồng nghiệp vụ song song trong cùng một khung nhìn: theo dõi các buổi tư vấn đã được Mentee đặt và quản lý lịch khả dụng để mở thêm slot mới.

Khu vực quản lý buổi tư vấn (cột trái): Hiển thị danh sách các buổi đã được đặt, phân loại theo trạng thái với badge số lượng. So với dashboard của Mentee, dashboard Mentor có thêm hai thao tác đặc trưng:

- *Edit Link*: Cho phép Mentor cập nhật link phòng họp (Google Meet, Zoom) trước khi buổi tư vấn diễn ra. Thiết kế này linh hoạt hơn việc hệ thống tự sinh link tích hợp, vì cho phép Mentor sử dụng các nền tảng họp mà họ đã quen thuộc.
- *Mark Completed*: Chuyển trạng thái buổi tư vấn sang *Completed*. Đây là một thao tác có ý nghĩa nghiệp vụ rất quan trọng vì nó kích hoạt một chuỗi side-effect được xử lý qua trigger trên SQL Server (đã đề cập ở mục 3.4.1):
 - Cập nhật thuộc tính dẫn xuất `total_mentee` của Mentor (đếm số buổi ở trạng thái Completed).
 - Mở khóa quyền đánh giá cho Mentee, theo ràng buộc ngữ nghĩa đã trình

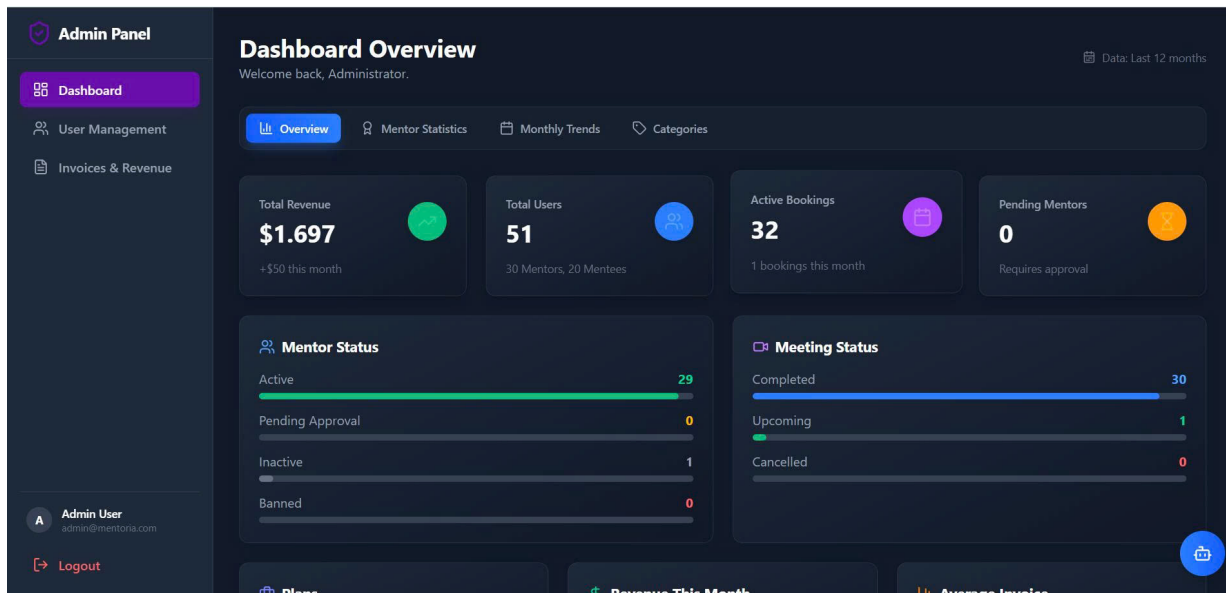
bày ở mục 3.4.3: “Chỉ những Mentee đã tham gia và hoàn thành ít nhất một buổi tư vấn của Mentor mới có thể gửi đánh giá”.

Khu vực Your Availability (cột phải): Cho phép Mentor mở thêm slot khả dụng thông qua form đơn giản gồm *Start Time*, *Duration* và *Plan Type*. Khi nhấn *Add Time Slot*, backend thực thi stored procedure kiểm tra xung đột thời gian (đã đề cập ở mục 3.4.1) trước khi tạo bản ghi mới, nhằm đảm bảo ràng buộc duy nhất (*mentor_id*, *date*, *start_time*, *end_time*) đã thiết kế trong EERD. Ngoài ra, slot mới được tạo bắt buộc phải gắn với một *Plan Type* cụ thể – đây chính là quan hệ “một phiên làm việc chỉ thuộc về duy nhất một gói mentoring” đã mô tả ở mục 3.4.2, giúp tránh xung đột khi cùng một slot có thể được book bởi nhiều loại gói khác nhau.

5.4 Giao diện dành cho Admin

Admin là vai trò có quyền hạn cao nhất trong hệ thống, chịu trách nhiệm vận hành toàn diện nền tảng Mentoria. Khác với ba nhóm người dùng end-user (Guest, Mentee, Mentor) sử dụng giao diện *customer-facing*, Admin được cung cấp một *Admin Panel* riêng biệt với layout sidebar đặc trưng, tập trung vào ba nhóm chức năng quản trị: giám sát số liệu vận hành (*Dashboard*), quản lý người dùng (*User Management*) và theo dõi tài chính (*Invoices & Revenue*).

5.4.1 Dashboard Overview



Hình 40: Giao diện tổng quan hệ thống

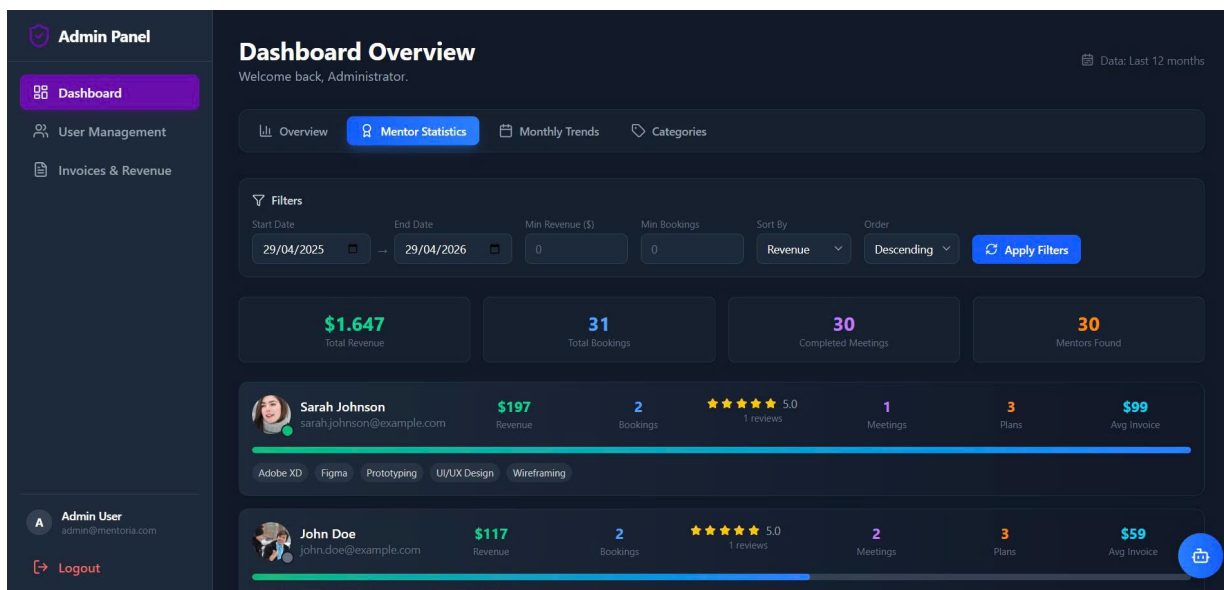
Dashboard Overview là màn hình mặc định khi Admin đăng nhập, cung cấp một cái nhìn tổng quát về tình trạng vận hành nền tảng trong 12 tháng gần nhất. Giao diện được tổ chức thành ba lớp thông tin theo nguyên tắc *visual hierarchy*:

- **Lớp KPI cấp cao (4 thẻ trên cùng):** Tổng doanh thu (\$1.697), tổng người dùng ($51 = 30 \text{ Mentors} + 20 \text{ Mentees}$), tổng buổi tư vấn đang hoạt động (32) và số Mentor đang chờ phê duyệt (0). Đây là các chỉ số mà Admin cần nắm trong vài giây đầu tiên.
- **Lớp phân tích trạng thái:** Hai khu vực *Mentor Status* và *Meeting Status* hiển thị phân bố theo từng trạng thái nội bộ. Việc trực quan hóa phân bố trạng thái dưới dạng progress bar giúp Admin nhanh chóng phát hiện bất thường, ví dụ tỉ lệ *Banned* tăng đột ngột có thể là dấu hiệu của lạm dụng hệ thống.
- **Lớp số liệu chi tiết (phần dưới, không hiển thị trong ảnh):** Bao gồm số gói mentoring đang hoạt động, doanh thu tháng và đơn giá trung bình.

Toàn bộ ba tab phân tích của Dashboard Overview (*Mentor Statistics*, *Monthly Trends*, *Categories*) đều được thực thi thông qua stored procedure

`dbo.sp_DashboardStatistics` với tham số `@GroupBy` nhận một trong ba giá trị: `'mentor'`, `'month'`, `'category'`. Cụ thể, thủ tục này sử dụng đầy đủ các aggregate function phổ biến nhất của SQL: `COUNT`, `COUNT(DISTINCT ...)`, `SUM`, `AVG`, kết hợp với mệnh đề `GROUP BY` (theo `mentor_id`, theo cặp (`YEAR`, `MONTH`), hoặc theo `category_id`) và `HAVING` để lọc kết quả sau khi gộp.

5.4.2 Mentor Statistics

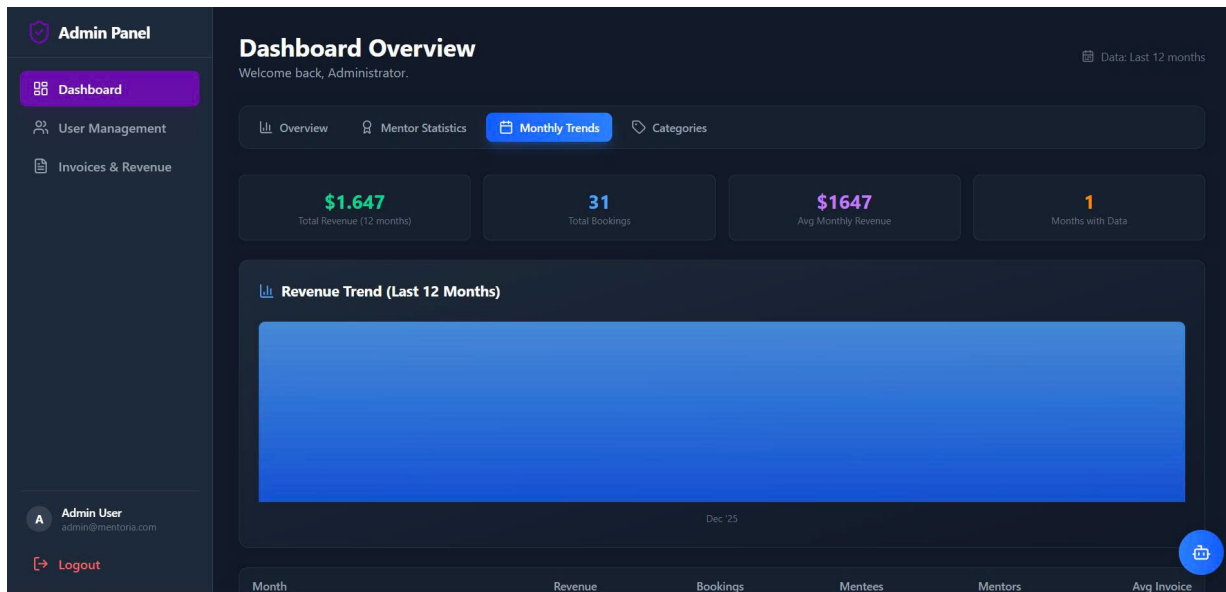


Hình 41: Giao diện thống kê hiệu suất Mentor

Tab *Mentor Statistics* cho phép Admin đánh giá hiệu suất từng Mentor dựa trên các chỉ số kinh doanh: doanh thu, số booking, đánh giá trung bình, số buổi đã hoàn thành, số gói đang chào bán và đơn giá trung bình. Giao diện được trang bị bộ lọc đa chiều (*Filters*) cho phép Admin truy vấn linh hoạt theo:

- **Khoảng thời gian:** *Start Date* và *End Date* – ánh xạ tới điều kiện `WHERE invoice.paid_at BETWEEN @start AND @end` trong stored procedure.
- **Ngưỡng tối thiểu:** *Min Revenue* và *Min Bookings* – ánh xạ tới điều kiện `HAVING` sau khi `GROUP BY` theo mentor.
- **Sắp xếp:** *Sort By* (Revenue, Bookings, Rating) và *Order* (ASC/DESC).

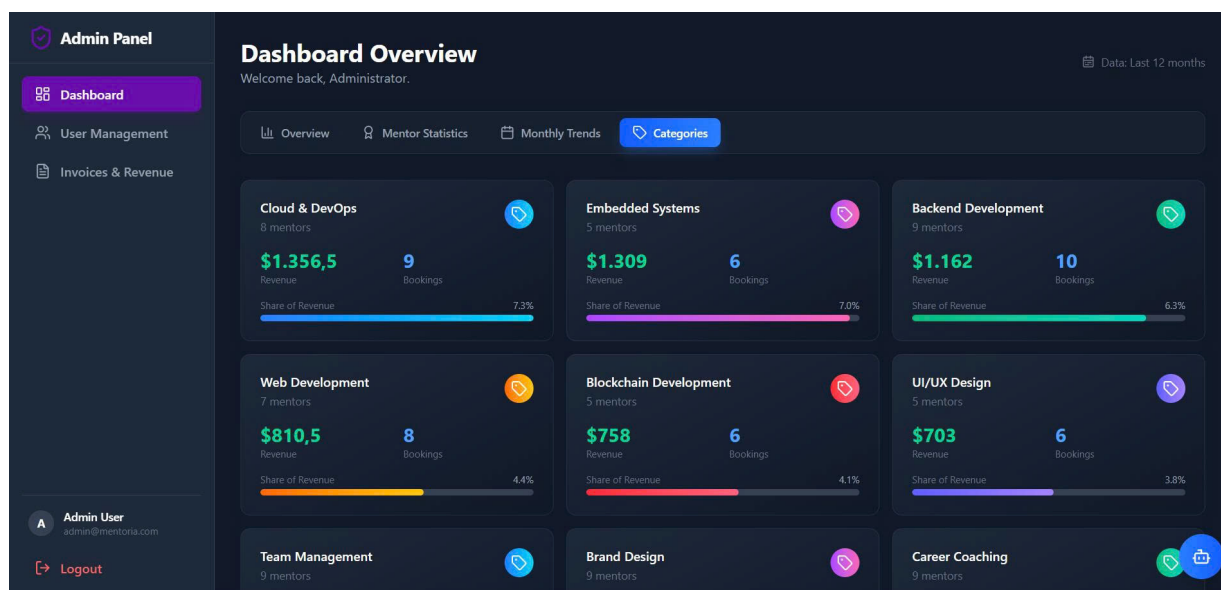
5.4.3 Monthly Trends



Hình 42: Giao diện theo dõi xu hướng theo tháng

Tab *Monthly Trends* cung cấp góc nhìn theo thực thời gian, giúp Admin nhận diện xu hướng tăng trưởng hoặc suy giảm của nền tảng. Bốn KPI chính được hiển thị: tổng doanh thu, tổng booking, doanh thu trung bình mỗi tháng và số tháng có dữ liệu. Phía dưới là biểu đồ *Revenue Trend* dạng bar chart trải dài 12 tháng gần nhất, kèm bảng chi tiết phân rã theo tháng (cột *Month*, *Revenue*, *Bookings*, *Mentees*, *Mentors*, *Avg Invoice*).

5.4.4 Categories



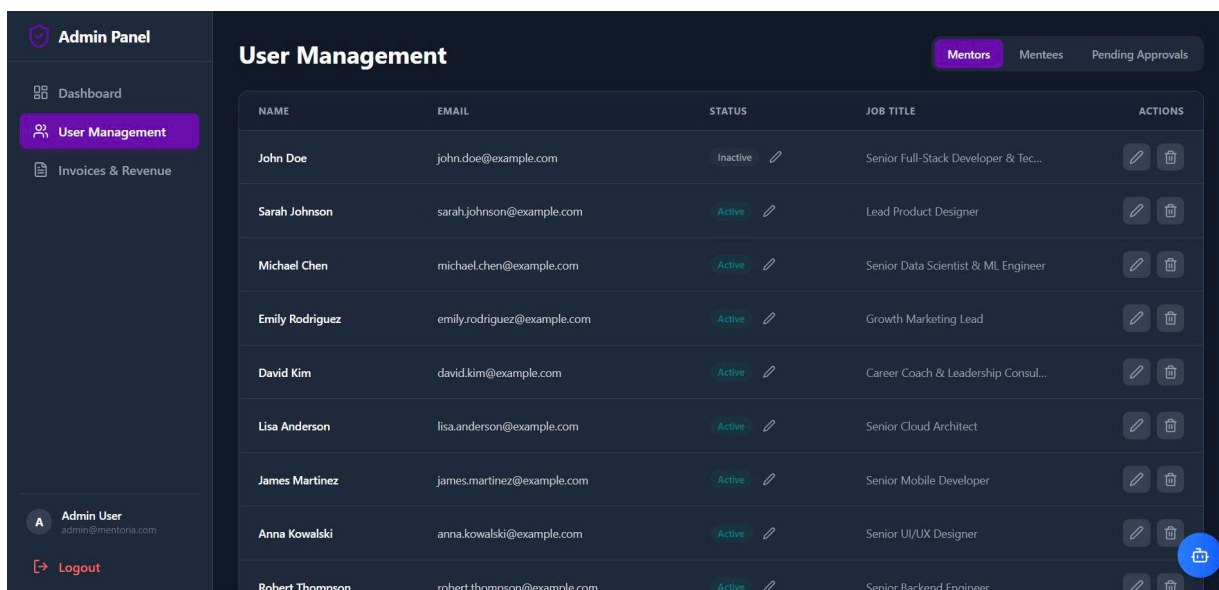
Hình 43: Giao diện thống kê theo lĩnh vực chuyên môn

Tab *Categories* trực quan hóa phân bố doanh thu và booking theo lĩnh vực chuyên môn (*Cloud & DevOps*, *Embedded Systems*, *Backend Development*, *Web Development*, *Blockchain Development*, *UI/UX Design*, ...). Mỗi card hiển thị: tên lĩnh vực, số mentor đang hoạt động, doanh thu, số booking và *Share of Revenue* dưới dạng progress bar có màu sắc riêng.

Giao diện này phục vụ hai mục đích quản trị quan trọng:

- **Phân tích chiến lược:** Admin có thể nhận diện các lĩnh vực đang hot (*Cloud & DevOps* chiếm 7.3% doanh thu) để tập trung tuyển thêm mentor, hoặc các lĩnh vực kém hiệu suất (*UI/UX Design* 3.8%) để xem xét chiến lược marketing.
- **Cân bằng cung-cầu:** Số lượng mentor mỗi lĩnh vực cũng được hiển thị, giúp Admin đánh giá xem nguồn cung mentor có đáp ứng đủ nhu cầu mentee hay không.

5.4.5 User Management



Hình 44: Giao diện quản lý người dùng

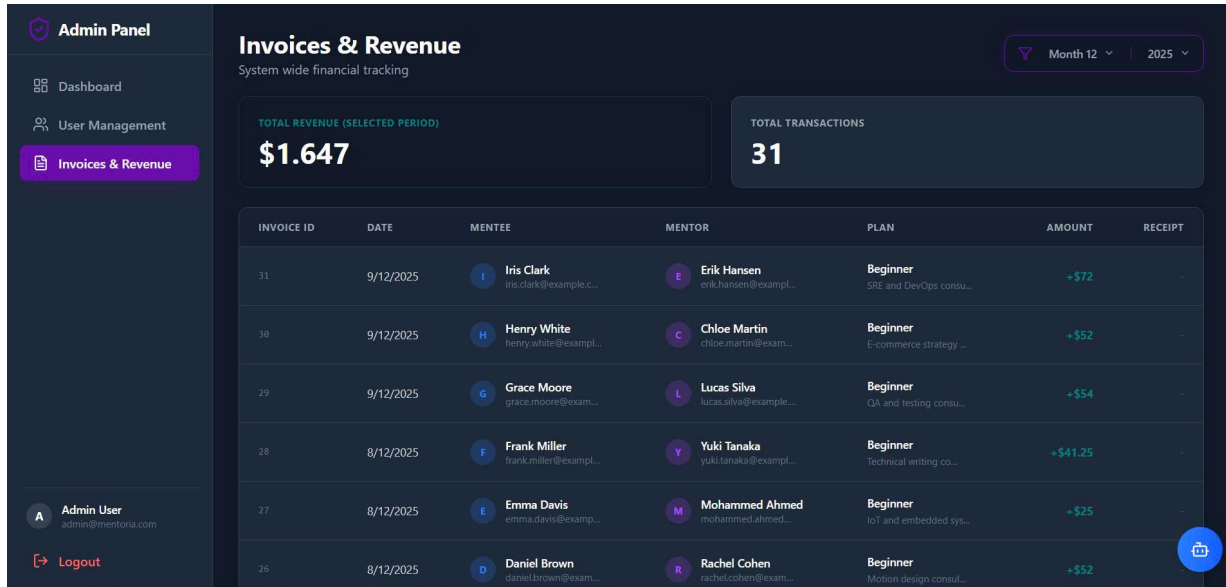
Trang *User Management* là nơi Admin thực thi quyền quản lý vòng đời tài khoản. Giao diện được phân chia thành ba tab tương ứng với ba nhóm người dùng cần xử lý khác nhau:

- **Tab *Mentors*:** Liệt kê toàn bộ Mentor đã được phê duyệt, kèm trạng thái (*Active/Inactive*) và chức danh chuyên môn. Admin có thể chỉnh sửa thông tin (icon bút chì) hoặc xóa tài khoản (icon thùng rác).
- **Tab *Mentees*:** Tương tự nhưng dành cho mentee.
- **Tab *Pending Approvals*:** Danh sách Mentor đang chờ phê duyệt – đây là cầu nối quan trọng với luồng đăng ký Mentor đã trình bày ở mục 5.1. Khi một Guest nộp hồ sơ ứng tuyển Mentor (kèm CV), bản ghi được tạo với **status** = 'pending' và xuất hiện ở tab này. Admin xem xét CV cùng các thông tin khai báo, sau đó nhấn *Approve* để chuyển trạng thái sang **active**, hoặc *Reject* kèm lý do.

Bên cạnh các thao tác xem và chỉnh sửa, *User Management* còn là nơi duy nhất trong toàn bộ hệ thống thực thi thao tác **DELETE** một cách trực tiếp trên dữ liệu nghiệp vụ. Khi Admin nhấn nút xóa (icon thùng rác) bên cạnh một tài khoản, backend

gọi tới hàm xử lý tại `admin.service.ts:193`, trong đó câu lệnh `DELETE FROM ...` được thực thi để xóa các thông tin liên quan tới người dùng đó.

5.4.6 Invoices & Revenue



Hình 45: Giao diện theo dõi hóa đơn và doanh thu

Trang *Invoices & Revenue* là module tài chính của Admin Panel, theo dõi toàn bộ giao dịch đã hoàn tất trên nền tảng. Giao diện gồm hai phần:

- **Card tổng quan:** *Total Revenue* và *Total Transactions* của khoảng thời gian đã chọn (lọc theo *Month* và *Year* ở góc trên phải).
- **Bảng chi tiết hóa đơn:** Mỗi dòng tương ứng với một bản ghi trong bảng *invoices*, hiển thị các trường: *Invoice ID*, *Date*, thông tin Mentee, thông tin Mentor, gói đã đăng ký (*Plan*), số tiền (*Amount*) và *Receipt*.

Cột *Receipt* liên kết trực tiếp tới đường dẫn biên lai do Stripe sinh ra (trường `stripe_receipt_url` đã đề cập ở mục 3.4.2). Thay vì tự sinh PDF biên lai, hệ thống tận dụng infrastructure của Stripe để giảm độ phức tạp. Mỗi biên lai do Stripe phát hành có URL signed có thời hạn, đảm bảo chỉ Admin có quyền truy cập mới xem được.

6 Kết luận

Trong khuôn khổ bài tập lớn môn Hệ Quản trị Cơ sở dữ liệu, nhóm đã nghiên cứu, phân tích và hiện thực hệ thống Mentoria – nền tảng kết nối mentor và mentee hỗ trợ tìm kiếm, đặt lịch và quản lý các phiên mentoring trực tuyến. Thông qua quá trình xây dựng hệ thống, nhóm đã áp dụng đầy đủ các kiến thức về phân tích yêu cầu, thiết kế cơ sở dữ liệu, xây dựng lược đồ quan hệ, hiện thực truy vấn và phát triển ứng dụng web theo mô hình Client-Server ba lớp.

Bên cạnh việc phát triển ứng dụng thực tế, trọng tâm chính của báo cáo là nghiên cứu và so sánh hai hệ quản trị cơ sở dữ liệu phổ biến hiện nay: MS SQL Server và Apache Cassandra. Nhóm đã tiến hành đánh giá trên bốn khía cạnh quan trọng gồm: Data Storage & Management, Indexing, Query Processing và Data Backup & Recovery thông qua cả lý thuyết lẫn các kịch bản thực nghiệm đo đặc hiệu năng thực tế.

Kết quả thực nghiệm cho thấy mỗi hệ quản trị đều có những ưu điểm và hạn chế riêng phù hợp với các bài toán khác nhau. Apache Cassandra nổi bật ở khả năng mở rộng ngang, hiệu suất ghi lớn và tính sẵn sàng cao trong môi trường phân tán. Tuy nhiên, Cassandra tồn tại nhiều hạn chế trong việc xử lý truy vấn phức tạp, hỗ trợ transaction và đảm bảo tính nhất quán mạnh của dữ liệu. Ngược lại, MS SQL Server cung cấp hệ sinh thái hoàn chỉnh cho các hệ thống nghiệp vụ với khả năng hỗ trợ JOIN, transaction ACID, tối ưu hóa truy vấn, indexing và backup/recovery mạnh mẽ, giúp việc phát triển và bảo trì hệ thống trở nên dễ dàng và ổn định hơn.

Dựa trên đặc thù nghiệp vụ của hệ thống Mentoria – nơi dữ liệu có mối quan hệ chặt chẽ, yêu cầu tính toàn vẹn cao, nhiều truy vấn tổng hợp và cần đảm bảo tính nhất quán trong các nghiệp vụ như đặt lịch, thanh toán và đánh giá – nhóm quyết định lựa chọn MS SQL Server làm hệ quản trị cơ sở dữ liệu chính cho hệ thống. Đây được xem là lựa chọn phù hợp hơn so với Cassandra trong bối cảnh hiện tại của ứng dụng.

Thông qua bài tập lớn này, nhóm không chỉ củng cố kiến thức lý thuyết về cơ sở dữ liệu mà còn có cơ hội tiếp cận quy trình phát triển một hệ thống phần mềm thực tế, từ khâu phân tích yêu cầu đến triển khai và đánh giá hiệu năng hệ quản trị cơ sở dữ liệu. Đồng thời, nhóm cũng hiểu rõ hơn về các đánh đổi trong thiết kế hệ thống dữ

liệu hiện đại, đặc biệt là sự khác biệt giữa mô hình cơ sở dữ liệu quan hệ và NoSQL.

Trong tương lai, hệ thống Mentoria có thể tiếp tục được mở rộng với các tính năng như tích hợp AI hỗ trợ gợi ý mentor, hệ thống realtime chat/video call, phân tích dữ liệu hành vi người dùng hoặc triển khai kiến trúc microservices nhằm nâng cao khả năng mở rộng và trải nghiệm người dùng. Đây cũng là hướng phát triển giúp nhóm tiếp tục nghiên cứu sâu hơn về các hệ quản trị cơ sở dữ liệu và kiến trúc hệ thống phân tán trong thực tế.

Tài liệu tham khảo

- [1] Cassandra documentation. Apache Cassandra. [Online]. Available: <https://cassandra.apache.org/doc/latest/>
- [2] Sql server technical documentation. Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/vi-vn/sql/sql-server/?view=sql-server-ver16>
- [3] R. Elmasri and S. B. Navathe, “Fundamentals of database systems seventh edition,” 2016.